

Universidad Nacional de Colombia

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas e Industrial

**Diseño de un modelo de evaluación de controles de  
seguridad para arquitecturas de identidad federada  
basadas en OAuth 2.0 y OpenID Connect**

Trabajo de

tesis

Autor:

Fredy Andres Rosero Cristancho

Versión 20

Bogotá D.C., Colombia

2026

# Tabla de contenido

## Diseño de un modelo de evaluación de controles de seguridad para arquitecturas de identidad federada basadas en OAuth 2.0 y OpenID Connect

|                                                                                                                              |    |
|------------------------------------------------------------------------------------------------------------------------------|----|
| 1. Introducción . . . . .                                                                                                    | 2  |
| 1.1. Contexto . . . . .                                                                                                      | 2  |
| 1.2. Planteamiento del problema . . . . .                                                                                    | 3  |
| 1.3. Pregunta de investigación . . . . .                                                                                     | 3  |
| 1.4. Justificación . . . . .                                                                                                 | 3  |
| 1.5. Objetivo general . . . . .                                                                                              | 4  |
| 1.6. Objetivos específicos . . . . .                                                                                         | 4  |
| 1.7. Alcance y limitaciones . . . . .                                                                                        | 4  |
| 1.8. Modelo general de evaluación . . . . .                                                                                  | 5  |
| 1.9. Estructura del documento . . . . .                                                                                      | 7  |
| 2. Marco conceptual . . . . .                                                                                                | 8  |
| 2.1. Identidad digital . . . . .                                                                                             | 8  |
| 2.2. Autenticación, autorización y federación . . . . .                                                                      | 8  |
| 2.3. OAuth 2.0 . . . . .                                                                                                     | 10 |
| 2.4. OpenID Connect . . . . .                                                                                                | 10 |
| 2.5. Roles de OAuth 2.0 . . . . .                                                                                            | 12 |
| 2.6. Tokens en OAuth 2.0 y OIDC . . . . .                                                                                    | 13 |
| 2.7. Token opaco y JWT embebido . . . . .                                                                                    | 15 |
| 2.8. JWT, JWS y JWE . . . . .                                                                                                | 16 |
| 2.9. Authorization Code Flow con PKCE . . . . .                                                                              | 17 |
| 2.10. SPA estática, BFF, cookies y tokens . . . . .                                                                          | 21 |
| 2.11. Single Sign-On . . . . .                                                                                               | 22 |
| 2.12. Flujos relevantes para la tesis . . . . .                                                                              | 23 |
| 2.13. Relación del marco conceptual con los casos de evaluación . . . . .                                                    | 24 |
| 2.14. Riesgos comunes en arquitecturas OAuth 2.0/OIDC . . . . .                                                              | 25 |
| 2.15. Fuentes técnicas base del marco conceptual . . . . .                                                                   | 26 |
| 3. Marco de aseguramiento, riesgo, control y auditoría . . . . .                                                             | 27 |
| 3.1. Aseguramiento de tecnologías de información . . . . .                                                                   | 27 |
| 3.2. Riesgo de tecnologías de información . . . . .                                                                          | 28 |
| 3.3. Controles de tecnologías de información . . . . .                                                                       | 28 |
| 3.4. Auditoría basada en riesgos . . . . .                                                                                   | 29 |
| 3.5. Relación riesgo-control-evidencia . . . . .                                                                             | 29 |
| 3.6. Uso del enfoque de Valencia Duque en esta tesis . . . . .                                                               | 31 |
| 3.7. Complemento con estándares y fuentes técnicas . . . . .                                                                 | 31 |
| 4. Casos arquitectónicos de contraste y escenarios de evaluación . . . . .                                                   | 32 |
| 4.1. Evolución arquitectónica: de cookies de sesión a OAuth 2.0 con PKCE . . . . .                                           | 32 |
| 4.2. Criterios para seleccionar los casos . . . . .                                                                          | 36 |
| 4.3. Vista general de los casos . . . . .                                                                                    | 38 |
| 4.4. Caso base 0A: monolito con autenticación y sesión local . . . . .                                                       | 39 |
| 4.5. Caso base 0B: SPA estática + BFF con gestión propia de usuarios . . . . .                                               | 42 |
| 4.6. Arquitectura 1: SPA estática + BFF + IdP corporativo usando OAuth 2.0/OIDC . . . . .                                    | 45 |
| 4.7. Arquitectura 2: M2M con Authorization Server actuando también como Resource Server . . . . .                            | 47 |
| 4.8. Arquitectura 3: API Gateway federado con Authorization Server corporativo y propagación/intercambio de tokens . . . . . | 49 |
| 4.9. Comparación sintética de los casos . . . . .                                                                            | 52 |
| 5. Diseño del modelo de evaluación . . . . .                                                                                 | 53 |
| 5.1. Propósito del modelo de evaluación . . . . .                                                                            | 53 |
| 5.2. Fundamento del modelo de evaluación . . . . .                                                                           | 77 |
| 5.3. Criterios de evaluación . . . . .                                                                                       | 82 |
| 5.4. Estructura de la matriz . . . . .                                                                                       | 83 |

|                                                                                                |     |
|------------------------------------------------------------------------------------------------|-----|
| 5.5. Campos de la matriz . . . . .                                                             | 84  |
| 5.6. Plantilla inicial de la matriz . . . . .                                                  | 90  |
| 6. Experimento / Aplicación de la matriz a seis corridas demostrativas . . . . .               | 91  |
| 6.1. Estructura de los datos demostrativos . . . . .                                           | 92  |
| 6.2. Criterio para construir casos buenos y malos . . . . .                                    | 92  |
| 6.3. Escenario demostrativo: SPA estática + BFF + IdP corporativo . . . . .                    | 92  |
| 6.4. Escenario demostrativo: M2M con Authorization Server como Resource Server . . . . .       | 95  |
| 6.5. Escenario demostrativo: API Gateway federado + Authorization Server corporativo . . . . . | 96  |
| 6.6. Comparación transversal de las seis corridas . . . . .                                    | 97  |
| 6.7. Resultados por riesgo en cada corrida . . . . .                                           | 98  |
| 6.8. Relación entre el capítulo 6 y el formulario web . . . . .                                | 99  |
| 6.9. Síntesis del experimento . . . . .                                                        | 100 |
| 7. Análisis de resultados . . . . .                                                            | 101 |
| 7.1. Riesgos más relevantes identificados . . . . .                                            | 101 |
| 7.2. Controles críticos por escenario . . . . .                                                | 101 |
| 7.3. Evidencias auditable más importantes . . . . .                                            | 102 |
| 7.4. Diferencias entre autenticación, autorización y federación desde el riesgo . . . . .      | 103 |
| 7.5. Hallazgos sobre el uso seguro de OAuth 2.0/OIDC . . . . .                                 | 103 |
| 7.6. Valor de la matriz como artefacto de evaluación . . . . .                                 | 103 |
| 8. Conclusiones y trabajo futuro . . . . .                                                     | 104 |
| 8.1. Conclusiones frente al objetivo general . . . . .                                         | 104 |
| 8.2. Conclusiones frente a los objetivos específicos . . . . .                                 | 104 |
| 8.3. Aporte de la tesis . . . . .                                                              | 105 |
| 8.4. Limitaciones del trabajo . . . . .                                                        | 105 |
| 8.5. Trabajo futuro . . . . .                                                                  | 106 |
| 9. Referencias . . . . .                                                                       | 106 |
| 10. Anexos . . . . .                                                                           | 107 |
| 10.1. Artefactos de datos del modelo . . . . .                                                 | 107 |
| 10.2. Diagramas de escenarios OAuth 2.0/OIDC . . . . .                                         | 108 |
| 10.3. Diagramas de casos base (contraste conceptual) . . . . .                                 | 108 |
| 10.4. Referencia del entregable web . . . . .                                                  | 108 |

## Diseño de un modelo de evaluación de controles de seguridad para arquitecturas de identidad federada basadas en OAuth 2.0 y OpenID Connect

### 1. Introducción

#### 1.1. Contexto

En el diseño de soluciones seguras para la comunicación entre aplicaciones, APIs y servicios distribuidos, las arquitecturas modernas dependen cada vez más de mecanismos de identidad federada. En este contexto, **OAuth 2.0 y OpenID Connect** se han convertido en tecnologías ampliamente utilizadas para delegar autorización, autenticar usuarios y proteger recursos expuestos mediante APIs.

Sin embargo, ningún **sistema es 100 % seguro**. El uso de OAuth 2.0 y OpenID Connect (en adelante, OIDC) no garantiza por sí mismo una arquitectura segura. La seguridad depende de cómo se implementan los flujos, cómo se emiten y validan los tokens, cómo se gestionan los clientes, cómo se aplican los controles de acceso y cómo se evidencia el cumplimiento de dichos controles.

Desde una perspectiva de **aseguramiento** de tecnologías de información, la evaluación de estas arquitecturas debe considerar la relación entre riesgos, controles y auditoría. Esto permite pasar de una revisión solamente técnica a una evaluación más estructurada sobre qué puede fallar, qué controles deben existir, cómo están diseñados y qué evidencias permiten verificar su operación.

En este sentido, el enfoque de Valencia Duque resulta pertinente para esta tesis, ya que propone evaluar los controles

no solo desde su existencia formal, sino desde su capacidad para mitigar riesgos. En particular, el autor plantea que la efectividad del control puede analizarse a partir de dos variables: la **eficiencia del control**, relacionada con su diseño, y la **eficacia del control**, relacionada con el cumplimiento real del objetivo para el cual fue definido.

Aplicado al caso de OAuth 2.0 y OpenID Connect, esto implica que no basta con afirmar que existe un control, por ejemplo, validación de tokens, uso de scopes o gestión de secretos. También es necesario analizar si dicho control está adecuadamente diseñado para el riesgo que pretende mitigar y si realmente opera de forma verificable en la arquitectura evaluada.

## 1.2. Planteamiento del problema

En muchas organizaciones, OAuth 2.0 y OpenID Connect son adoptados como mecanismos estándar para autenticación, autorización y protección de APIs. No obstante, la presencia de estos protocolos en una solución no implica necesariamente que los riesgos estén correctamente:

1. identificados,
2. controlados,
3. evaluados y auditados.

Una arquitectura puede utilizar OAuth 2.0 u OIDC y aun así presentar debilidades como exposición de tokens, validación incompleta de claims, uso excesivo de privilegios, gestión deficiente de secretos, clientes mal configurados, tokens con vigencia inadecuada o falta de trazabilidad sobre las operaciones realizadas.

El problema no consiste únicamente en determinar si una solución utiliza OAuth 2.0 u OIDC, sino en evaluar si su implementación cuenta con controles adecuados frente a los riesgos del escenario **arquitectónico** en el que se aplica.

Adicionalmente, en la práctica puede existir una diferencia entre el control declarado y el control realmente implementado. Por ejemplo, un equipo puede afirmar que una API valida correctamente los tokens, pero dicha afirmación no necesariamente demuestra que se validen elementos como firma, expiración, issuer, audience, scopes o permisos de acuerdo con el riesgo del recurso protegido.

Por tanto, se requiere una forma estructurada de analizar arquitecturas basadas en OAuth 2.0 u OIDC desde la relación entre:

1. riesgos,
2. controles,
3. diseño del control,
4. evidencia de operación,
5. y evidencias auditables.

Esta tesis aborda ese problema mediante un modelo de evaluación, materializado en una matriz, que no solo identifica riesgos y controles, sino que también permite clasificar el estado del control y considerar su eficiencia y eficacia desde un enfoque de aseguramiento orientado a riesgos.

## 1.3. Pregunta de investigación

¿Cómo evaluar el aseguramiento (los riesgos, controles y evidencias auditables) asociados a escenarios representativos de arquitecturas de identidad federada basadas en OAuth 2.0 y OpenID Connect, considerando el diseño y operación de los controles?

## 1.4. Justificación

La identidad digital y la protección de APIs son componentes críticos en arquitecturas de sistemas distribuidos modernas. Una configuración incorrecta de OAuth 2.0 u OIDC puede afectar la **confidencialidad**, **integridad**, **disponibilidad** de los recursos protegidos, pero además la **trazabilidad** de las operaciones realizadas por usuarios, clientes técnicos o servicios.

Aunque ya existen estándares, guías y buenas prácticas sobre OAuth 2.0, OpenID Connect, seguridad de APIs y gestión de riesgos, estos recursos suelen encontrarse dispersos o formulados con distintos niveles de abstracción. Esto dificulta

su aplicación directa y pragmática en una evaluación arquitectónica orientada a riesgos.

Esta tesis busca aportar un modelo de evaluación, materializado en una matriz, que organice riesgos, controles, estados de implementación y evidencias auditables para escenarios representativos de uso de OAuth 2.0 u OIDC. Una parte sustantiva de ese aporte consiste en construir un catálogo estructurado de riesgos globales, riesgos específicos por escenario y controles esperados para las arquitecturas evaluadas. El aporte no consiste en crear un nuevo marco de gestión de riesgos, sino en adaptar un enfoque de aseguramiento orientado a riesgos al análisis de arquitecturas de identidad federada que implementan OAuth 2.0 y OpenID Connect.

El trabajo se apoya conceptualmente en el enfoque de riesgo, control y auditoría presentado por Valencia Duque en el libro *Aseguramiento y auditoría de tecnologías de información orientados a riesgos: un enfoque basado en estándares internacionales*. En particular, se toma como referencia la idea de que la evaluación de controles debe considerar su efectividad, entendida a partir de la eficiencia y la eficacia del control.

Desde esta perspectiva, la matriz propuesta no solo pregunta si un control existe, sino también si está adecuadamente diseñado para el riesgo que pretende mitigar y si puede verificarse mediante evidencia suficiente.

Adicionalmente, la propuesta se condensa en un entregable aplicado que también forma parte del aporte de la tesis: un sitio web publicado en GitHub Pages, concebido como una interfaz de usuario (UI) para la evaluación. Tras seleccionar una de las tres arquitecturas del modelo, la UI carga el catálogo predefinido de riesgos globales, riesgos específicos del escenario y controles esperados. A partir de ese catálogo, el evaluador diligencia los datos de análisis, procesa y califica los controles, y genera un informe en formato Excel. Los algoritmos implementados en JavaScript no se definen de forma arbitraria, sino que derivan de las fórmulas y reglas de valoración que se expongan matemáticamente en esta tesis.

### **1.5. Objetivo general**

Diseñar un modelo de evaluación de controles de seguridad para arquitecturas de identidad federada basadas en OAuth 2.0 y OpenID Connect desde un enfoque de aseguramiento orientado a riesgos.

### **1.6. Objetivos específicos**

1. Caracterizar los conceptos fundamentales de identidad federada, OAuth 2.0, OpenID Connect, tokens, clientes, servidores de autorización y servidores de recursos.
2. Identificar los riesgos de seguridad asociados a identidad federada y los controles técnicos y organizacionales aplicables.
3. Definir criterios de evaluación que permitan analizar la eficiencia del diseño y la eficacia operativa de los controles asociados a cada riesgo bajo ISO/IEC 27002 delimitados por arquitectura.
4. Aplicar el modelo de evaluación sobre las tres arquitecturas con OAuth 2.0/OIDC para demostrar su utilidad como artefacto de análisis de riesgos, controles y evidencias auditables.
5. Condensar el modelo propuesto en un entregable web publicado en GitHub Pages, materializado como un formulario que cargue el catálogo maestro JSON de riesgos, controles, relaciones riesgo-control y referencias técnicas del escenario seleccionado, procese las entradas de evaluación, calcule la calificación de los controles y genere un informe en Excel a partir de algoritmos JavaScript derivados de las fórmulas expuestas en la tesis.

### **1.7. Alcance y limitaciones**

El alcance conceptual de esta tesis incluye dos casos base sin OAuth 2.0/OIDC y tres escenarios representativos donde sí se utilizan OAuth 2.0 y OpenID Connect.

Los dos casos base se usan únicamente como referencia del marco teórico. Su función es explicar el punto de partida arquitectónico antes de introducir identidad federada, Authorization Server, tokens, scopes y clientes OAuth:

1. Caso base 0A: monolito con autenticación y sesión local.
2. Caso base 0B: SPA estática con BFF y gestión local de usuarios.

El modelo de evaluación y su matriz se aplican únicamente a las tres arquitecturas OAuth 2.0/OIDC:

1. Arquitectura 1: SPA estática con BFF e IdP corporativo usando OAuth 2.0/OIDC.
2. Arquitectura 2: comunicación M2M donde el Authorization Server actúa también como Resource Server.
3. Arquitectura 3: API Gateway federado con Authorization Server corporativo, validación de tokens y posible propagación o intercambio de tokens.

La tesis no busca evaluar si un sistema utiliza OAuth 2.0 y OpenID Connect de forma nominal, sino analizar qué riesgos, controles y evidencias auditables deberían considerarse cuando estos protocolos aparecen en un escenario arquitectónico determinado. Los casos base permiten contraste conceptual, pero no hacen parte de la matriz del modelo de evaluación.

La evaluación propuesta no se basa únicamente en entrevistas o declaraciones de los responsables técnicos o funcionales. La matriz diferencia entre controles declarados, controles documentados y controles evidenciados técnicamente. Por tanto, una afirmación como “el sistema valida correctamente los tokens” no se considera evidencia suficiente si no está respaldada por configuración, documentación técnica, logs, pruebas o revisión de código, según corresponda.

La revisión de código puede ser considerada una fuente de evidencia cuando el control evaluado depende de la implementación de la aplicación, por ejemplo, validación de claims, manejo de tokens, uso de librerías de seguridad, almacenamiento de secretos o protección de sesiones. Sin embargo, esta tesis no pretende realizar una auditoría exhaustiva de repositorios ni inspeccionar todos los componentes de una organización específica.

La tesis considera la evaluación de controles desde dos dimensiones: su **eficiencia**, entendida como la adecuación del diseño del control frente al riesgo que busca mitigar; y su **eficacia**, entendida como la evidencia de que el control cumple realmente su objetivo. No obstante, la medición propuesta tiene un alcance académico y no reemplaza una auditoría formal de una entidad.

La tesis no pretende implementar un proveedor de identidad, comparar productos comerciales ni realizar una auditoría real sobre una organización específica.

Como materialización aplicada de la propuesta, sí contempla un entregable web publicado en GitHub Pages, concebido como formulario de evaluación para seleccionar una de las tres arquitecturas del modelo, cargar el catálogo predefinido de riesgos y controles asociados, registrar las entradas de evaluación, procesarlas, calificar los controles y generar un informe en Excel. Este entregable operacionaliza el modelo, pero no reemplaza el juicio profesional del evaluador ni constituye por sí mismo una plataforma integral de auditoría automatizada. Los algoritmos implementados en JavaScript derivan de las fórmulas y reglas de valoración expuestas matemáticamente en la tesis.

Tampoco busca cubrir todos los flujos, extensiones o perfiles existentes de OAuth 2.0 y OpenID Connect. El análisis se concentra en los elementos necesarios para construir una matriz de evaluación aplicable a los escenarios definidos.

### 1.8. Modelo general de evaluación

El modelo de evaluación propuesto en esta tesis está orientado a evaluar controles de seguridad en arquitecturas de identidad federada basadas en OAuth 2.0 y OpenID Connect.

El componente central del modelo es una matriz de evaluación que relaciona riesgos aplicables al escenario, controles esperados, evidencia auditable, score del control, eficacia frente al riesgo, eficiencia frente al riesgo y nivel de exposición observado. El enfoque no se limita a verificar la existencia nominal de OAuth 2.0 u OIDC, sino a evaluar si los riesgos relevantes para el escenario están contemplados y caracterizados, y si los controles asociados son diseñados, implementados y evidenciados de forma suficiente.

Además de su formulación conceptual y matemática, el modelo se condensa en un entregable web publicado en GitHub Pages. Esta materialización integra tres componentes: el catálogo maestro de riesgos y controles, una interfaz de usuario web que guía la evaluación, y algoritmos JavaScript que traducen operativamente las fórmulas y reglas de homologación expuestas en este documento.

La aplicación del modelo se estructura así:

En términos operativos, el entregable web no funciona como un motor abierto para que el usuario invente riesgos desde cero. El evaluador primero selecciona una de las tres arquitecturas del modelo; con esa selección, el formulario

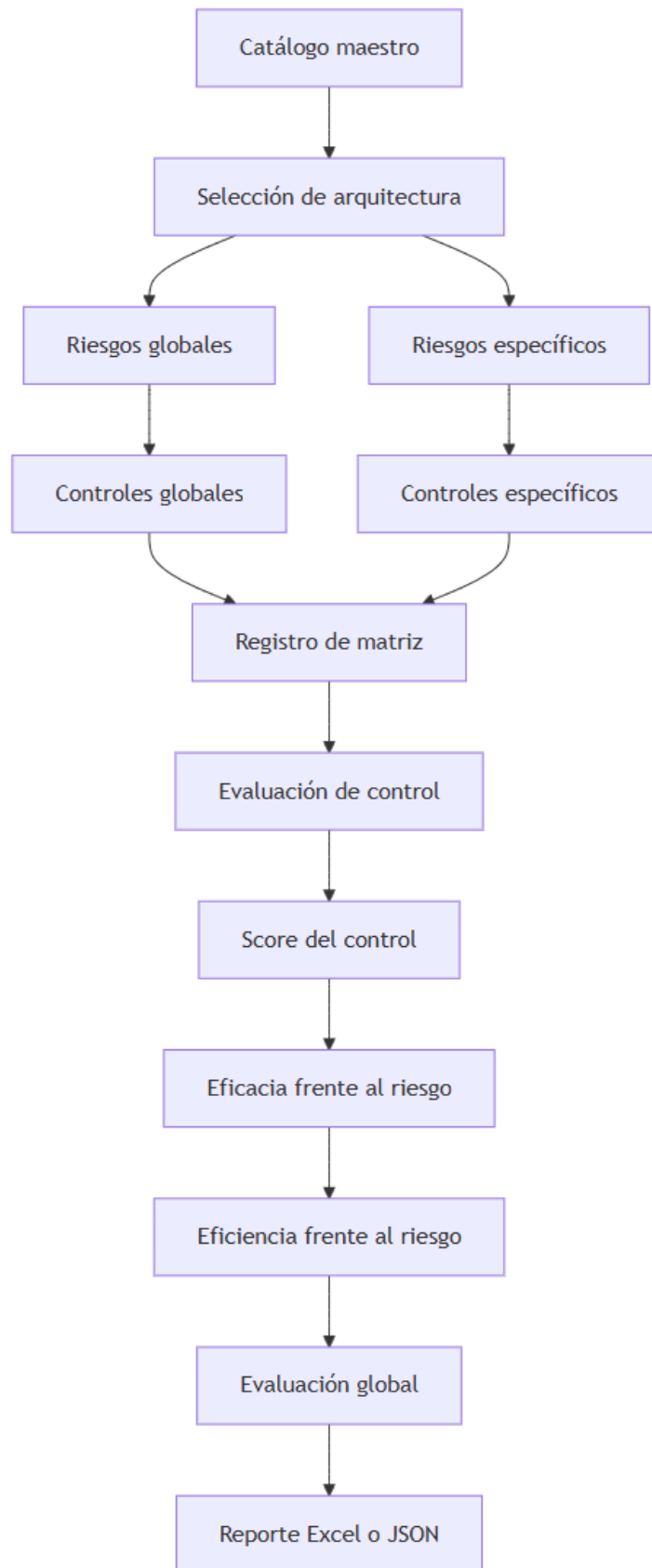


Figura 1: diagram  
6

carga un catálogo predefinido de riesgos globales OAuth 2.0/OIDC, riesgos específicos del escenario y controles esperados. Lo dinámico son los datos diligenciados por el evaluador para cada riesgo y control: activo afectado, amenaza, vulnerabilidad, probabilidad, impacto, costo, evidencia y estado de las dimensiones del control.

La evaluación se apoya en cinco dimensiones del control:

- **Madurez:** diseñada, declarada, implementada o auditada.
- **Automatización:** manual, semiautomática o automática.
- **Momento:** preventivo, detectivo o correctivo.
- **Periodicidad:** ocasional, periódica o permanente.
- **Alcance funcional:** específico o general.

La matriz incorpora tres niveles de cálculo:

1. **Score del control:** calificación del control en escala 0 a 100, calculada con las cinco dimensiones ponderadas.
2. **Eficacia frente al riesgo:** cobertura del riesgo a partir de los controles asociados y sus pesos de mitigación.
3. **Eficiencia frente al riesgo o global:** relación entre beneficio de mitigación estimado y costo asignado.

Esta separación evita confundir la existencia de un control con su aporte real al riesgo. Un control puede estar implementado, pero aportar poco a un riesgo específico; también puede mitigar varios riesgos y, por tanto, su costo debe distribuirse para evitar doble conteo.

Las fuentes de evidencia pueden variar según el tipo de control evaluado. Estas pueden incluir documentación de arquitectura, configuración del proveedor de identidad, configuración del API Gateway, configuración del Resource Server, logs, resultados de pruebas técnicas, pipelines, políticas de seguridad o revisión puntual de código.

Este enfoque permite diferenciar entre la existencia declarada de un control, su diseño esperado, su implementación real, su verificación mediante evidencia auditable y su efecto sobre el nivel de exposición observado.

## 1.9. Estructura del documento

El documento se organiza de la siguiente manera:

El capítulo 1 presenta la introducción, el planteamiento del problema, la pregunta de investigación, la justificación, los objetivos, el alcance y el modelo general de evaluación.

El capítulo 2 desarrolla el marco conceptual sobre identidad federada, OAuth 2.0, OpenID Connect, tokens, clientes, servidores de autorización y APIs protegidas.

El capítulo 3 presenta el marco de aseguramiento orientado a riesgos, con énfasis en la relación entre riesgo, control y auditoría, y en la evaluación de controles a partir de eficiencia, eficacia y efectividad.

El capítulo 4 describe los dos casos base usados como referencia conceptual y los tres escenarios OAuth 2.0/OIDC que serán evaluados: SPA + BFF + IdP, M2M con Authorization Server como Resource Server y API Gateway federado con Authorization Server corporativo.

El capítulo 5 presenta el diseño del modelo de evaluación y de la matriz de riesgos, controles, estados de implementación, eficiencia, eficacia y evidencias auditables.

El capítulo 6 aplica la matriz mediante seis corridas demostrativas: un caso bueno y un caso malo para cada uno de los tres escenarios OAuth 2.0/OIDC definidos. Además, muestra cómo esos valores se serializan en el archivo `demo-prediligenciamiento.json` consumido por el formulario web.

El capítulo 7 presenta las conclusiones, limitaciones del trabajo y posibles líneas de trabajo futuro.

Los anexos se gestionan como archivos enlazados del repositorio y no como contenido embebido en el cuerpo del documento. Incluyen la matriz completa, diagramas, checklist, trazabilidad riesgo-control-evidencia, catálogo maestro JSON, filtros por escenario, mapeo de score, datos de prediligenciamiento, referencia del sitio en GitHub Pages y formato del reporte Excel.



## 2. Marco conceptual

Este capítulo establece el lenguaje técnico mínimo para evaluar arquitecturas de autenticación, autorización e identidad federada desde una perspectiva de riesgos, controles y evidencia auditable.

El propósito del marco conceptual no es describir exhaustivamente todos los perfiles y extensiones de OAuth 2.0 y OpenID Connect, sino explicar los conceptos que serán usados posteriormente en los casos base y en los tres escenarios OAuth 2.0/OIDC de la tesis.

La lógica del capítulo es la siguiente:

1. Primero se diferencia identidad digital, autenticación, autorización y federación.
  2. Luego se explica qué resuelve OAuth 2.0 y qué no resuelve por sí solo.
  3. Después se explica qué agrega OpenID Connect sobre OAuth 2.0.
  4. Luego se analizan roles, tokens, validación, firmas JWT, PKCE y sesión con BFF.
  5. Finalmente, se conecta el marco conceptual con los casos arquitectónicos de evaluación.
- 

### 2.1. Identidad digital

La identidad digital es la representación de un sujeto dentro de un sistema de información. Ese sujeto puede ser:

- una persona;
- una aplicación;
- un servicio;
- una cuenta técnica;
- una carga de trabajo;
- un dispositivo;
- un proceso automático.

En el contexto de esta tesis, la identidad digital permite responder preguntas como:

1. ¿Quién o qué intenta acceder?
2. ¿Cómo se verifica esa identidad?
3. ¿Qué atributos describen al sujeto?
4. ¿Qué permisos tiene?
5. ¿Qué acciones ejecutó?
6. ¿Qué evidencia queda disponible para auditoría?

La identidad digital no se limita al usuario humano. En arquitecturas modernas, muchos accesos son ejecutados por servicios, pipelines, integraciones, aplicaciones backend o clientes técnicos. Por esta razón, la tesis considera tanto identidades humanas como identidades no humanas.

Desde la perspectiva de riesgo, una identidad digital mal gobernada puede generar:

- suplantación;
- exceso de privilegios;
- pérdida de trazabilidad;
- uso de cuentas huérfanas;
- abuso de credenciales técnicas;
- dificultad para atribuir acciones a un usuario, aplicación o servicio.

La identidad digital es el punto de partida para evaluar controles de autenticación, autorización, federación, emisión de tokens, validación de tokens y auditoría.

---

### 2.2. Autenticación, autorización y federación

**2.2.1. Autenticación** La autenticación es el proceso mediante el cual un sistema verifica la identidad de un sujeto.

Responde la pregunta:

¿Quién eres y cómo lo demuestras?

Ejemplos de mecanismos de autenticación:

- usuario y contraseña;
- certificado digital;
- llave criptográfica;
- autenticación multifactor;
- biometría;
- autenticación delegada ante un proveedor de identidad.

En OpenID Connect, la autenticación del usuario se representa principalmente mediante el `id_token`. Este token contiene claims sobre el evento de autenticación y sobre el usuario autenticado.

**2.2.2. Autorización** La autorización es el proceso mediante el cual un sistema decide si un sujeto autenticado o representado puede ejecutar una acción sobre un recurso.

Responde la pregunta:

¿Qué puedes hacer?

En OAuth 2.0, la autorización se expresa mediante:

- `access_token`;
- scopes;
- permisos;
- claims;
- políticas de acceso;
- relación entre cliente, recurso y servidor de autorización.

Un error común consiste en asumir que un token válido implica autorización suficiente. Esta suposición es débil. Un token puede ser válido criptográficamente, pero no estar destinado a la API correcta, no tener el scope requerido o no representar al sujeto adecuado para la operación solicitada.

**2.2.3. Federación de identidad** La federación de identidad ocurre cuando un sistema confía en una identidad autenticada o afirmada por otro sistema.

Responde la pregunta:

¿Acepto una identidad emitida por un tercero confiable?

En una arquitectura federada, una aplicación no necesariamente autentica directamente al usuario. En cambio, delega esa función en un proveedor de identidad o servidor de autorización.

Ejemplos de proveedores o plataformas de identidad:

- Keycloak;
- Microsoft Entra ID;
- Okta;
- Auth0;
- Ping Identity;
- ADFS;
- Amazon Cognito.

Federar no elimina el riesgo. Traslada parte de la confianza hacia un componente especializado. Por tanto, la evaluación debe revisar:

- quién emite la identidad o el token;
- qué relaciones de confianza existen;
- qué claims se aceptan;

- qué validaciones realiza el consumidor;
  - qué evidencia queda de la autenticación y autorización.
- 

## 2.3. OAuth 2.0

OAuth 2.0 es un marco de autorización. Permite que un cliente obtenga acceso limitado a recursos protegidos, ya sea en nombre de un usuario o en nombre de la propia aplicación.

Su propósito central no es autenticar usuarios, sino delegar autorización.

En términos simples:

OAuth 2.0 permite que una aplicación acceda a un recurso protegido sin que el usuario entregue directamente sus credenciales a esa aplicación.

**2.3.1. Qué resuelve OAuth 2.0** OAuth 2.0 ayuda a resolver problemas como:

- evitar que una aplicación conozca la contraseña del usuario;
- limitar el acceso mediante scopes;
- emitir tokens con vigencia controlada;
- separar autorización de consumo de recursos;
- permitir acceso delegado entre aplicaciones y APIs;
- soportar escenarios con usuario y escenarios sin usuario humano.

**2.3.2. Qué no resuelve OAuth 2.0 por sí solo** OAuth 2.0 no garantiza automáticamente que una arquitectura sea segura. Su seguridad depende de la configuración, del flujo seleccionado, de la validación de tokens, del manejo de secretos y de la operación de controles complementarios.

OAuth 2.0 no resuelve por sí solo:

- autenticación completa del usuario, salvo que se use junto con OIDC;
- autorización fina de negocio;
- protección automática contra XSS o robo de sesión;
- gestión segura de secretos;
- trazabilidad completa;
- segregación de funciones;
- gobierno de clientes técnicos;
- auditoría de accesos.

Por esta razón, en la tesis OAuth 2.0 se evalúa como parte de una arquitectura de controles, no como garantía suficiente de seguridad.

---

## 2.4. OpenID Connect

OpenID Connect es una capa de identidad construida sobre OAuth 2.0. Permite que un cliente verifique la identidad de un usuario final y obtenga información básica de su perfil mediante claims.

En términos simples:

OAuth 2.0 responde qué acceso fue autorizado; OpenID Connect permite además saber quién fue autenticado.

OIDC agrega elementos como:

- id\_token;
- endpoint de autorización;
- endpoint de token;

- endpoint de UserInfo, si aplica;
- claims de identidad;
- discovery metadata;
- JWKS para validación criptográfica;
- parámetro nonce para mitigar repetición o sustitución.

#### 2.4.1. Qué agrega OIDC sobre OAuth 2.0

OIDC no reemplaza a OAuth 2.0. Lo extiende.

OAuth 2.0 proporciona el mecanismo de autorización delegada. OIDC usa esa base para representar autenticación de usuario mediante un `id_token`.

Elementos clave que suma OIDC:

- **id\_token:** JWT que representa el evento de autenticación del usuario.
- **Claims de identidad:** atributos como sujeto, nombre, correo o grupos, según configuración.
- **nonce:** valor usado para mitigar ataques de repetición o sustitución en el flujo de autenticación.
- **UserInfo endpoint:** endpoint para consultar claims adicionales del usuario.
- **Discovery y JWKS:** metadatos y claves públicas para validar tokens.

**2.4.2. OAuth 2.0 no es lo mismo que OIDC** La tesis trata explícitamente esta diferencia porque la confusión entre OAuth 2.0 y OIDC genera riesgos de diseño. Una forma práctica de explicarlo es separar tres momentos: la comprobación de credenciales o identidad ante el Authorization Server, el artefacto que se emite y el componente que consume ese artefacto.

#### Registro 1: Propósito

- **OAuth 2.0:** autorización delegada. Permite obtener un `access_token` para acceder a recursos protegidos bajo ciertos scopes, audiencia y vigencia.
- **OIDC:** autenticación federada del usuario sobre OAuth 2.0. Permite obtener un `id_token` que representa quién fue autenticado y bajo qué contexto.

**Registro 2: Autenticación ante el Authorization Server** En ambos casos puede existir autenticación contra el Authorization Server o Identity Provider. En OAuth 2.0, el usuario puede autenticarse ante el Authorization Server para aprobar una autorización, o el cliente puede autenticarse en el endpoint de token. En OIDC también existe autenticación del usuario ante el proveedor de identidad.

La diferencia no está en que uno autentique y el otro no autentique nunca. La diferencia está en el significado del artefacto resultante.

#### Registro 3: Artefacto principal emitido

- **OAuth 2.0:** entrega principalmente un `access_token`. Ese artefacto representa autorización: qué acceso fue concedido, para qué recurso, con qué scopes, durante cuánto tiempo y bajo qué cliente o sujeto.
- **OIDC:** agrega un `id_token`. Ese artefacto representa autenticación: quién fue autenticado, por qué emisor, para qué cliente, en qué momento y con qué claims de identidad.

#### Registro 4: Consumidor del artefacto

- **access\_token:** está destinado al Resource Server o API protegida. La API lo usa para decidir si permite una operación.
- **id\_token:** está destinado al cliente OIDC, por ejemplo un BFF o aplicación cliente. Sirve para establecer o validar la sesión de usuario en el cliente, no para autorizar APIs de negocio.

Por tanto, una explicación precisa es: OAuth 2.0 puede implicar autenticación ante el Authorization Server, pero el artefacto que entrega para el Resource Server es una autorización. OIDC también implica autenticación ante el proveedor de identidad, pero además estandariza un artefacto de autenticación, el `id_token`, que le dice al cliente quién fue autenticado.

### Registro 5: Pregunta que responde

- **OAuth 2.0:** ¿qué acceso fue autorizado?
- **OIDC:** ¿quién fue autenticado y bajo qué contexto?

### Registro 6: Error frecuente

- Usar OAuth 2.0 como si fuera autenticación completa de usuario.
- Usar el `id_token` para consumir APIs de negocio.
- Aceptar claims sin validar emisor, audiencia, firma, expiración o contexto.
- Asumir que login federado implica autorización automática sobre recursos de negocio.

La matriz de evaluación debe verificar que cada token se use para el propósito correcto: el `access_token` para autorización ante Resource Servers y el `id_token` para identidad/autenticación ante el cliente OIDC.

---

## 2.5. Roles de OAuth 2.0

OAuth 2.0 define roles lógicos. Estos roles pueden estar desplegados en sistemas separados o convivir dentro de una misma plataforma.

**2.5.1. Resource Owner** Es el sujeto que posee o está asociado al recurso protegido.

En escenarios con usuario final, suele ser una persona. En escenarios técnicos, puede no haber usuario humano directo y el acceso puede representar a un cliente técnico.

Riesgos asociados:

- ambigüedad sobre quién es el sujeto real;
- pérdida de trazabilidad;
- confusión entre acción de usuario y acción de sistema.

**2.5.2. Client** Es la aplicación que solicita autorización para acceder a un recurso protegido.

Puede ser:

- SPA;
- aplicación móvil;
- BFF;
- backend;
- proceso batch;
- servicio técnico;
- API Gateway actuando como intermediario.

La matriz debe evaluar si el tipo de cliente coincide con los controles aplicados. Un cliente público no debe depender de la confidencialidad de un `client_secret` embebido en la aplicación.

**2.5.3. Authorization Server** Es el componente que autentica al cliente, procesa la autorización y emite tokens.

Puede coincidir con un proveedor de identidad cuando se usa OIDC.

Controles relevantes:

- registro de clientes;
- redirect URIs;
- scopes;
- políticas de token;
- algoritmos permitidos;
- gestión de claves;

- autenticación de clientes;
- logs de emisión y revocación;
- rotación de secretos y certificados.

**2.5.4. Resource Server** Es la API o servicio que protege recursos y recibe tokens de acceso.

Su responsabilidad no es solo recibir el token. Debe validarlo y aplicar autorización.

Controles relevantes:

- validación de firma o introspección;
- validación de issuer;
- validación de audience;
- validación de expiración;
- validación de scopes o permisos;
- aplicación de reglas de negocio;
- trazabilidad de decisiones de acceso.

**2.5.5. Authorization Server y Resource Server pueden convivir** Conceptualmente, Authorization Server y Resource Server son roles distintos.

Sin embargo, en la práctica pueden convivir dentro del mismo servicio o producto. Esto puede ocurrir por:

- simplicidad operativa;
- bajo presupuesto;
- equipos pequeños;
- aplicaciones monolíticas;
- plataformas IAM que emiten tokens y exponen APIs administrativas propias.

En estos casos, el riesgo aumenta porque una misma plataforma concentra:

- emisión de tokens;
- validación de tokens;
- APIs administrativas;
- gestión de usuarios;
- gestión de clientes;
- gestión de claves;
- logs críticos de identidad.

La evaluación debe verificar que, aunque los roles convivan físicamente, estén separados lógicamente mediante controles de autorización, logging, segregación de funciones y mínimo privilegio.

## 2.6. Tokens en OAuth 2.0 y OIDC

Los tokens son artefactos de seguridad emitidos por un Authorization Server o proveedor de identidad. Representan información sobre autorización, autenticación, cliente, usuario, permisos o contexto.

Los tokens relevantes para la tesis son:

- access\_token;
- id\_token;
- refresh\_token;
- código de autorización;
- tokens opacos;
- tokens JWT.

**2.6.1. Access token** El `access_token` es el token que el cliente presenta ante un Resource Server o API protegida para acceder a un recurso.

Aspectos que deben evaluarse:

- emisor;
- audiencia;
- expiración;
- scopes;
- permisos;
- sujeto o cliente representado;
- mecanismo de validación;
- protección en tránsito y almacenamiento;
- trazabilidad del uso.

Un `access_token` no debe tratarse como credencial de larga duración. Si se expone, puede permitir acceso indebido al recurso protegido, especialmente cuando se trata de tokens bearer.

**2.6.2. ID token** El `id_token` es propio de OpenID Connect. Representa información sobre la autenticación del usuario final.

Aspectos que deben evaluarse:

- firma;
- issuer;
- audience;
- expiración;
- nonce, cuando aplique;
- claims de autenticación;
- uso correcto por parte del cliente.

El `id_token` no debe usarse como token para invocar APIs de negocio. Su propósito principal es transmitir información de autenticación al cliente.

**2.6.3. Refresh token** El `refresh_token` permite obtener nuevos tokens de acceso sin repetir todo el flujo de autorización.

Es sensible porque suele tener mayor vida útil que el `access_token`.

Controles esperados:

- almacenamiento seguro;
- rotación;
- revocación;
- detección de reutilización;
- restricción por cliente;
- protección especial en clientes públicos.

**2.6.4. Código de autorización** El código de autorización es un artefacto temporal usado en Authorization Code Flow. No es el token final. Es un código intermedio que el cliente cambia por tokens en el endpoint de token.

Riesgos asociados:

- interceptación del código;
- reutilización del código;
- redirect URI mal registrada;
- ausencia de PKCE;
- mezcla de códigos entre sesiones.

Controles esperados:

- PKCE;
- redirect URI exacta;
- expiración corta;
- un solo uso;
- validación de state;
- validación de cliente.

---

## 2.7. Token opaco y JWT embebido

OAuth 2.0 no obliga a que el access\_token sea JWT. Un access\_token puede ser opaco o auto-contenido.

**2.7.1. Token opaco** Un token opaco funciona como identificador de referencia.

Ejemplo conceptual:

2YotnFZFEjr1zCsicMWpAA

El token no contiene información visible para el cliente ni para el Resource Server. Para saber si sigue activo y qué permisos representa, el Resource Server consulta al Authorization Server mediante introspección.

Flujo conceptual:

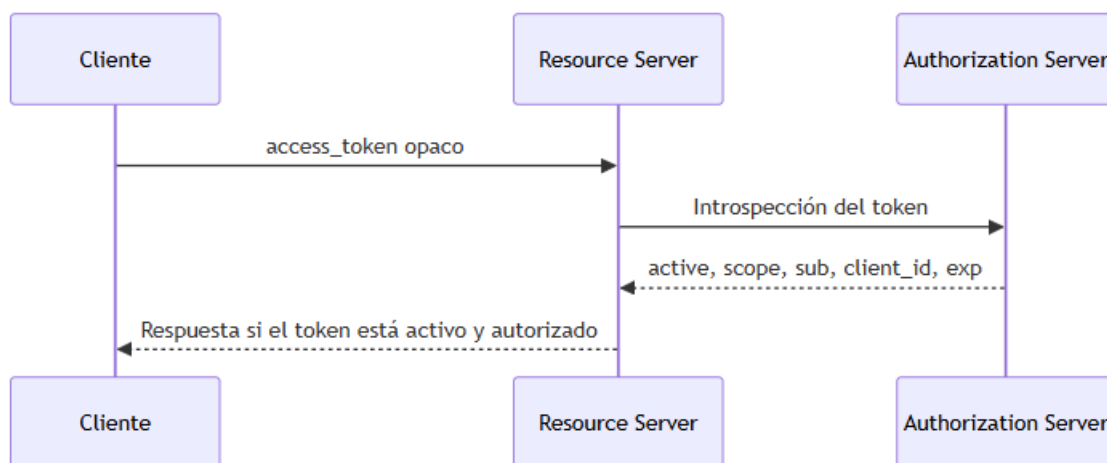


Figura 2: Token opaco validado mediante introspección

Ventajas:

- facilita revocación inmediata;
- evita exponer claims dentro del token;
- centraliza la verificación en el Authorization Server.

Costos:

- introduce una llamada adicional al Authorization Server;
- acopla al Resource Server con el Authorization Server en tiempo de ejecución;
- requiere caché o diseño de disponibilidad para evitar degradación.



**2.7.2. JWT embebido o self-contained** Un JWT auto-contenido lleva claims dentro del token.

Estructura conceptual:

`header.payload.signature`

El payload puede contener claims como:

- `sub`;
- `iss`;
- `aud`;
- `exp`;
- `iat`;
- `scope`;
- `client_id`;
- roles o permisos, si aplica.

El Resource Server puede validar el token sin consultar al Authorization Server en cada request. Para esto usa la clave pública publicada por el Authorization Server, normalmente a través de JWKS.

Ventajas:

- permite validación local;
- escala bien en microservicios;
- reduce llamadas al Authorization Server.

Costos:

- revocar antes de `exp` es más complejo;
- si el token está firmado pero no cifrado, los claims pueden ser leídos por quien vea el token;
- exige buena gestión de claves y rotación de JWKS.

**2.7.3. Decisión arquitectónica** La elección entre token opaco y JWT embebido es una decisión de arquitectura.

Un token opaco favorece control centralizado y revocación. Un JWT auto-contenido favorece escalabilidad y validación distribuida.

La matriz debe evaluar si la decisión es coherente con:

- criticidad del recurso;
- necesidad de revocación;
- latencia aceptable;
- disponibilidad del Authorization Server;
- sensibilidad de los claims;
- capacidad de validar correctamente firma, issuer, audience, expiración y scopes.

---

## 2.8. JWT, JWS y JWE

JWT significa JSON Web Token. Es un formato compacto para representar claims entre dos partes.

Un JWT puede estar:

- firmado mediante JWS;
- cifrado mediante JWE;
- firmado y cifrado, según el diseño.

**2.8.1. JWS** JWS significa JSON Web Signature. Proporciona integridad y autenticidad.

En una arquitectura OAuth 2.0/OIDC, JWS permite verificar que:

- el contenido del token no fue alterado;
- el token fue emitido por una entidad confiable;
- la firma corresponde a una clave privada controlada por el emisor.

Riesgos típicos:

- aceptar tokens sin verificar firma;
- permitir algoritmos inseguros;
- no validar el algoritmo esperado;
- confiar en claims sin firma válida;
- no refrescar JWKS tras rotación de claves.

**2.8.2. JWE** JWE significa JSON Web Encryption. Proporciona confidencialidad sobre el contenido.

No debe confundirse con JWS:

- JWS protege integridad y autenticidad.
- JWE protege confidencialidad.

Muchos tokens en arquitecturas OAuth 2.0/OIDC están firmados, pero no cifrados. Por tanto, no se debe incluir información sensible innecesaria dentro de un JWT, incluso si está firmado.

**2.8.3. Cómo se firma y valida un JWT** De forma simplificada, el Authorization Server firma y el Resource Server verifica.

#### **Authorization Server: emisión**

1. Construye el header y el payload.
2. Codifica ambos en Base64URL.
3. Concatena `header.payload`.
4. Calcula una huella criptográfica sobre esa cadena.
5. Firma con su clave privada.
6. Entrega un token con forma `header.payload.signature`.

Ejemplo conceptual:

#### **Resource Server: validación**

1. Recibe el JWT.
2. Descarga o consulta la clave pública del Authorization Server desde JWKS.
3. Recalcula la huella criptográfica sobre `header.payload`.
4. Verifica la firma con la clave pública.
5. Valida claims obligatorios como `iss`, `aud`, `exp` y `scopes`.
6. Aplica autorización de negocio.

La firma solo demuestra integridad y autenticidad del token. No reemplaza la autorización de negocio.

Riesgo conceptual importante:

Un JWT con firma válida puede seguir siendo inválido para una API si la audiencia, expiración, emisor o scopes no corresponden.

---

## **2.9. Authorization Code Flow con PKCE**

Authorization Code Flow es un flujo donde el cliente recibe un código de autorización y luego lo intercambia por tokens en el endpoint de token.

PKCE agrega una prueba criptográfica entre la solicitud inicial y el intercambio del código.

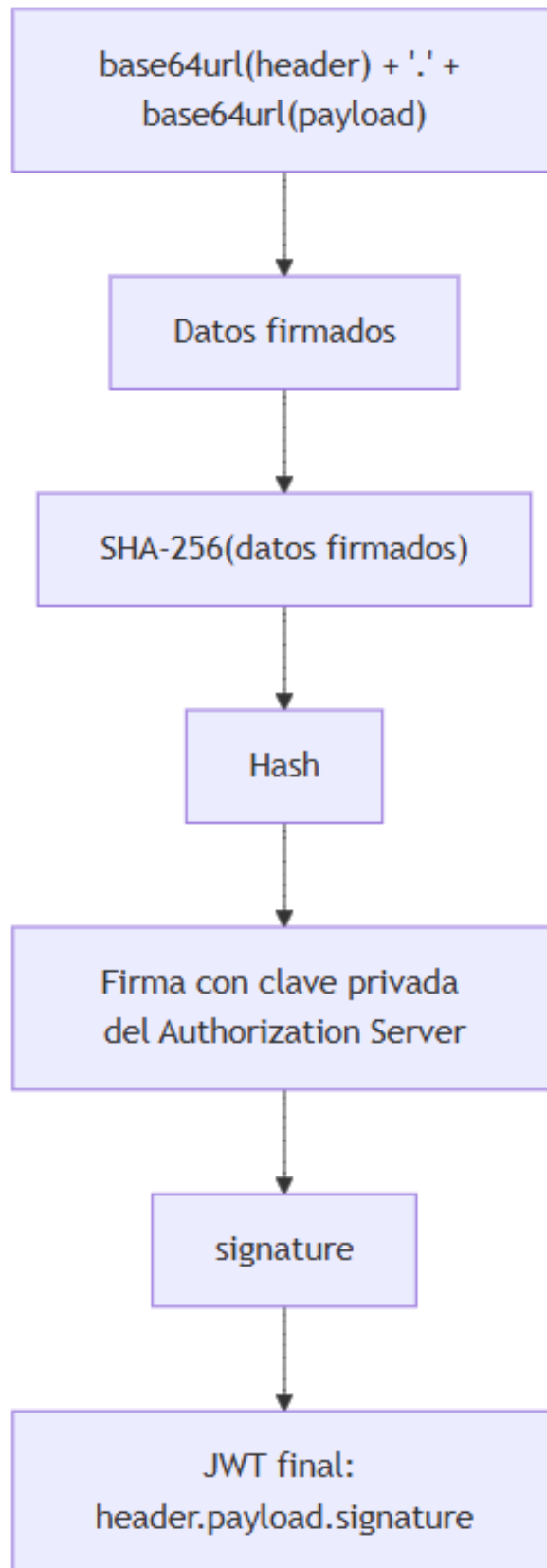


Figura 3: Firma de JWT mediante JWS

**2.9.1. Problema: robo del authorization code** Antes de PKCE, un atacante que interceptara el `authorization_code` podía intentar canjearlo por tokens.

Este riesgo es especialmente importante en clientes públicos, como aplicaciones móviles o SPAs, porque no pueden proteger un `client_secret` de forma confiable.

Ejemplo conceptual:

1. App legítima inicia login.
2. Authorization Server devuelve `authorization_code`.
3. App maliciosa intercepta el callback.
4. App maliciosa canjea el code.
5. Authorization Server entrega tokens al atacante.

El Authorization Server no puede distinguir adecuadamente a las dos aplicaciones si ambas comparten elementos observables como `client_id` o esquemas de redirección inseguros.

**2.9.2. Solución: PKCE** PKCE introduce dos valores:

- `code_verifier`: secreto aleatorio generado por el cliente.
- `code_challenge`: derivado criptográfico del `code_verifier`.

Normalmente:

```
code_challenge = BASE64URL(SHA-256(code_verifier))
```

Flujo conceptual:

1. Cliente genera `code_verifier`.
2. Cliente calcula `code_challenge`.
3. Cliente inicia `/authorize` con `code_challenge` y `code_challenge_method=S256`.
4. Authorization Server guarda el challenge asociado al code.
5. Authorization Server devuelve `authorization_code`.
6. Cliente canjea code + `code_verifier` en `/token`.
7. Authorization Server verifica que `SHA-256(code_verifier)` coincida con el challenge guardado.
8. Si coincide, entrega tokens.

Metáfora útil:

El `authorization_code` es el ticket. El `code_verifier` es el comprobante de retiro. Sin ambos, no se entrega el paquete.

**2.9.3. Qué mitiga PKCE** PKCE mitiga:

- interceptación del `authorization code`;
- inyección de código entre aplicaciones;
- canje indebido del código por un cliente que no conoce el `code_verifier`.

PKCE no hace que el navegador o el dispositivo sean seguros por sí solos. Solo protege el intercambio `authorization_code` → tokens.

**2.9.4. PKCE no es bala de plata** Aunque PKCE esté bien implementado, permanecen riesgos residuales:

**Malware en el dispositivo** Si el atacante ejecuta código en el dispositivo, podría leer el `code_verifier` de memoria, logs o almacenamiento local.

Lectura de control:

PKCE protege el flujo, no todo el cliente.

**XSS en una SPA** Si un atacante inyecta JavaScript en una SPA, puede iniciar un flujo propio, manejar su propio `code_verifier` o robar tokens si la SPA los conserva.

Control complementario:

- patrón BFF;
- cookies `HttpOnly`;
- Content Security Policy;
- no almacenar tokens en `localStorage`.

**Phishing del consentimiento** Si el usuario autoriza una aplicación maliciosa que se hace pasar por legítima, PKCE puede completar el flujo correctamente para la aplicación equivocada.

Control complementario:

- UX clara del Authorization Server;
- validación estricta del cliente;
- redirect URIs seguras;
- políticas de consentimiento.

**Downgrade a plain** Si el Authorization Server acepta `code_challenge_method=plain`, el `code_challenge` puede equivaler al `code_verifier` en claro.

Control esperado:

- exigir S256;
- rechazar `plain` salvo justificación excepcional.

**Verifier con baja entropía** Si el cliente genera el `code_verifier` con baja aleatoriedad, podría ser adivinado.

Control esperado:

- generador criptográfico seguro;
- suficiente entropía;
- no derivar el verifier de datos predecibles.

**Después del token, PKCE termina** PKCE no protege el `access_token` ni el `refresh_token` después de emitidos.

Controles complementarios:

- expiración corta;
- rotación de refresh tokens;
- audience específica;
- scopes mínimos;
- almacenamiento seguro;
- monitoreo de uso anómalo.

**Implicación para la matriz** No basta con preguntar:

¿Usan PKCE?

La matriz debe evaluar:

1. si se usa S256;
2. si el Authorization Server rechaza `plain`;
3. si el `code_verifier` tiene entropía suficiente;
4. si el `code_verifier` se almacena solo durante el flujo;
5. si existen controles complementarios contra XSS, malware y phishing;
6. si los tokens emitidos tienen controles propios después del canje.

---

## 2.10. SPA estática, BFF, cookies y tokens

En arquitecturas con SPA estática y Backend for Frontend, puede aparecer una confusión frecuente:

¿Es OAuth 2.0 con cookies?

La respuesta precisa es:

No. OAuth 2.0/OIDC ocurre entre el BFF y el Authorization Server. La cookie ocurre entre el navegador y el BFF.

Son dos conversaciones separadas.

**2.10.1. Canal A: navegador hacia BFF** El navegador o SPA conserva una cookie de sesión emitida por el BFF.

Ejemplo conceptual:

Cookie: SESSION\_ID=abc123

La SPA no debería guardar:

- access\_token;
- refresh\_token;
- client\_secret;
- tokens en localStorage;
- tokens en sessionStorage, salvo diseños justificados y controlados.

Controles esperados para la cookie:

- HttpOnly;
- Secure;
- SameSite;
- expiración adecuada;
- protección contra CSRF;
- rotación o renovación controlada de sesión.

**2.10.2. Canal B: BFF hacia Authorization Server y APIs** El BFF actúa como cliente OAuth 2.0/OIDC.

Responsabilidades del BFF:

- iniciar el flujo Authorization Code + PKCE;
- recibir el authorization\_code;
- canjear el código por tokens;
- custodiar tokens server-side;
- mapear SESSION\_ID a contexto de usuario y tokens;
- refrescar tokens cuando corresponda;
- llamar APIs protegidas con Authorization: Bearer.

Ejemplo conceptual:

Authorization: Bearer eyJhbGciOiJSUzI1NiJ9...

### 2.10.3. Regla mental

Frontend = cookie

BFF = custodia tokens

Authorization Server / IdP = emite tokens

API / Resource Server = valida tokens

**2.10.4. Implicación arquitectónica** Si la SPA solo consume al BFF con cookie, la SPA no ejecuta OAuth 2.0 directamente.

Pero el sistema completo sí usa OAuth 2.0/OIDC cuando el BFF delega identidad en un Authorization Server o consume APIs protegidas mediante tokens.

Si el BFF tiene gestión propia de usuarios y no consume APIs protegidas por un Authorization Server externo, entonces ese flujo de login puede operar como autenticación web tradicional basada en sesión, sin OAuth 2.0/OIDC.

Esta distinción justifica los dos casos base de la tesis:

- monolito con sesión local;
- SPA estática + BFF con gestión propia de usuarios.

Ambos sirven como línea base para comparar qué cambia cuando se introduce identidad federada, Authorization Server, tokens, scopes y controles OAuth 2.0/OIDC.

---

## 2.11. Single Sign-On

Single Sign-On es un patrón en el cual un usuario se autentica una vez ante un Identity Provider y varias aplicaciones aceptan esa identidad sin volver a pedir contraseña.

En el contexto de OIDC, el IdP autentica al usuario y emite tokens o claims que las aplicaciones pueden validar.

### 2.11.1. SSO personal Ejemplo:

Iniciar sesión con Google.

Flujo conceptual:

1. Una aplicación como Notion, Spotify o Figma ofrece iniciar sesión con Google.
2. El usuario es redirigido a `accounts.google.com`.
3. Google autentica al usuario o reutiliza una sesión ya activa.
4. Google devuelve al cliente información de identidad mediante OIDC.
5. La aplicación crea su propia sesión y administra sus propios permisos.

Lectura de riesgo:

- el IdP es Google;
- el usuario controla parte del consentimiento;
- cada aplicación sigue siendo responsable de su autorización interna.

### 2.11.2. SSO corporativo Ejemplo:

Iniciar sesión con Microsoft Entra ID.

Flujo conceptual:

1. El usuario abre Teams, Outlook, Power BI o una aplicación interna.
2. La aplicación redirige al tenant corporativo.
3. Entra ID aplica políticas como MFA, dispositivo gestionado, grupos o ubicación.
4. El IdP devuelve tokens y claims a la aplicación.
5. La aplicación crea su sesión y aplica autorización propia.

Lectura de riesgo:

- el IdP es la organización;
- TI gobierna usuarios, MFA, ciclo de vida y revocación;
- las aplicaciones deben validar tokens y claims;
- el login corporativo no reemplaza la autorización de negocio.

**2.11.3. Regla conceptual** La mecánica OIDC puede ser similar en SSO personal y corporativo. Lo que cambia es:

- quién opera el IdP;
  - qué políticas aplica;
  - qué claims emite;
  - qué confianza deposita la aplicación en esos claims;
  - qué evidencia queda disponible para auditoría.
- 

## 2.12. Flujos relevantes para la tesis

La tesis no cubre todos los flujos de OAuth 2.0. Se concentra en los que son necesarios para los escenarios seleccionados.

**2.12.1. Authorization Code + PKCE** Aplica al escenario de usuario final con SPA estática + BFF + Authorization Server.

Aspectos evaluables:

- redirect URI exacta;
- state;
- nonce en OIDC;
- PKCE con S256;
- custodia server-side de tokens;
- cookie segura entre navegador y BFF;
- ausencia de tokens en el frontend;
- protección contra XSS y CSRF.

**2.12.2. Client Credentials** Aplica al escenario M2M.

Aspectos evaluables:

- registro de cliente técnico;
- client\_id;
- client\_secret, certificado o private\_key\_jwt;
- scopes mínimos;
- separación por ambiente;
- separación por propósito;
- vault;
- rotación;
- monitoreo del client\_id.

**2.12.3. Token Introspection** Aplica cuando se usan tokens opacos o cuando un Resource Server necesita consultar estado activo y metadatos del token.

Aspectos evaluables:

- autenticación del Resource Server ante el endpoint de introspección;
- canal seguro;
- validación del campo active;
- validación de scopes, cliente, sujeto y expiración;
- cache controlado;
- disponibilidad del Authorization Server.

**2.12.4. Token Exchange** Aplica cuando un componente, por ejemplo un API Gateway, recibe un token y necesita obtener otro token para un Resource Server o dominio diferente.

Aspectos evaluables:



- audiencia específica por dominio;
  - reducción o transformación de scopes;
  - trazabilidad de delegación;
  - separación entre token entrante y token saliente;
  - evidencia de intercambio;
  - logs correlacionados entre gateway, Authorization Server y Resource Server.
- 

### 2.13. Relación del marco conceptual con los casos de evaluación

La tesis usa dos casos base y tres escenarios OAuth 2.0/OIDC.

**Caso base 0A: monolito con sesión local** No usa OAuth 2.0/OIDC como mecanismo principal.

Sirve para observar una arquitectura donde:

- la aplicación autentica localmente;
- la sesión vive en el propio monolito;
- la autorización suele estar embebida en la lógica del sistema;
- la evidencia se concentra en logs, base de datos, configuración y código del monolito.

Riesgos dominantes:

- robo de sesión;
- contraseñas mal protegidas;
- autorización mezclada con lógica de negocio;
- baja separación de responsabilidades;
- auditoría limitada.

**Caso base 0B: SPA estática + BFF con usuarios propios** No usa OAuth 2.0/OIDC para el login si el BFF gestiona sus propios usuarios.

Sirve para observar una arquitectura de tres capas donde:

- el frontend no autentica directamente;
- el BFF administra sesión con cookie;
- la identidad sigue siendo local al sistema;
- no existen tokens OAuth 2.0/OIDC en el flujo de login.

Riesgos dominantes:

- cookie mal configurada;
- CSRF contra el BFF;
- XSS en la SPA;
- mala gestión local de usuarios;
- autorización interna poco trazable.

**Arquitectura 1: SPA estática + BFF + Authorization Server** Usa OAuth 2.0/OIDC.

El BFF actúa como cliente OAuth 2.0/OIDC. La SPA conserva cookie. El BFF custodia tokens. El Authorization Server emite tokens. Las APIs validan tokens.

Riesgo dominante:

- exposición de tokens si la SPA los maneja;
- robo de sesión del BFF;
- errores de PKCE, state, nonce o redirect URI;
- scopes excesivos.

Control crítico:

- Authorization Code + PKCE;
- tokens server-side;
- cookie HttpOnly, Secure, SameSite;
- validación de state y nonce;
- CSP contra XSS.

**Arquitectura 2: M2M contra recursos protegidos del propio Authorization Server** Usa OAuth 2.0 mediante `client_credentials`.

El cliente técnico obtiene un `access_token` para consumir APIs protegidas que expone el propio Authorization Server, por ejemplo APIs administrativas de usuarios, clientes, scopes o claves.

Riesgo dominante:

- abuso de credenciales técnicas;
- privilegios administrativos excesivos;
- secretos hardcodeados;
- clientes huérfanos;
- movimiento lateral dentro de la plataforma de identidad.

Control crítico:

- scopes mínimos;
- inventario de clientes técnicos;
- vault;
- rotación;
- mTLS o `private_key_jwt`, si aplica;
- monitoreo por `client_id`.

**Arquitectura 3: API Gateway federado + Authorization Server corporativo** Usa OAuth 2.0/OIDC en una arquitectura con múltiples dominios de recursos.

El API Gateway valida tokens frente a un Authorization Server corporativo y enruta hacia APIs de dominio. Según el diseño, puede propagar el token original o intercambiarlo por otro token con audiencia específica.

Riesgo dominante:

- confusión de audiencia;
- backend que confía ciegamente en el gateway;
- criterios de validación inconsistentes;
- propagación de tokens sin reducción de scopes;
- JWKS desactualizado;
- baja trazabilidad extremo a extremo.

Control crítico:

- validación de `audience` por dominio;
- revalidación en cada Resource Server;
- política única de firma, `issuer`, `exp` y scopes;
- token exchange con reducción de privilegios;
- logs correlacionados gateway-backend.

---

## 2.14. Riesgos comunes en arquitecturas OAuth 2.0/OIDC

Los riesgos en arquitecturas OAuth 2.0/OIDC no provienen únicamente del protocolo. Surgen de la interacción entre protocolo, configuración, arquitectura, implementación y operación.

#### 2.14.1. Riesgos transversales Ejemplos:

- gestión deficiente de clientes OAuth;
- scopes excesivos;
- tokens con vigencia inadecuada;
- falta de rotación de secretos;
- ausencia de trazabilidad;
- configuración inconsistente entre ambientes;
- falta de revisión periódica de clientes y permisos;
- aceptación de algoritmos inseguros;
- ausencia de monitoreo sobre eventos de autenticación y autorización;
- falta de gobierno sobre cambios en el Authorization Server.

#### 2.14.2. Riesgos de usuario final Ejemplos:

- exposición de tokens en navegador;
- almacenamiento inseguro en localStorage o logs;
- robo de sesión;
- redirect URIs débiles;
- ausencia o mala validación de state;
- ausencia o mala validación de nonce;
- uso de flujos no recomendados para clientes públicos;
- exceso de permisos solicitados al usuario.

#### 2.14.3. Riesgos de API protegida o Resource Server Ejemplos:

- aceptar tokens sin validar firma;
- aceptar tokens emitidos por un issuer no confiable;
- no validar audience;
- aceptar tokens expirados;
- validar autenticación pero no autorización;
- confiar completamente en el API Gateway sin controles en backend;
- no registrar decisiones de acceso;
- tratar tokens opacos como si fueran JWT;
- usar claims no confiables para decisiones de negocio.

#### 2.14.4. Riesgos de M2M Ejemplos:

- secretos hardcodeados;
- secretos almacenados en repositorios o pipelines sin protección;
- ausencia de rotación;
- clientes técnicos compartidos por varios servicios;
- scopes demasiado amplios;
- clientes técnicos sin responsable;
- uso cruzado de credenciales entre ambientes;
- dificultad para atribuir operaciones a un sistema específico;
- uso de M2M para operaciones que deberían conservar contexto de usuario.

---

### 2.15. Fuentes técnicas base del marco conceptual

Este marco conceptual se apoya principalmente en las siguientes fuentes técnicas:

- RFC 6749: OAuth 2.0 Authorization Framework.
- RFC 6750: OAuth 2.0 Bearer Token Usage.
- RFC 7515: JSON Web Signature.

- RFC 7516: JSON Web Encryption.
- RFC 7517: JSON Web Key.
- RFC 7519: JSON Web Token.
- RFC 7636: Proof Key for Code Exchange.
- RFC 7662: OAuth 2.0 Token Introspection.
- RFC 8693: OAuth 2.0 Token Exchange.
- RFC 8705: OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens.
- RFC 9068: JWT Profile for OAuth 2.0 Access Tokens.
- RFC 9449: OAuth 2.0 Demonstrating Proof of Possession.
- RFC 9700: Best Current Practice for OAuth 2.0 Security.
- OpenID Connect Core 1.0.
- OWASP OAuth 2.0 Cheat Sheet.
- OWASP Session Management Cheat Sheet.
- OWASP Cross-Site Request Forgery Prevention Cheat Sheet.

### 3. Marco de aseguramiento, riesgo, control y auditoría

Este capítulo define el marco conceptual de aseguramiento que soporta el modelo de evaluación propuesto. Su propósito es explicar por qué la tesis no se limita a revisar configuraciones OAuth 2.0/OIDC, sino que estructura una evaluación basada en riesgos, controles y evidencias auditables.

El punto central es que una arquitectura puede usar protocolos y productos reconocidos sin estar adecuadamente asegurada. El aseguramiento no depende solo de la existencia nominal de un Authorization Server, un API Gateway, un flujo OIDC o un conjunto de tokens. Depende de la relación entre:

1. los riesgos relevantes del escenario;
2. los controles esperados para esos riesgos;
3. el estado real del diseño e implementación del control;
4. la evidencia disponible para comprobar su operación;
5. la conclusión sobre exposición observada.

---

#### 3.1. Aseguramiento de tecnologías de información

El aseguramiento de tecnologías de información busca aportar confianza razonable sobre la forma en que los sistemas, procesos, datos, controles y responsabilidades de TI contribuyen al cumplimiento de objetivos organizacionales, de seguridad y de cumplimiento.

En esta tesis, el aseguramiento se entiende como una evaluación estructurada que permite responder preguntas como:

1. ¿Qué puede fallar en la arquitectura evaluada?
2. ¿Qué controles deberían existir frente a ese riesgo?
3. ¿El control está diseñado, declarado, implementado o auditado?
4. ¿La evidencia encontrada permite sustentar esa calificación?
5. ¿Qué nivel de exposición permanece después de evaluar los controles?

La expresión “confianza razonable” es relevante porque ningún sistema es completamente seguro. El objetivo no es demostrar seguridad absoluta, sino contar con una base trazable para juzgar si los controles son suficientes y verificables frente a los riesgos identificados.

En arquitecturas OAuth 2.0/OIDC, el aseguramiento exige observar tanto componentes técnicos como decisiones de gobierno:

- clientes OAuth registrados;
- flujos permitidos;
- redirect URIs;
- scopes y audiencias;

- secretos, certificados y claves;
  - validación de tokens;
  - trazabilidad de autenticación y autorización;
  - evidencia de configuración y operación.
- 

### 3.2. Riesgo de tecnologías de información

Un riesgo de tecnologías de información puede entenderse como la posibilidad de que una amenaza explote una vulnerabilidad o condición habilitante y produzca un impacto sobre un activo.

Para efectos de esta tesis, cada riesgo se caracteriza mediante:

- **activo afectado:** componente, dato, token, secreto, API, sesión o registro comprometible;
- **amenaza o causa:** evento, actor o condición que puede materializar el riesgo;
- **vulnerabilidad o condición habilitante:** debilidad técnica, operativa o documental que permite el evento;
- **probabilidad:** posibilidad estimada de ocurrencia;
- **impacto:** consecuencia esperada sobre confidencialidad, integridad, disponibilidad, trazabilidad, cumplimiento o negocio;
- **impacto económico estimado:** valor usado para calcular eficiencia del tratamiento.

En OAuth 2.0/OIDC, el riesgo no siempre está en el protocolo. Muchas veces surge de su implementación:

- tokens aceptados fuera de audiencia;
- PKCE ausente o degradado;
- clientes técnicos con privilegios excesivos;
- secretos en repositorios o pipelines;
- Resource Servers que confían ciegamente en el Gateway;
- trazabilidad insuficiente de decisiones de autorización.

Por eso el modelo no pregunta únicamente “¿se usa OAuth 2.0?”, sino “¿qué riesgos específicos aparecen en esta arquitectura y qué controles los mitigan?”.

---

### 3.3. Controles de tecnologías de información

Un control es una medida técnica, organizacional o procedimental diseñada para reducir probabilidad, impacto o exposición frente a un riesgo.

En esta tesis se distinguen tres niveles de control:

1. **Control esperado:** control definido en el catálogo como mitigación aplicable a un riesgo.
2. **Control observado:** estado real del control durante la evaluación.
3. **Control evidenciado:** control cuya existencia u operación puede sustentarse mediante evidencia verificable.

Esta distinción evita confundir una afirmación con una prueba. Por ejemplo, decir que “la API valida tokens” no es suficiente. La matriz debe poder registrar evidencia de:

- validación de firma o introspección;
- validación de issuer;
- validación de audience;
- validación de expiración;
- validación de scopes;
- decisión de autorización;
- logs de aceptación o rechazo.

Los controles pueden ser globales o específicos:

- **Globales:** aplican a cualquier arquitectura OAuth 2.0/OIDC, como gobierno de clientes, validación completa de tokens o logging.
  - **Específicos:** aplican a un escenario particular, como PKCE en SPA+BFF+IdP, vault y rotación en M2M, o token exchange en API Gateway federado.
- 

### 3.4. Auditoría basada en riesgos

La auditoría basada en riesgos prioriza la evaluación de controles según la exposición que representan los activos, procesos y arquitecturas revisadas. En lugar de aplicar una lista genérica de verificación sin contexto, parte de los riesgos relevantes y evalúa los controles más importantes para mitigarlos.

En esta tesis, la lógica de auditoría basada en riesgos se traduce así:

La evidencia puede provenir de varias fuentes:

- configuración del Authorization Server;
- configuración del API Gateway;
- configuración del Resource Server;
- consola del proveedor de identidad;
- archivos de configuración;
- logs;
- trazas;
- pruebas técnicas;
- revisión puntual de código;
- documentación de arquitectura;
- pipelines;
- inventario de clientes o secretos.

Esta evidencia permite distinguir entre controles diseñados, declarados, implementados y auditados.

---

### 3.5. Relación riesgo-control-evidencia

El modelo propuesto se apoya en una relación muchos-a-muchos entre riesgos y controles. Un riesgo puede requerir varios controles, y un mismo control puede mitigar varios riesgos.

La relación se representa mediante `RiesgoControl`, que contiene:

- identificador del riesgo;
- identificador del control;
- peso de mitigación ( $w_i$ );
- grado de mitigación;
- obligatoriedad;
- justificación técnica;
- referencias.

La evidencia aparece como soporte del estado observado del control. Sin evidencia, el modelo penaliza el score final del control mediante el factor de evidencia ( $F_{\{ev\}}$ ).

La relación conceptual es:

Esta estructura permite responder una pregunta clave:

¿Cuánto aporta realmente un control evidenciado frente a un riesgo específico?

---

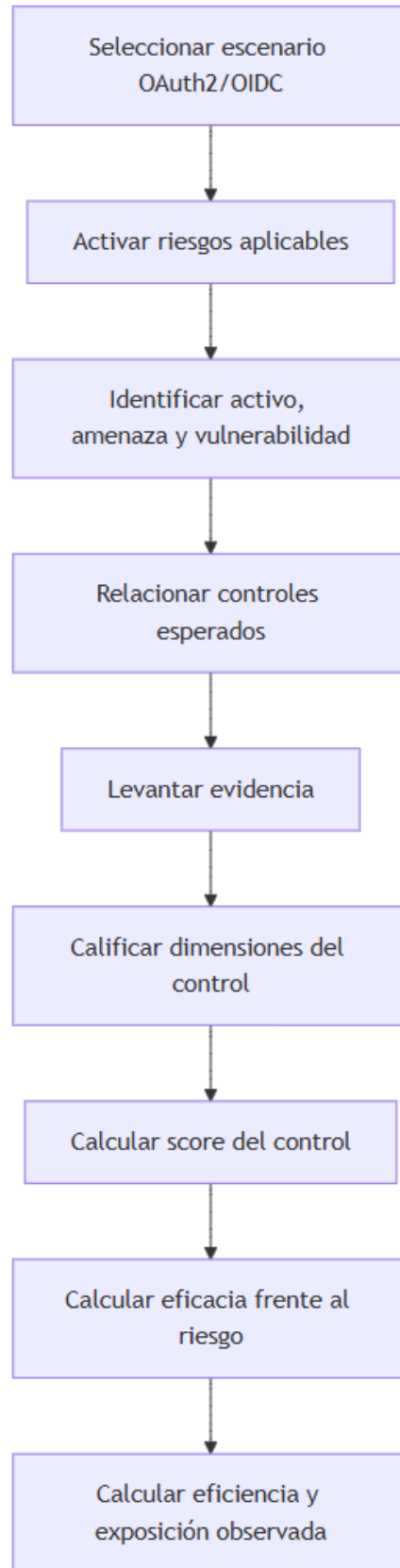


Figura 4: diagram

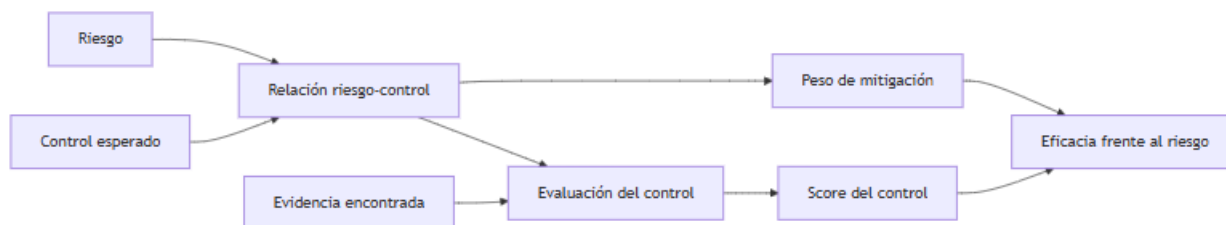


Figura 5: diagram

### 3.6. Uso del enfoque de Valencia Duque en esta tesis

La tesis toma como referencia el enfoque de aseguramiento y auditoría de TI orientado a riesgos de Valencia Duque, especialmente la idea de evaluar controles desde su relación con el riesgo y no solo desde su existencia formal.

En este trabajo, esa perspectiva se adapta al dominio de OAuth 2.0/OIDC mediante tres decisiones metodológicas:

1. **Separar riesgo, control y evidencia.**

Un riesgo no queda mitigado solo porque exista un control en el catálogo. Debe evaluarse si el control está implementado y evidenciado.

2. **Distinguir score del control y eficacia frente al riesgo.**

El score del control mide su estado observado. La eficacia frente al riesgo mide su aporte ponderado dentro de un conjunto de controles asociados al riesgo.

3. **Incorporar eficiencia como relación beneficio-coste.**

La eficiencia no se mide solo por diseño técnico, sino como relación entre beneficio de mitigación estimado y coste asignado.

La tesis no pretende reproducir literalmente un marco de auditoría completo. Su aporte consiste en adaptar el enfoque de aseguramiento orientado a riesgos a tres escenarios concretos de identidad federada.

### 3.7. Complemento con estándares y fuentes técnicas

El modelo combina aseguramiento orientado a riesgos con fuentes técnicas especializadas. Esta combinación evita dos extremos:

- una matriz genérica de riesgos sin precisión técnica;
- una lista técnica de buenas prácticas sin estructura de riesgo, control y evidencia.

**3.7.1. ISO/IEC 27001 e ISO/IEC 27002** ISO/IEC 27001 e ISO/IEC 27002 aportan el lenguaje general de sistema de gestión de seguridad de la información y controles de seguridad. En esta tesis se usan como referencia para comprender que los controles deben tener propósito, responsable, evidencia y relación con riesgos.

El modelo no busca certificar una organización en ISO/IEC 27001. Usa ese enfoque para ordenar controles relacionados con acceso, gestión de identidades, registros, configuración, criptografía, continuidad y gobierno.

**3.7.2. NIST** NIST aporta una estructura útil para caracterizar riesgos mediante amenaza, vulnerabilidad, probabilidad e impacto. También aporta familias de controles relacionadas con acceso, auditoría, identificación, autenticación, configuración, monitoreo y protección de sistemas.

En la tesis, NIST se usa para reforzar la lógica de evaluación de riesgo y la necesidad de evidencia, no para copiar un catálogo completo de controles.



**3.7.3. OWASP** OWASP aporta guías prácticas sobre seguridad de aplicaciones, APIs, sesiones y OAuth 2.0. Su utilidad principal está en traducir riesgos técnicos de implementación a controles verificables.

Ejemplos de controles influenciados por OWASP:

- uso de Authorization Code + PKCE;
- protección de cookies;
- reducción de exposición de tokens;
- validación de state y nonce;
- protección contra CSRF y XSS;
- revisión de configuración de APIs.

**3.7.4. RFCs de OAuth 2.0/OIDC** Los RFCs y especificaciones OIDC son la base técnica para definir controles esperados. En particular:

- RFC 6749 define roles, flujos y conceptos base de OAuth 2.0.
- RFC 6750 define el uso de bearer tokens.
- RFC 7636 define PKCE.
- RFC 7662 define token introspection.
- RFC 8693 define token exchange.
- RFC 9700 consolida mejores prácticas de seguridad para OAuth 2.0.
- OpenID Connect Core define el id\_token, nonce, claims y validaciones asociadas al login federado.

Estas fuentes permiten que el catálogo tenga trazabilidad técnica y no dependa solo de criterio subjetivo.

## 4. Casos arquitectónicos de contraste y escenarios de evaluación

### 4.1. Evolución arquitectónica: de cookies de sesión a OAuth 2.0 con PKCE

Antes de comparar los casos arquitectónicos, conviene presentar una lectura evolutiva. La tesis no asume que OAuth 2.0 y OpenID Connect aparecieron en un vacío técnico. Su adopción puede entenderse como una transición desde aplicaciones que resolvían autenticación y sesión de forma local, hacia arquitecturas federadas donde la autorización, los tokens y la confianza se distribuyen entre varios componentes.

Esta evolución se resume en tres etapas:

1. aplicaciones con autenticación local y cookies de sesión;
2. arquitecturas OAuth 2.0 con Authorization Code sin PKCE;
3. arquitecturas OAuth 2.0/OIDC con Authorization Code + PKCE y controles complementarios.

La importancia de esta secuencia es metodológica: permite ver qué riesgo se resuelve en cada etapa y qué riesgos nuevos aparecen al incorporar identidad federada.

**4.1.1. Etapa 1: autenticación local y cookies de sesión** En una primera etapa, la aplicación autentica directamente al usuario. El usuario entrega sus credenciales al sistema, el backend las valida contra una base local o directorio interno y, si son correctas, crea una sesión server-side. El navegador conserva una cookie que representa esa sesión.

El patrón puede representarse así:

Este modelo tiene una ventaja operativa: es simple y concentra la lógica de autenticación, autorización y sesión en un solo sistema. Sin embargo, también concentra responsabilidades y riesgos:

- la aplicación conoce o procesa directamente credenciales;
- la sesión depende de la seguridad de la cookie;
- la autorización suele estar acoplada a la lógica del sistema;
- la trazabilidad depende de los logs internos de la aplicación;
- la integración con terceros exige crear mecanismos propios o compartir credenciales, lo cual aumenta la exposición.

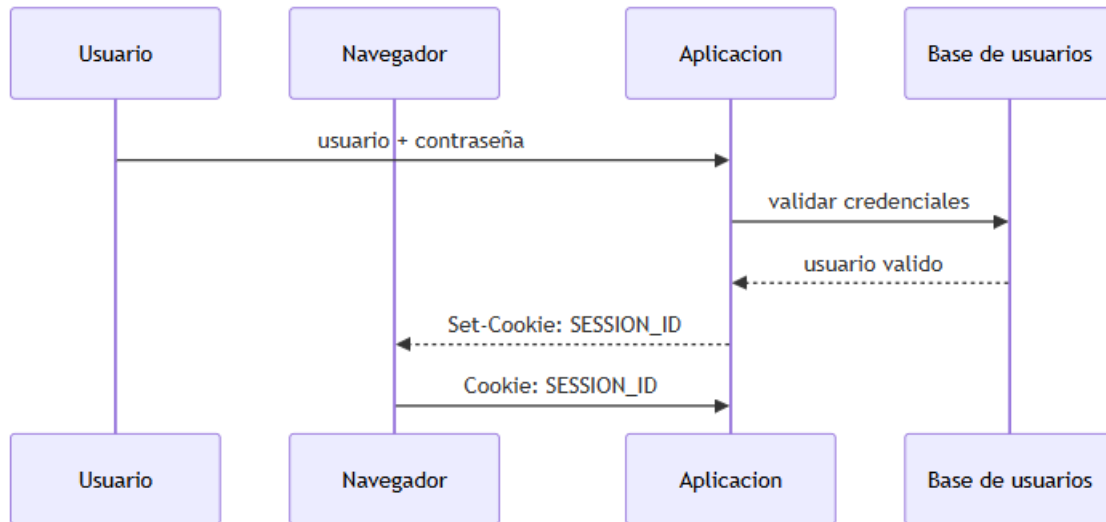


Figura 6: Etapa 1 - Autenticación local y cookies de sesión

En esta etapa, los controles críticos son la protección de contraseñas, la configuración de cookies HttpOnly, Secure y SameSite, la expiración de sesión, la protección contra CSRF y XSS, y la autorización server-side.

Los casos base 0A y 0B representan esta etapa. No usan OAuth 2.0/OIDC. Su función es servir como contraste para entender qué cambia cuando se introduce un Authorization Server.

**4.1.2. Etapa 2: OAuth 2.0 Authorization Code sin PKCE** La segunda etapa aparece cuando se introduce OAuth 2.0 como mecanismo de autorización delegada. En vez de que la aplicación gestione directamente toda la autenticación y autorización, aparece un Authorization Server que emite tokens. El cliente obtiene un `authorization_code` y luego lo intercambia por tokens en el endpoint de token.

El patrón básico es:

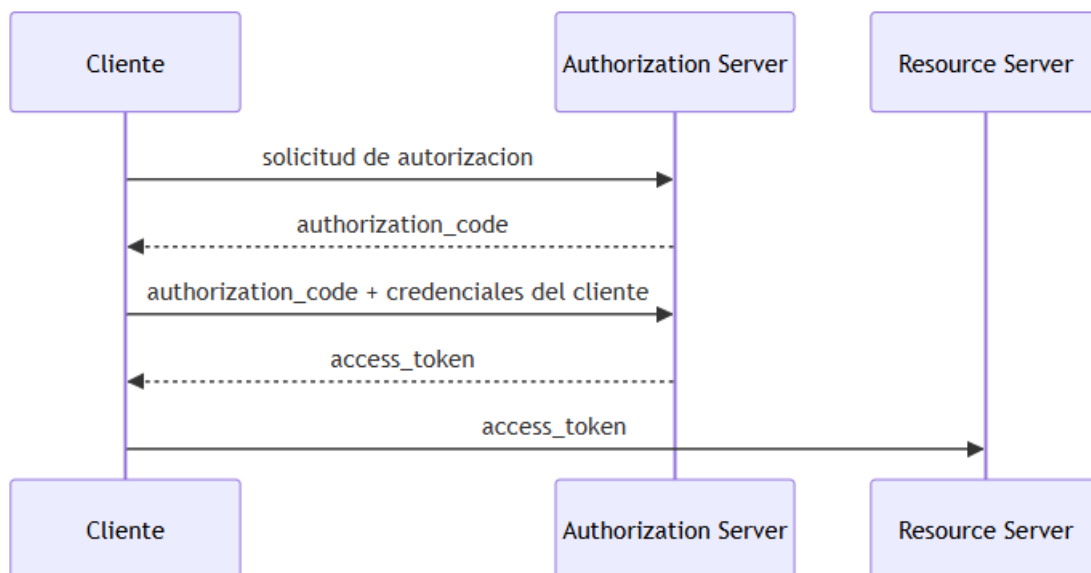


Figura 7: Etapa 2 - OAuth 2.0 Authorization Code sin PKCE

Este diseño mejora el modelo anterior porque evita que el cliente tenga que conocer directamente la contraseña del usuario y permite delegar acceso limitado mediante tokens, scopes y audiencias. También introduce una separación conceptual entre cliente, Authorization Server y Resource Server.

No obstante, el flujo Authorization Code sin PKCE deja un problema relevante: el `authorization_code` puede convertirse en un artefacto sensible. Si un atacante intercepta el código y logra presentarlo ante el token endpoint, podría obtener tokens. Esto es especialmente delicado en clientes públicos, aplicaciones móviles, SPAs u otros contextos donde el cliente no puede proteger un secreto de forma confiable.

En términos de riesgo, OAuth 2.0 sin PKCE reduce algunos problemas del modelo basado solo en credenciales y cookies, pero introduce nuevos activos que deben protegerse:

- `authorization_code`;
- `access_token`;
- `refresh_token`, si aplica;
- `client_id` y credenciales del cliente;
- redirect URI;
- scopes;
- audiencia;
- configuración del Authorization Server.

Por tanto, el salto a OAuth 2.0 no elimina la necesidad de controles de sesión. En arquitecturas con BFF, el navegador puede seguir usando cookie frente al BFF, mientras el BFF usa OAuth 2.0/OIDC hacia el Authorization Server y las APIs.

**4.1.3. Etapa 3: OAuth 2.0/OIDC con PKCE** PKCE aparece para mitigar el riesgo de interceptación del `authorization_code`. La idea central es que el código de autorización, por sí solo, no sea suficiente para obtener tokens.

Para ello, el cliente genera un secreto temporal llamado `code_verifier` y deriva de él un `code_challenge`. En la solicitud de autorización se envía el `code_challenge`; en el intercambio del código por tokens se envía el `code_verifier`. El Authorization Server verifica que el `code_verifier` corresponda al `code_challenge` registrado inicialmente.

El patrón queda así:

La consecuencia de seguridad es directa: si un atacante obtiene únicamente el `authorization_code`, no puede canjearlo por tokens porque no conoce el `code_verifier`. En esta tesis, ese es el problema específico que PKCE resuelve: reducir el riesgo de canje indebido del código de autorización interceptado.

PKCE no resuelve todos los riesgos de OAuth 2.0/OIDC. No protege automáticamente contra XSS, robo de cookie, mala validación de audience, scopes excesivos, tokens en `localStorage`, phishing de consentimiento o backends que confían ciegamente en un Gateway. Por esa razón, la matriz no pregunta solamente si existe PKCE. Evalúa también:

1. si se usa S256;
2. si se rechaza `plain` salvo justificación excepcional;
3. si el `code_verifier` tiene entropía suficiente;
4. si el código de autorización tiene expiración corta y uso único;
5. si se validan `state` y `nonce` cuando corresponde;
6. si los tokens quedan fuera del navegador en arquitecturas con BFF;
7. si las APIs validan firma, issuer, audience, expiración y scopes.

**4.1.4. Lectura para los escenarios de la tesis** La evolución anterior permite ubicar los casos de esta tesis en una secuencia conceptual:

- El **caso base 0A** representa una arquitectura monolítica con autenticación local y cookie de sesión.
- El **caso base 0B** representa una arquitectura SPA+BFF con sesión local por cookie, pero sin identidad federada.
- La **arquitectura 1** representa la incorporación de OIDC y OAuth 2.0 con Authorization Code + PKCE, conservando cookie entre navegador y BFF.
- La **arquitectura 2** representa el uso de OAuth 2.0 sin usuario humano mediante `client_credentials`.

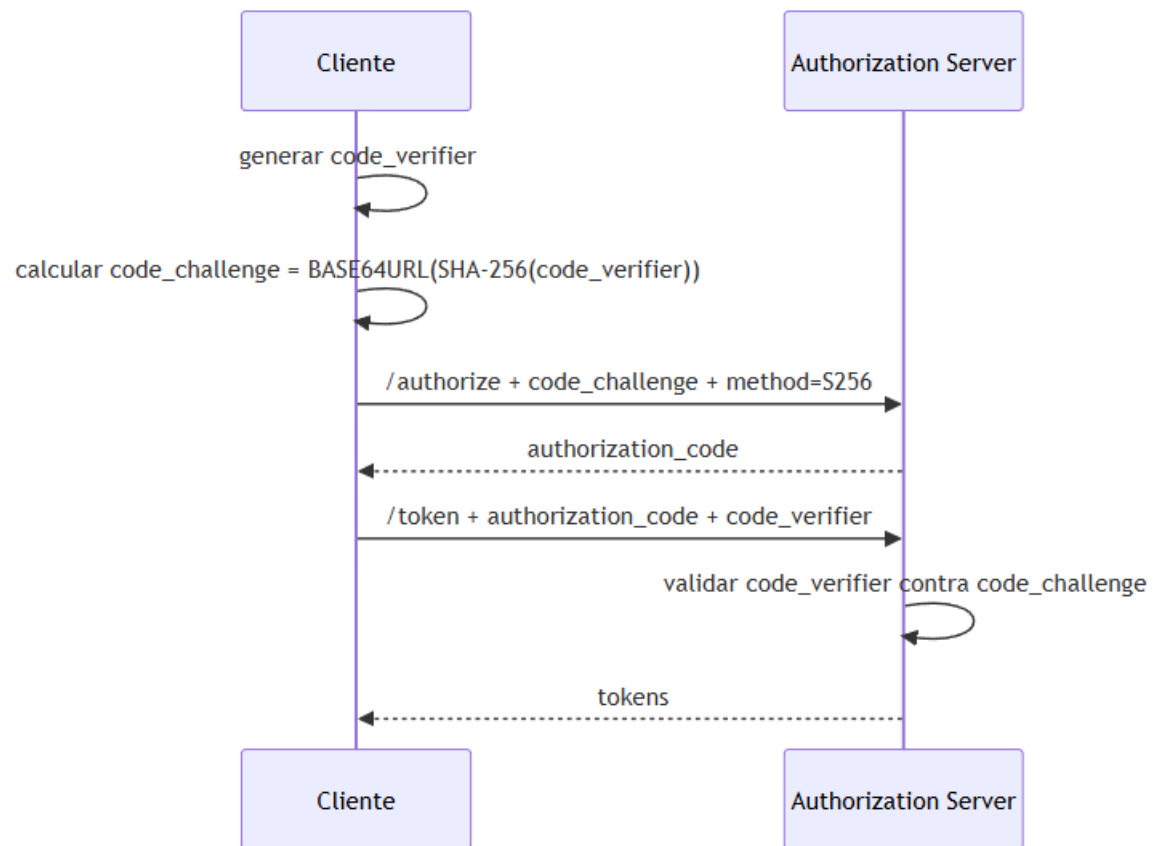


Figura 8: Etapa 3 - OAuth 2.0/OIDC con Authorization Code + PKCE

- La **arquitectura 3** representa una evolución hacia validación federada en API Gateway, Resource Servers y posible token exchange.

Esta lectura evita dos errores conceptuales. El primero es pensar que OAuth 2.0 reemplaza todas las cookies: en una arquitectura BFF, la cookie puede seguir siendo el mecanismo de sesión entre navegador y BFF. El segundo es pensar que PKCE hace segura toda la arquitectura: PKCE protege el canje del código de autorización, pero debe coexistir con controles de sesión, validación de tokens, autorización de negocio, gobierno de clientes y evidencia auditable.

## 4.2. Criterios para seleccionar los casos

La tesis no parte directamente de OAuth 2.0/OIDC. Primero introduce dos casos base sin identidad federada para contrastar qué responsabilidades de seguridad existen antes de incorporar Authorization Server, tokens, scopes, clientes OAuth y federación de identidad.

Los casos se seleccionan con base en estos criterios:

1. **Contraste arquitectónico:** incluir arquitecturas sin OAuth 2.0/OIDC y arquitecturas donde OAuth 2.0/OIDC sí cambia la distribución de responsabilidades.
2. **Diferencia de superficie de riesgo:** cada caso debe exponer riesgos distintos: sesión local, BFF, tokens, clientes técnicos, API Gateway, propagación de identidad o token exchange.
3. **Aplicabilidad de la tríada riesgo-control-auditoría:** cada caso debe permitir relacionar riesgos aplicables, controles esperados y evidencias verificables.
4. **Pertinencia para sistemas reales:** los casos se expresan con ejemplos cercanos a sistemas bancarios, comercio electrónico, portales corporativos, APIs internas y plataformas IAM.
5. **Delimitación razonable:** la tesis no pretende cubrir todos los flujos OAuth 2.0/OIDC, sino escenarios suficientemente representativos para construir una matriz evaluable.

Con estos criterios se delimitan cinco casos de análisis: dos casos base de contraste conceptual y tres escenarios OAuth 2.0/OIDC que sí serán evaluados por la matriz.

1. **Caso base 0A:** monolito con autenticación y sesión local.
2. **Caso base 0B:** tres capas con SPA estática, BFF y gestión propia de usuarios.
3. **Arquitectura 1:** SPA estática + BFF + IdP corporativo usando OAuth 2.0/OIDC.
4. **Arquitectura 2:** M2M donde el Authorization Server actúa también como Resource Server.
5. **Arquitectura 3:** API Gateway federado con Authorization Server corporativo, propagación o intercambio de tokens.

Los dos primeros casos no son escenarios OAuth 2.0/OIDC. Son líneas base para explicar qué se pierde, qué se gana y qué riesgos nuevos aparecen cuando se introduce identidad federada.

**Tipo de flujo de autenticación o autorización por caso** Esta clasificación evita una confusión conceptual: no todos los casos usan OAuth 2.0/OIDC. Los casos base usan autenticación local y sesión web; los escenarios OAuth 2.0/OIDC introducen Authorization Server, tokens, clientes OAuth, scopes, audiencia y validación de tokens.

### Caso base 0A: monolito con autenticación local

- **Tipo de flujo:** autenticación web tradicional con sesión local.
- **Protocolo principal:** HTTP + cookie de sesión.
- **No usa:** OAuth 2.0, OIDC, Authorization Server externo, access\_token, id\_token ni PKCE.
- **Autenticación:** el monolito valida usuario y contraseña contra su propia base de usuarios o directorio interno.
- **Sesión:** el monolito crea una sesión server-side y entrega una cookie SESSION\_ID al navegador.
- **Autorización:** el propio monolito aplica roles, permisos, perfiles o reglas internas de negocio.
- **Ejemplo:** portal bancario tradicional donde la misma aplicación maneja login, consultas, pagos y autorización funcional.

Lectura conceptual para el contraste: el riesgo principal no es la validación de tokens, sino la seguridad de sesión, credenciales locales, control de acceso interno, expiración de sesión, trazabilidad y protección contra CSRF/XSS.

### Caso base 0B: tres capas con SPA estática, BFF y usuarios propios

- **Tipo de flujo:** autenticación local en el BFF con sesión web por cookie.
- **Protocolo principal:** SPA ↔ BFF mediante HTTPS + cookie de sesión.
- **No usa:** OAuth 2.0/OIDC para el login del usuario.
- **Autenticación:** el BFF valida correo/usuario y contraseña contra su propia base de usuarios.
- **Sesión:** el BFF crea una sesión server-side y entrega una cookie segura a la SPA.
- **Autorización:** el BFF aplica permisos propios y decide qué operaciones expone al frontend.
- **Ejemplo:** tienda virtual regional con frontend Angular/React en CDN y BFF propio para login, pedidos, pagos e inventario.

Lectura conceptual para el contraste: aparece separación de capas, pero no identidad federada. La SPA no maneja tokens OAuth; solo conserva cookie. El foco está en cookies seguras, CSRF, CORS, CSP, control de sesión y autorización interna del BFF.

### Arquitectura 1: SPA estática + BFF + IdP corporativo

- **Tipo de flujo:** OpenID Connect Authorization Code Flow con PKCE.
- **Protocolo principal:** OIDC/OAuth 2.0 entre BFF y Authorization Server; cookie entre navegador y BFF.
- **Autenticación:** el usuario se autentica en el IdP corporativo.
- **Autorización:** el BFF recibe access\_token para consumir APIs protegidas.
- **Tokens esperados:** id\_token, access\_token y eventualmente refresh\_token.
- **PKCE:** protege el intercambio authorization\_code → tokens mediante code\_verifier y code\_challenge.
- **Sesión frontend:** la SPA no guarda tokens; usa cookie de sesión del BFF.
- **Ejemplo:** portal de pagos donde usuarios internos o clientes entran con Keycloak, Entra ID, Okta o Auth0, mientras el BFF custodia tokens server-side.

Lectura para la matriz: el riesgo se desplaza desde “contraseña local” hacia configuración OIDC, redirect URI, state, nonce, PKCE, custodia server-side de tokens, cookies seguras y control de scopes.

### Arquitectura 2: M2M con Authorization Server como Resource Server

- **Tipo de flujo:** OAuth 2.0 Client Credentials Grant.
- **Protocolo principal:** cliente técnico ↔ Authorization Server mediante token endpoint; cliente técnico ↔ API protegida mediante Bearer token.
- **Autenticación:** se autentica el cliente técnico, no un usuario humano.
- **Autorización:** el Authorization Server emite un access\_token con scopes técnicos.
- **Tokens esperados:** access\_token M2M.
- **Particularidad:** la misma plataforma puede cumplir dos roles lógicos: Authorization Server y Resource Server.
- **Ejemplo:** proceso batch de gobierno IAM que usa client\_id + secreto/certificado para consumir una API administrativa del propio Keycloak/Auth Server.

Lectura para la matriz: el riesgo dominante es abuso de identidad técnica: secretos hardcoded, cliente con permisos administrativos excesivos, falta de rotación, scopes amplios, clientes huérfanos y baja trazabilidad por client\_id.

### Arquitectura 3: API Gateway federado + Authorization Server corporativo

- **Tipo de flujo:** combinación de validación Bearer, propagación de token y/o OAuth 2.0 Token Exchange.
- **Flujo de entrada:** depende del cliente:
  - usuario final: OIDC Authorization Code + PKCE;
  - sistema o partner: OAuth 2.0 Client Credentials;
  - aplicación ya autenticada: presenta un Bearer token vigente.
- **En el Gateway:** se valida el token recibido: firma o introspección, issuer, audience, expiración y scopes.
- **Hacia APIs internas:** existen dos variantes:
  - **propagación:** el Gateway reenvía el token original hacia el backend;
  - **intercambio:** el Gateway solicita al Authorization Server un nuevo token con audiencia y scopes específicos para el dominio destino.

- **Tokens esperados:** `access_token` de entrada y, si aplica, `access_token` intercambiado por dominio.
- **Ejemplo:** aplicación de originación de crédito que entra por API Gateway y consume servicios internos de clientes, cupos, cartera, listas restrictivas y facturación.

Lectura para la matriz: el riesgo dominante es confusión de audiencia y exceso de confianza en el Gateway. El control crítico es que cada Resource Server valide su propia audiencia y que el intercambio de tokens reduzca scopes al cruzar dominios.

---

#### 4.3. Vista general de los casos

**Caso base 0A: monolito con autenticación local** Ejemplo de vida real: un **portal bancario tradicional** donde la misma aplicación maneja login, sesión, reglas de autorización, consultas de saldo y pagos de tarjeta. El usuario entra al portal, el monolito valida usuario y contraseña contra su propia base o directorio interno, crea una sesión HTTP y ejecuta las operaciones dentro del mismo despliegue.

Este caso no usa OAuth 2.0/OIDC. La seguridad se concentra en el monolito.

**Caso base 0B: SPA estática + BFF con usuarios propios** Ejemplo de vida real: una **tienda virtual regional** donde el frontend React/Angular se publica como archivos estáticos en un CDN, pero todas las llamadas de negocio pasan por un BFF. El BFF valida correo/contraseña contra su propia base de usuarios, crea una cookie de sesión y luego consulta la base de pedidos, pagos o inventario.

Este caso tampoco usa OAuth 2.0/OIDC. Se diferencia del monolito porque separa frontend, BFF y backend/datos.

**Arquitectura 1: SPA estática + BFF + IdP corporativo** Ejemplo de vida real: un **portal de pagos o portal de clientes** que no quiere administrar contraseñas localmente. La SPA consume un BFF mediante cookie, pero el BFF delega la autenticación en un IdP corporativo como Keycloak, Microsoft Entra ID, Okta o Auth0. El BFF actúa como cliente OAuth 2.0/OIDC.

En este escenario la SPA no maneja tokens OAuth. La SPA mantiene una cookie de sesión frente al BFF; el BFF obtiene y custodia tokens del lado servidor.

**Arquitectura 2: M2M con Authorization Server como Resource Server** Ejemplo de vida real: un **servicio de automatización IAM** que debe consultar o administrar información en la misma plataforma de identidad. Por ejemplo, un proceso de gobierno de identidades que consume una API administrativa del Authorization Server para listar clientes, consultar configuración o ejecutar tareas controladas. En productos como Keycloak, la plataforma de identidad expone una Admin REST API protegida, lo cual ilustra que el componente que emite tokens también puede exponer recursos protegidos.

Este escenario usa OAuth 2.0 `client_credentials`. El cliente técnico obtiene un `access_token` y lo usa contra un recurso expuesto por la misma plataforma que emitió el token.

**Arquitectura 3: API Gateway federado + Authorization Server corporativo** Ejemplo de vida real: una **aplicación de originación de crédito** o un **portal de clientes** consume APIs internas mediante un API Gateway. El Gateway valida tokens emitidos por un Authorization Server corporativo y enruta hacia servicios internos como cupos, cartera, listas restrictivas, productos, clientes o facturación.

En este escenario puede existir propagación del token original hacia APIs internas o intercambio de token para emitir un token nuevo con audiencia específica para el backend.

---

#### 4.4. Caso base 0A: monolito con autenticación y sesión local

**4.4.1. Descripción técnica** En el monolito, la aplicación concentra interfaz web, lógica de negocio, autenticación, autorización y acceso a datos. El usuario se autentica directamente contra el monolito. Si las credenciales son válidas, el monolito crea una sesión HTTP y envía una cookie al navegador.

No hay Authorization Server externo, no hay access\_token, no hay id\_token, no hay scopes OAuth y no hay federación de identidad. La autorización se aplica internamente con roles, permisos o reglas propias de la aplicación.

Este caso permite mostrar el punto de partida: antes de OAuth 2.0/OIDC, la confianza se concentra en una aplicación central. Su función es servir como línea base conceptual para contrastar riesgos de sesión, contraseñas, controles de acceso internos y trazabilidad local frente a los escenarios federados.

**4.4.2. Ejemplo de vida real** Un portal bancario monolítico permite al cliente:

1. iniciar sesión con usuario y contraseña;
2. consultar saldos;
3. pagar tarjeta de crédito;
4. descargar certificados;
5. cerrar sesión.

Todo ocurre dentro de la misma aplicación. El navegador solo conserva una cookie de sesión. El monolito decide si el usuario puede ejecutar cada operación.

#### 4.4.3. Diagrama de despliegue

#### 4.4.4. Diagrama de secuencia

**4.4.5. Activos y controles relevantes** Activos:

- credenciales locales del usuario;
- hash de contraseña;
- cookie de sesión;
- identificador de sesión;
- roles y permisos locales;
- logs de autenticación y operación;
- datos transaccionales.

Controles esperados:

- almacenamiento seguro de contraseñas con hash robusto y salt;
- cookie HttpOnly, Secure y SameSite;
- expiración e invalidación de sesión;
- protección contra fijación de sesión;
- autorización por operación;
- bloqueo o control de fuerza bruta;
- trazabilidad de login, logout y operaciones sensibles;
- protección CSRF en operaciones que cambian estado.

**4.4.6. Superficie de riesgo** Riesgos principales:

- robo de cookie de sesión;
- fijación de sesión;
- credenciales débiles o reutilizadas;
- autorización interna incompleta;
- escalamiento de privilegios dentro del monolito;
- ausencia de trazabilidad suficiente;
- concentración de responsabilidades en una sola aplicación.



### Caso base 0A - Monolito con autenticación y sesión local

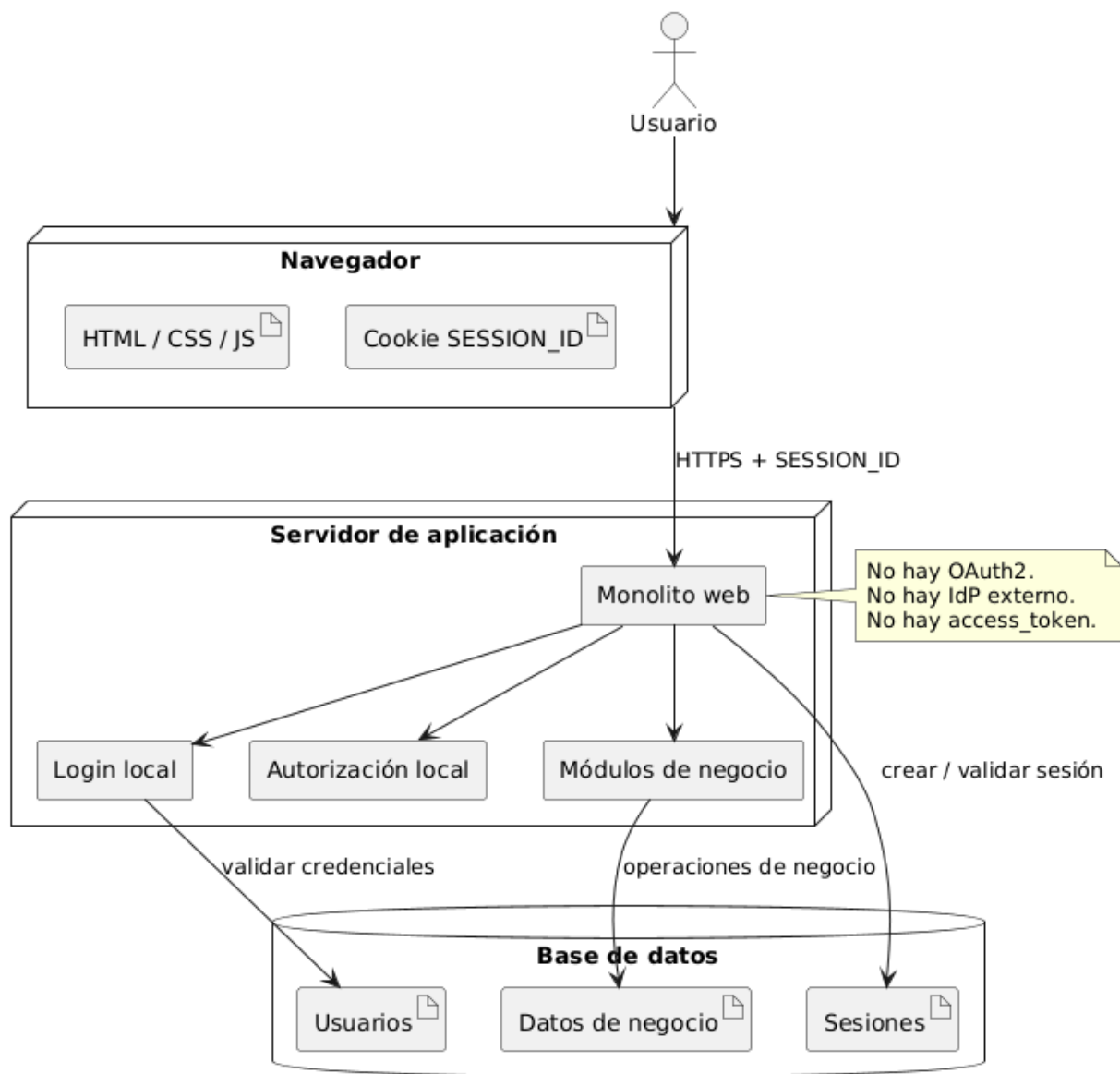


Figura 9: Caso base 0A - Monolito con sesión local

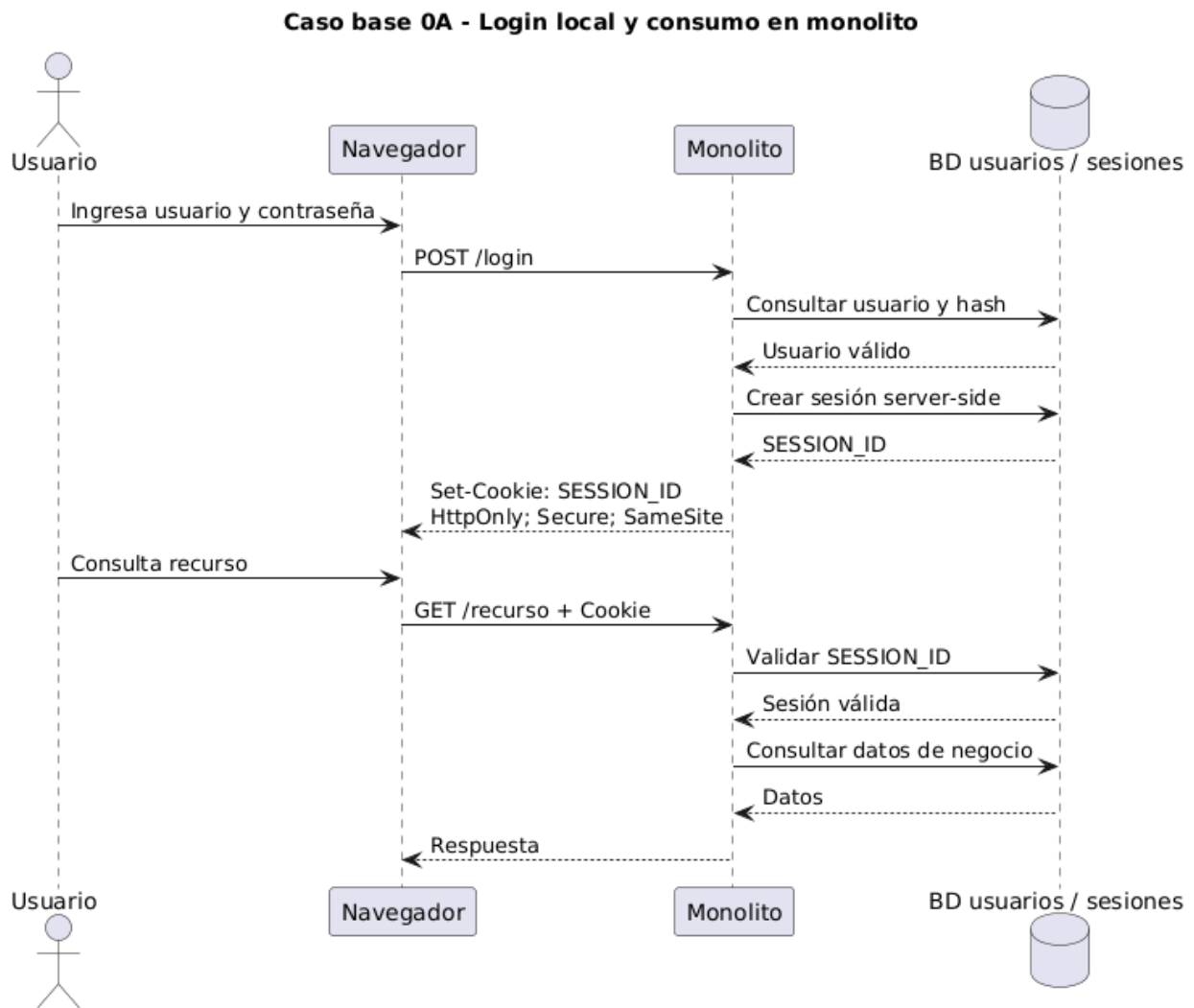


Figura 10: Caso base 0A - Login local y operación en monolito

---

#### 4.5. Caso base 0B: SPA estática + BFF con gestión propia de usuarios

**4.5.1. Descripción técnica** En este caso, el frontend está desacoplado del backend. La SPA se publica como contenido estático y consume un BFF. El BFF autentica usuarios contra su propia base, crea una sesión y entrega una cookie al navegador.

Tampoco hay OAuth 2.0/OIDC. La diferencia frente al monolito es arquitectónica: la interfaz está separada, el BFF concentra sesión y APIs de experiencia, y los servicios o bases de datos quedan detrás del BFF.

El patrón reduce exposición directa de APIs internas frente al navegador, pero introduce riesgos propios: CSRF contra el BFF, CORS mal configurado, sesión distribuida, autorización en endpoints REST y separación correcta entre frontend estático y backend sensible.

**4.5.2. Ejemplo de vida real** Una tienda virtual regional publica su frontend Angular en un CDN. El usuario inicia sesión con correo y contraseña. La SPA llama al BFF para consultar pedidos, actualizar dirección o iniciar un pago. El BFF valida la cookie y ejecuta operaciones contra servicios internos de pedidos, inventario y facturación.

#### 4.5.3. Diagrama de despliegue

#### 4.5.4. Diagrama de secuencia

#### 4.5.5. Activos y controles relevantes

 Activos:

- SPA estática;
- cookie de sesión;
- sesión server-side;
- base local de usuarios;
- endpoints del BFF;
- APIs internas;
- logs de acceso.

Controles esperados:

- cookies HttpOnly, Secure, SameSite;
- protección CSRF en endpoints del BFF;
- CORS restrictivo;
- validación server-side de sesión en cada operación;
- autorización por endpoint y recurso;
- no exponer APIs internas directamente a la SPA;
- expiración, renovación e invalidación de sesión;
- logs correlacionables entre SPA, BFF y servicios internos.

#### 4.5.6. Superficie de riesgo

 Riesgos principales:

- CSRF contra el BFF;
  - XSS en la SPA que abuse la sesión activa;
  - CORS permisivo;
  - endpoints del BFF sin autorización fina;
  - fuga de información por respuestas del BFF;
  - acoplamiento excesivo entre BFF y servicios internos;
  - controles de sesión débiles.
-

### Caso base 0B - SPA estática + BFF con gestión propia de usuarios

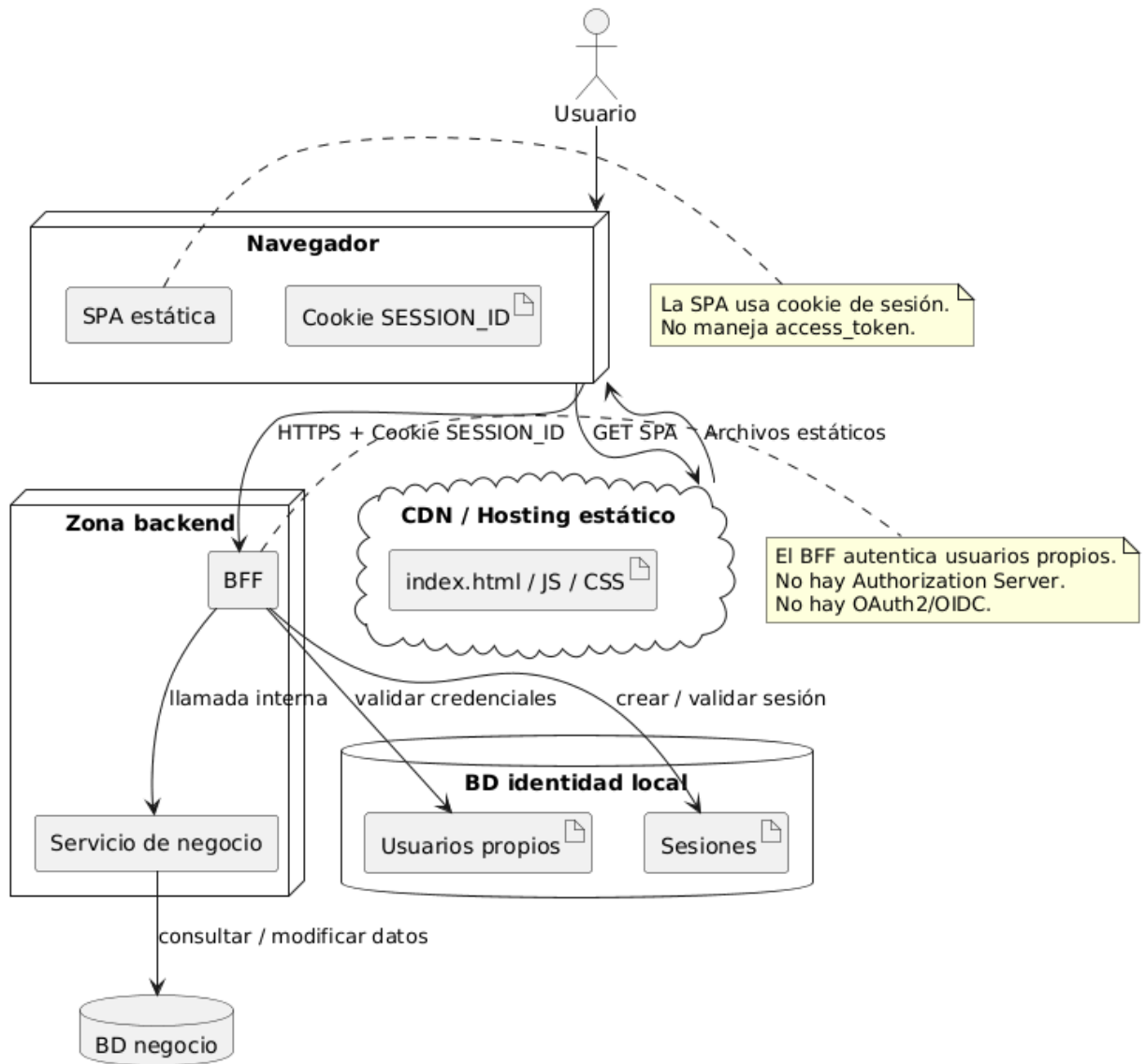


Figura 11: Caso base 0B - SPA estática + BFF con usuarios propios

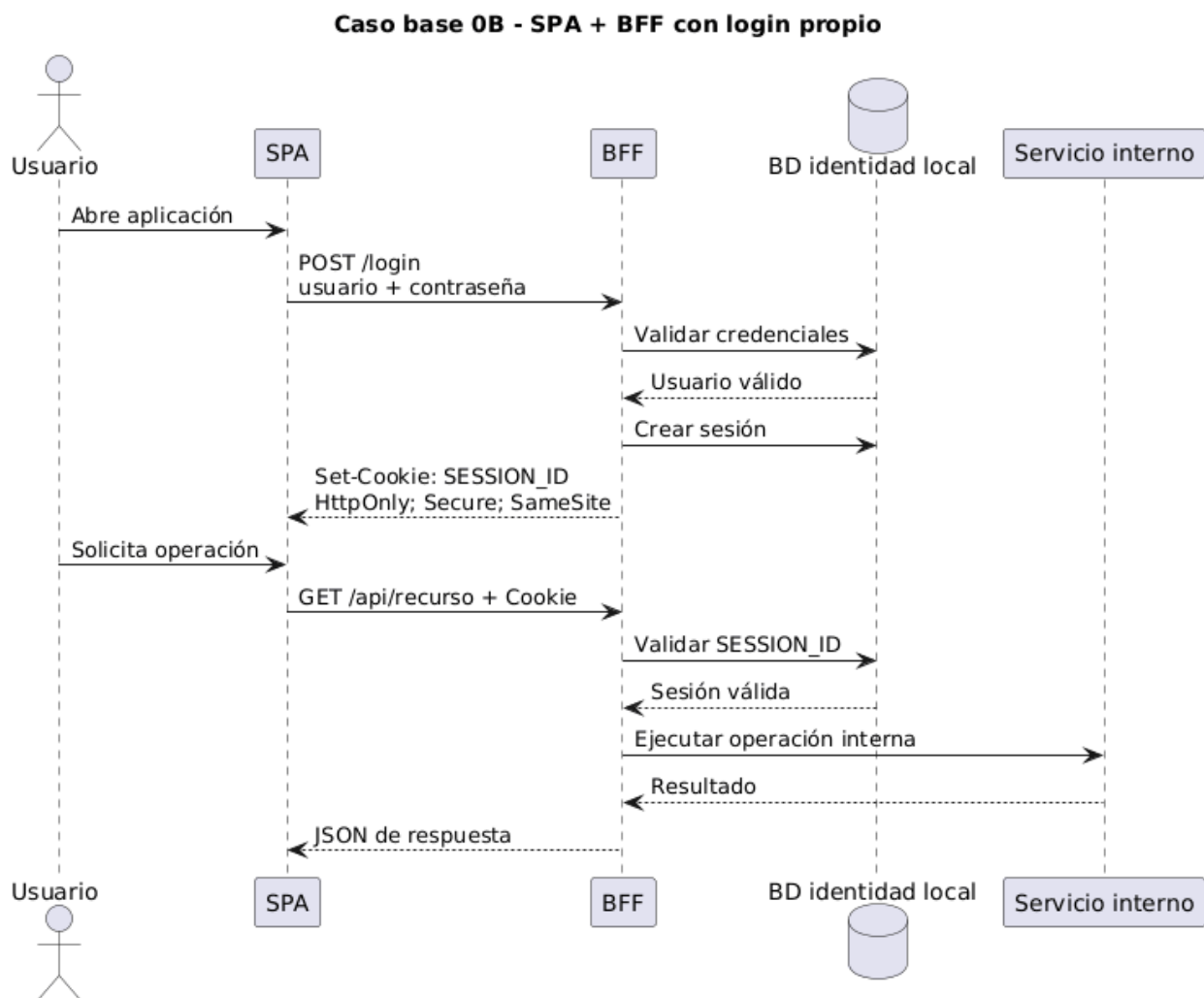


Figura 12: Caso base 0B - SPA + BFF con login propio

## 4.6. Arquitectura 1: SPA estática + BFF + IdP corporativo usando OAuth 2.0/OIDC

**4.6.1. Descripción técnica** Este escenario conserva la separación SPA + BFF, pero elimina la gestión local de contraseñas en el BFF. La autenticación del usuario se delega en un IdP corporativo. El BFF actúa como cliente confidencial OAuth 2.0/OIDC.

La SPA no ejecuta OAuth 2.0 directamente. La SPA consume el BFF con cookie de sesión. El BFF inicia el flujo Authorization Code con PKCE, recibe el `authorization_code`, lo canjea por tokens y conserva los tokens del lado servidor. La cookie representa la sesión entre navegador y BFF; los tokens representan la relación entre BFF, Authorization Server y APIs protegidas.

OAuth 2.0 define el Authorization Code Grant como un flujo en el que el cliente obtiene un código de autorización y luego lo intercambia por un token. OIDC agrega el `id_token`, que contiene claims sobre la autenticación del usuario. PKCE agrega `code_verifier` y `code_challenge` para mitigar interceptación del código.

**4.6.2. Ejemplo de vida real** Un portal de pagos de una entidad financiera publica una SPA estática para que el cliente consulte obligaciones y pague productos. El banco no quiere que el portal administre contraseñas. El BFF redirige al usuario al IdP corporativo. Tras autenticarse, el BFF crea una sesión local y llama APIs internas usando `access_token`.

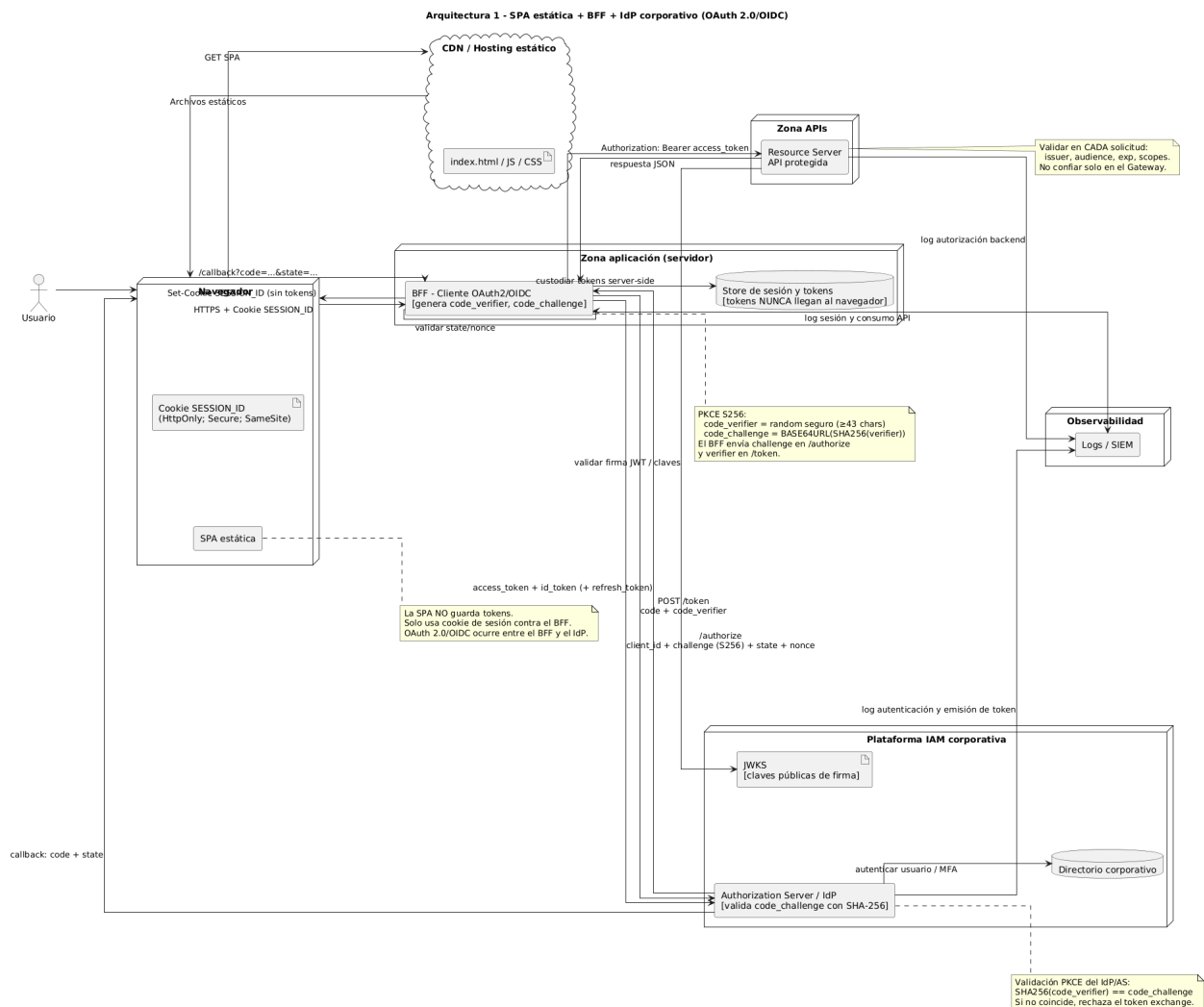


Figura 13: Arquitectura 1 - SPA estática + BFF + IdP corporativo

### 4.6.3. Diagrama de despliegue

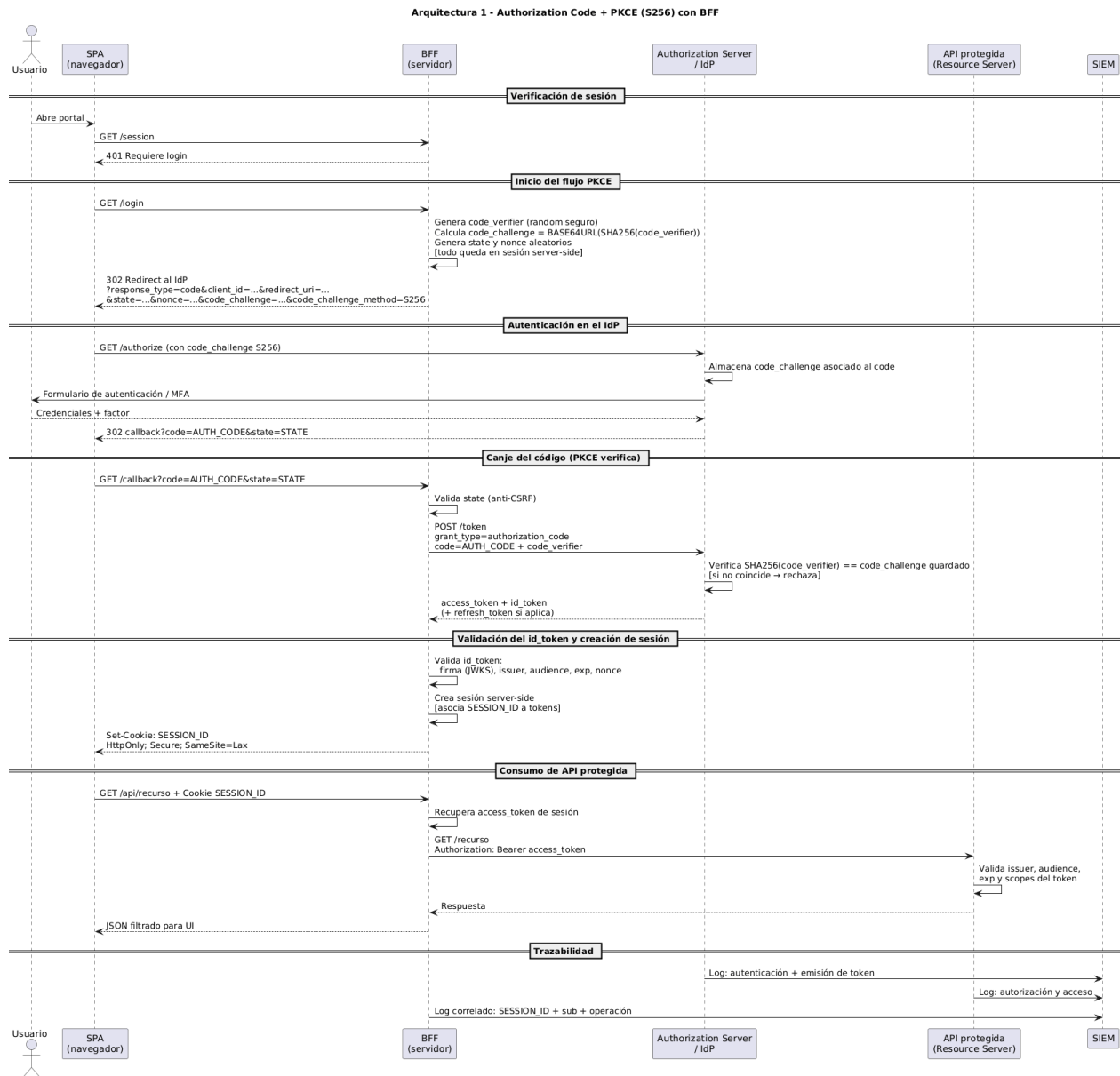


Figura 14: Arquitectura 1 - Authorization Code + PKCE con BFF

### 4.6.4. Diagrama de secuencia

### 4.6.5. Activos y controles relevantes Activos:

- authorization\_code;
- code\_verifier y code\_challenge;
- state;
- nonce;
- id\_token;
- access\_token;
- refresh\_token, si aplica;

- cookie de sesión;
- configuración del cliente OIDC;
- redirect URI;
- JWKS;
- logs de autenticación.

Controles esperados:

- Authorization Code Flow con PKCE;
- validación de `state`;
- validación de `nonce` en OIDC;
- redirect URI exacta;
- tokens almacenados server-side;
- cookie `HttpOnly`, `Secure`, `SameSite`;
- validación de `issuer`, `audience`, expiración y firma del `id_token`;
- no exposición de tokens en la SPA;
- rotación y protección de secretos del cliente, si aplica;
- trazabilidad de login, emisión de sesión y consumo de APIs.

#### 4.6.6. Superficie de riesgo Riesgos principales:

- interceptación del `authorization_code`;
- CSRF en el callback OAuth 2.0/OIDC;
- sustitución o repetición de `id_token`;
- redirect URI insegura;
- exposición de tokens en navegador por diseño incorrecto;
- robo de cookie de sesión;
- BFF con permisos excesivos hacia APIs;
- falta de correlación entre usuario, sesión, token y operación.

---

## 4.7. Arquitectura 2: M2M con Authorization Server actuando también como Resource Server

**4.7.1. Descripción técnica** Este escenario representa una comunicación servicio-a-servicio donde no hay usuario humano. Un cliente técnico se autentica ante el Authorization Server usando `client_credentials` y obtiene un `access_token`. Ese token se usa para consumir un recurso protegido que está expuesto por la misma plataforma que actúa como Authorization Server.

OAuth 2.0 define roles lógicos. El Authorization Server y el Resource Server pueden estar separados o coincidir en la misma plataforma. Por eso este escenario es válido: la misma plataforma IAM puede emitir tokens y exponer APIs protegidas, por ejemplo APIs de administración, introspección, configuración o gobierno.

**4.7.2. Ejemplo de vida real** Un proceso de gobierno IAM necesita revisar clientes OAuth registrados, scopes asignados o configuración de realms/tenants. El proceso se autentica como cliente técnico, obtiene un token por `client_credentials` y consume una API administrativa del Authorization Server. Keycloak, por ejemplo, documenta una Admin REST API bajo rutas administrativas, lo que ilustra este patrón de plataforma IAM que también expone recursos protegidos.

#### 4.7.3. Diagrama de despliegue

#### 4.7.4. Diagrama de secuencia

#### 4.7.5. Activos y controles relevantes Activos:

- `client_id`;



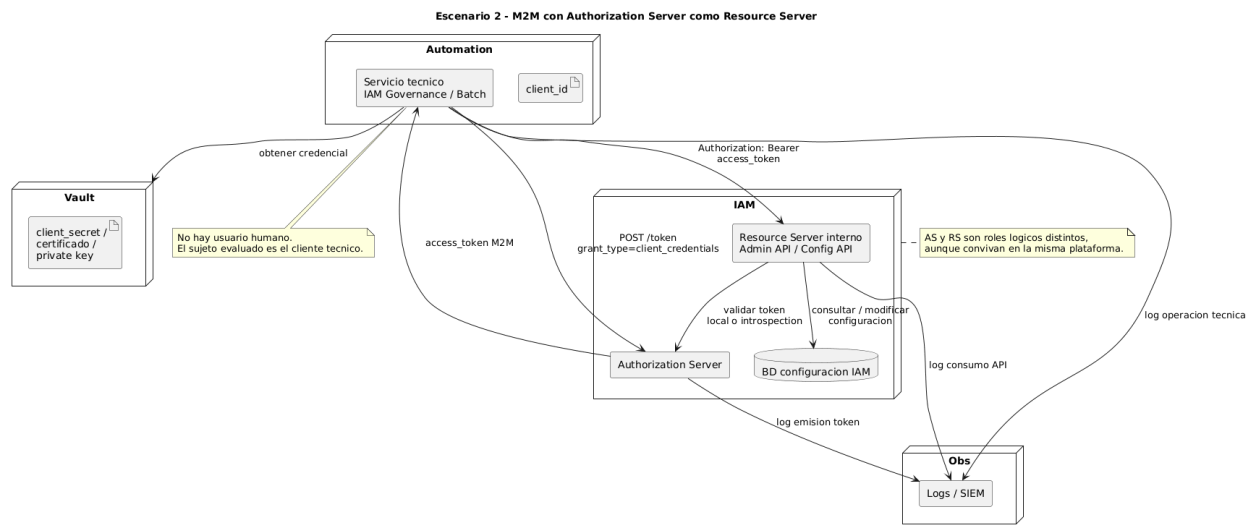


Figura 15: Arquitectura 2 - M2M con Authorization Server como Resource Server

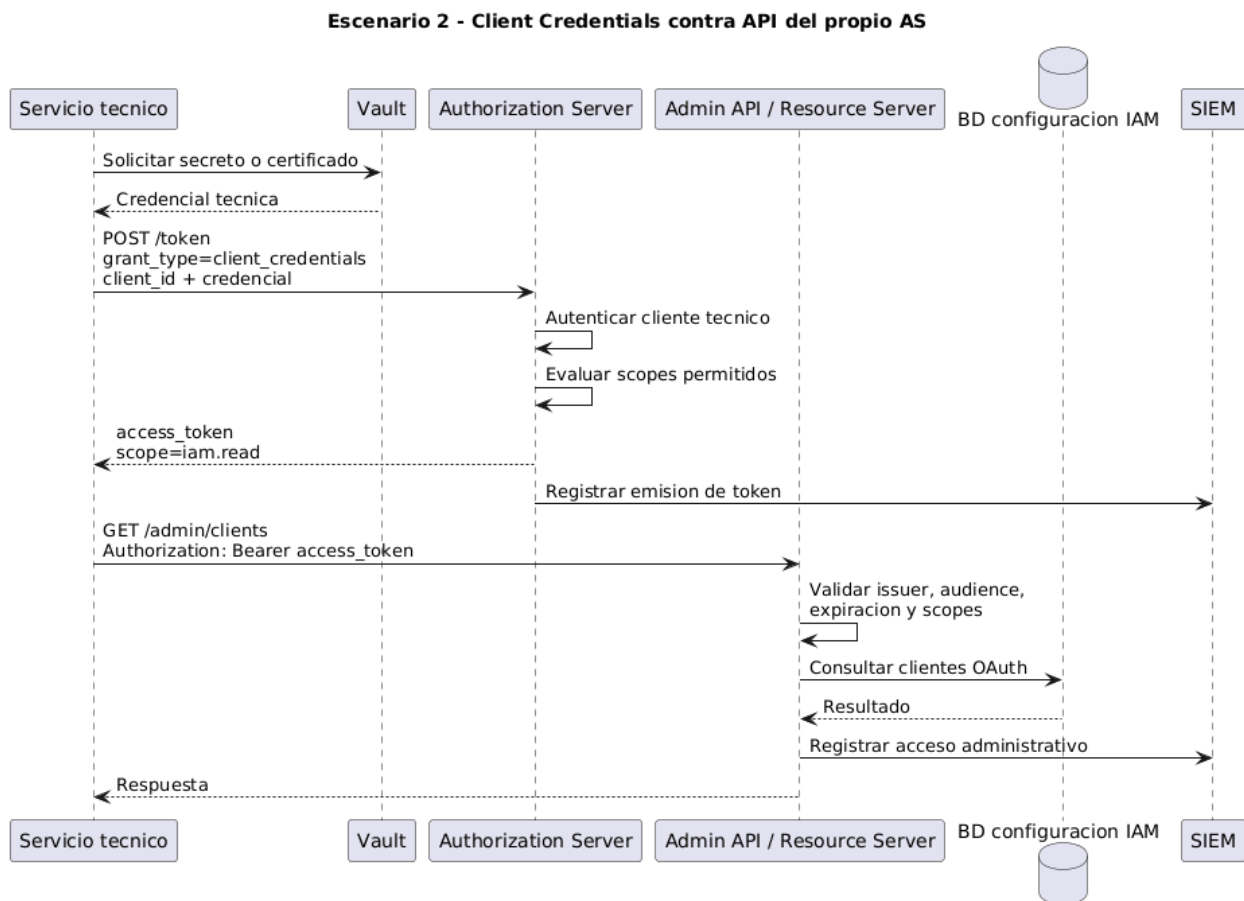


Figura 16: Arquitectura 2 - Client Credentials contra API del propio AS

- `client_secret` o certificado;
- `access_token` M2M;
- scopes técnicos;
- API administrativa o recurso protegido del AS;
- configuración IAM;
- logs de emisión y consumo;
- inventario de clientes técnicos.

Controles esperados:

- registro formal del cliente técnico;
- scopes mínimos y específicos;
- autenticación robusta del cliente;
- almacenamiento de secretos en vault;
- rotación de secretos o certificados;
- separación por ambiente y propósito;
- validación de audiencia, issuer, expiración y scopes;
- monitoreo del uso del `client_id`;
- revocación de clientes huérfanos;
- segregación de privilegios administrativos.

#### 4.7.6. Superficie de riesgo Riesgos principales:

- abuso de cliente técnico privilegiado;
- secreto hardcodeado o filtrado;
- scopes administrativos excesivos;
- falta de rotación;
- cliente técnico sin dueño;
- concentración de riesgo por colocación AS/RS;
- uso de credenciales de un ambiente en otro;
- trazabilidad insuficiente de acciones administrativas.

---

### 4.8. Arquitectura 3: API Gateway federado con Authorization Server corporativo y propagación/intercambio de tokens

**4.8.1. Descripción técnica** Este escenario representa una arquitectura donde el API Gateway actúa como punto de entrada hacia APIs internas. El Gateway valida tokens emitidos por un Authorization Server corporativo. Luego enruta la solicitud hacia uno o varios servicios backend.

El punto crítico es la frontera de confianza. No basta con que el Gateway valide el token. Debe definirse qué ocurre después:

1. **Propagación de token:** el Gateway envía el mismo `access_token` hacia el backend.
2. **Token exchange:** el Gateway intercambia el token original por otro token con audiencia y permisos específicos para el backend.
3. **Trusted subsystem:** el backend confía en el Gateway y recibe contexto de seguridad, pero este patrón exige controles fuertes de red, autenticación mutua, trazabilidad y no repudio interno.

RFC 8693 define OAuth 2.0 Token Exchange para solicitar y obtener tokens a partir de otro token. Este patrón es útil cuando cambia el destinatario del token o cuando se requiere representar delegación entre fronteras de confianza.

**4.8.2. Ejemplo de vida real** Un portal de originación de crédito recibe una solicitud del cliente. La SPA o canal llama al API Gateway. El Gateway valida el token corporativo y consulta servicios internos: cupos, cartera, listas restrictivas, perfilamiento de riesgo y datos maestros. Si el mismo token se propaga a todos los servicios, aparece riesgo de confusión de audiencia y privilegios excesivos. Si se usa token exchange, cada backend puede recibir un token con audiencia y scopes más precisos.

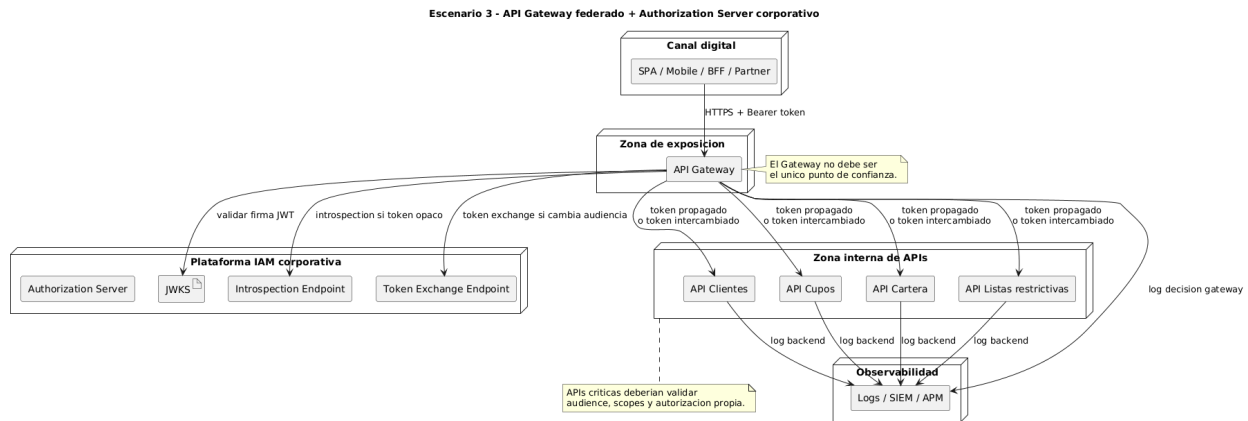


Figura 17: Arquitectura 3 - API Gateway federado + Authorization Server corporativo

#### 4.8.3. Diagrama de despliegue

#### 4.8.4. Diagrama de secuencia

#### 4.8.5. Activos y controles relevantes Activos:

- access\_token original;
- tokens intercambiados, si aplica;
- claims de usuario y cliente;
- scopes;
- audiencia (audience);
- políticas del Gateway;
- configuración de introspection o JWKS;
- APIs internas;
- logs de decisión de acceso;
- correlación de trazas entre Gateway y backends.

Controles esperados:

- validación de firma o introspection;
- validación de issuer, audience, expiración y scopes;
- autorización por operación en Gateway;
- validación mínima también en backend;
- token exchange cuando cambia la audiencia o frontera de confianza;
- restricción de propagación de tokens;
- mTLS o canal seguro Gateway-backend;
- trazabilidad de sujeto, cliente, audiencia y decisión;
- manejo seguro de errores;
- separación entre autorización técnica y autorización de negocio.

#### 4.8.6. Superficie de riesgo Riesgos principales:

- confusión de audiencia;
- propagación excesiva del token original;
- backends que confían ciegamente en el Gateway;
- pérdida de contexto de usuario;
- autorización de negocio incompleta;
- scopes demasiado amplios;
- trazabilidad fragmentada;

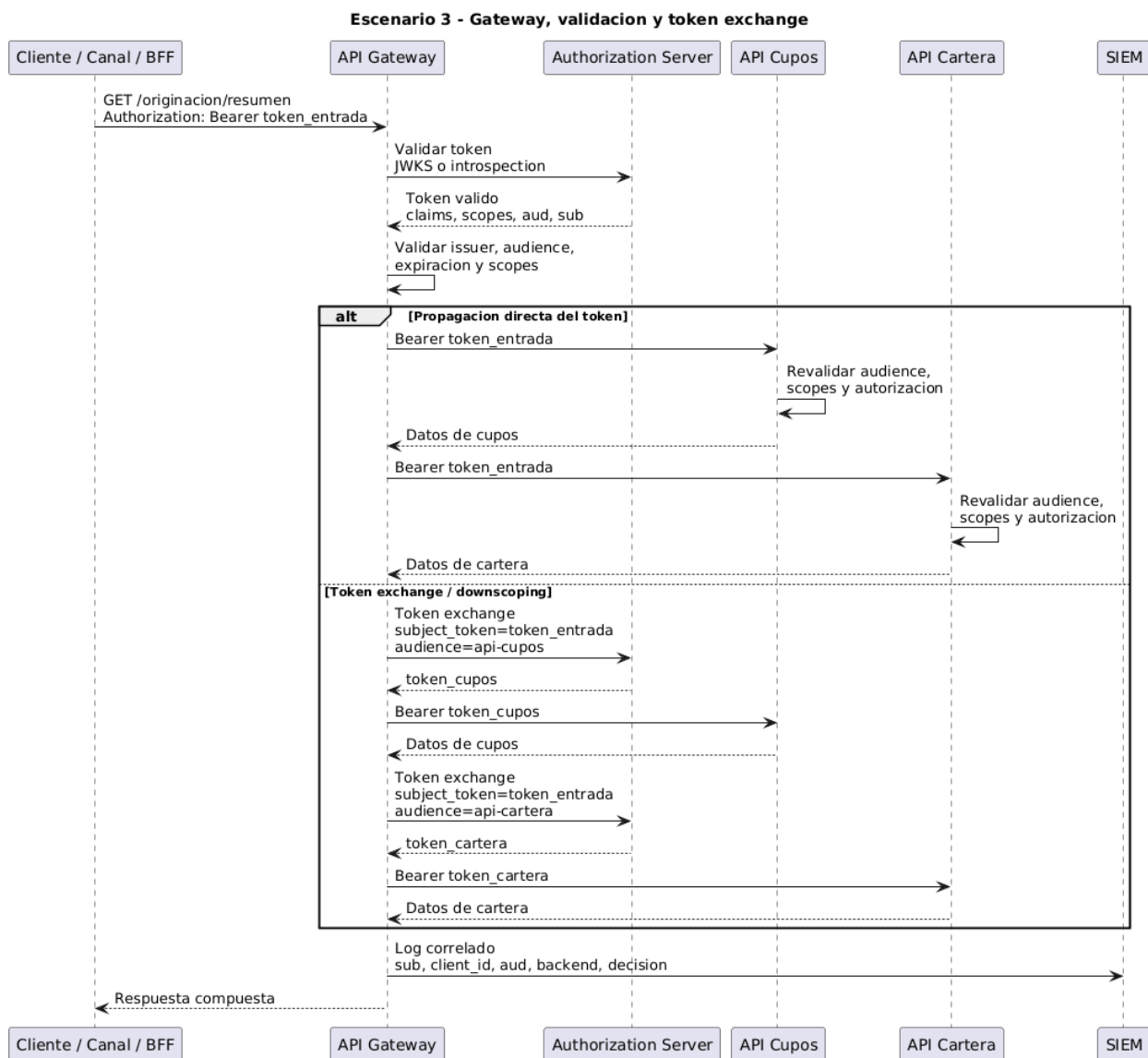


Figura 18: Arquitectura 3 - Gateway, validacion y token exchange

- abuso de relaciones de confianza entre zonas;
  - dificultad para auditar qué token autorizó qué operación.
- 

#### 4.9. Comparación sintética de los casos

##### Registro 1: Uso de OAuth 2.0/OIDC

- **Caso base 0A - Monolito:** no usa OAuth 2.0/OIDC.
- **Caso base 0B - SPA + BFF local:** no usa OAuth 2.0/OIDC.
- **Arquitectura 1 - SPA + BFF + IdP:** usa OAuth 2.0/OIDC para login federado.
- **Arquitectura 2 - M2M AS=RS:** usa OAuth 2.0 `client_credentials`.
- **Arquitectura 3 - Gateway federado:** usa OAuth 2.0/OIDC, validación de tokens y posible token exchange.

##### Registro 2: Componente que autentica

- **Monolito:** la misma aplicación.
- **SPA + BFF local:** el BFF.
- **SPA + BFF + IdP:** el IdP corporativo.
- **M2M AS=RS:** el Authorization Server autentica al cliente técnico.
- **Gateway federado:** el Authorization Server autentica/emite tokens; el Gateway valida.

##### Registro 3: Artefacto de sesión o autorización dominante

- **Monolito:** cookie de sesión.
- **SPA + BFF local:** cookie de sesión.
- **SPA + BFF + IdP:** cookie navegador-BFF y tokens BFF-IdP/API.
- **M2M AS=RS:** `access_token` M2M.
- **Gateway federado:** `access_token` validado, propagado o intercambiado.

##### Registro 4: Riesgo dominante

- **Monolito:** concentración de responsabilidades y robo de sesión.
- **SPA + BFF local:** CSRF/CORS/sesión en BFF y exposición de APIs internas por mal diseño.
- **SPA + BFF + IdP:** errores en flujo OIDC, callback, PKCE, sesión y custodia server-side de tokens.
- **M2M AS=RS:** abuso de credenciales técnicas privilegiadas.
- **Gateway federado:** confusión de audiencia, exceso de confianza y propagación indebida.

##### Registro 5: Control crítico

- **Monolito:** seguridad de sesión, hash de contraseñas, autorización local y logs.
- **SPA + BFF local:** cookie segura, CSRF, CORS restrictivo y autorización server-side.
- **SPA + BFF + IdP:** Authorization Code + PKCE, `state`, `nonce`, redirect URI exacta y tokens fuera del navegador.
- **M2M AS=RS:** secretos en vault, scopes mínimos, rotación, trazabilidad por `client_id`.
- **Gateway federado:** validación de issuer, audience, expiración, scopes y token exchange cuando aplique.

##### Registro 6: Evidencia esperada

- **Monolito:** configuración de sesión, política de contraseñas, logs y pruebas de autorización.
- **SPA + BFF local:** configuración de cookies, CSRF/CORS, logs del BFF y pruebas de endpoint.
- **SPA + BFF + IdP:** configuración OIDC, cliente, redirect URI, PKCE, logs de IdP/BFF y evidencia de tokens server-side.
- **M2M AS=RS:** inventario de clientes, scopes, vault, rotación, logs de token y consumo.
- **Gateway federado:** configuración de Gateway, JWKS/introspection, políticas de audiencia, trazas, logs y pruebas de propagación o token exchange.

## 5. Diseño del modelo de evaluación

### 5.1. Propósito del modelo de evaluación

El modelo de evaluación tiene como propósito evaluar controles de seguridad en arquitecturas basadas en OAuth 2.0 y OpenID Connect desde la relación entre riesgo, control y auditoría.

En términos instrumentales, el modelo se operacionaliza mediante una matriz de evaluación y un checklist calificable.

Como entregable aplicado, el modelo se condensará en un sitio web publicado en GitHub Pages. Este sitio funcionará como formulario de evaluación: permitirá seleccionar una de las tres arquitecturas del modelo, cargará automáticamente el catálogo predefinido de riesgos y controles esperados del escenario, recibirá los datos de evaluación diligenciados por el usuario, ejecutará la lógica de calificación definida por la tesis y generará un informe en Excel. Los algoritmos implementados en JavaScript serán una traducción operativa de las fórmulas y reglas matemáticas definidas en este documento.

El modelo no se aplica a los casos base sin OAuth 2.0/OIDC. El monolito y la arquitectura SPA + BFF con usuarios propios pertenecen al marco conceptual porque ayudan a explicar qué cambia cuando aparece identidad federada. El uso del modelo inicia cuando la arquitectura ya incorpora Authorization Server, clientes OAuth, tokens, scopes, Resource Servers o propagación de identidad.

Por tanto, la matriz se aplica a los tres escenarios OAuth 2.0/OIDC definidos en la tesis:

1. Arquitectura 1: SPA estática + BFF + IdP corporativo.
2. Arquitectura 2: M2M con Authorization Server actuando también como Resource Server.
3. Arquitectura 3: API Gateway federado + Authorization Server corporativo.

El modelo permite relacionar:

1. escenario OAuth 2.0/OIDC evaluado;
2. riesgo transversal o específico;
3. activo afectado;
4. riesgo identificado;
5. control esperado;
6. evidencia auditable;
7. dimensiones del control;
8. evaluación del control;
9. nivel de exposición observado;
10. hallazgo, brecha u observación.

**5.1.1. Diagrama de modelo de dominio** El siguiente diagrama presenta el modelo de dominio conceptual que soporta la matriz de evaluación, el catálogo maestro JSON y el entregable web. El diagrama distingue tres planos:

1. **Plano de catálogo:** contiene escenarios, filtros, riesgos, controles, relaciones riesgo-control, referencias técnicas y configuración de score.
2. **Plano de aplicación:** representa una evaluación concreta sobre una arquitectura, ambiente y versión del catálogo.
3. **Plano de resultado:** consolida registros, evidencias, evaluación por riesgo y evaluación global.

Esta separación evita mezclar el catálogo propuesto por la tesis con las evaluaciones diligenciadas por el usuario. El catálogo es estable y versionado; la evaluación cambia según el sistema, el ambiente, la evidencia disponible y el momento de revisión.

El diagrama no pretende representar una base de datos física. Representa los conceptos del dominio y sus relaciones. En particular, `RiesgoControl` existe porque la relación entre riesgo y control es muchos-a-muchos: un riesgo puede necesitar varios controles y un control puede mitigar varios riesgos. `FiltroEscenario` existe porque el catálogo maestro no se duplica por arquitectura; la UI selecciona subconjuntos de IDs según el escenario elegido. `ConfiguracionScore` existe porque la homologación cuantitativa no debe quedar embebida en el código JavaScript sin trazabilidad documental.



**5.1.2. Diccionario del modelo de dominio** El diccionario del modelo de dominio precisa qué representa cada entidad y evita ambigüedad entre catálogo, matriz, configuración y resultado. En cada concepto se incluye un ejemplo para mostrar cómo se materializa en el catálogo o en una evaluación concreta.

**CatalogoEvaluacion** Representa el artefacto maestro construido por la tesis. No corresponde a una evaluación diligenciada, sino a la base predefinida que alimenta el formulario y la matriz.

Atributos principales:

- id: identificador estable del catálogo.
- version: versión usada para trazabilidad.
- idioma: idioma del catálogo.
- fechaBase: fecha de construcción o corte.
- alcance: arquitecturas cubiertas.
- supuestoCosto: cómo interpretar los costos.
- monedaCosto: moneda usada en las estimaciones del catálogo.

Ejemplo:

```
{
  "id": "CAT-OAUTH2-OIDC-2026-01",
  "version": "1.0.0",
  "idioma": "es",
  "fechaBase": "2026-06-04",
  "alcance": "Catálogo maestro de riesgos y controles para SPA_BFF_IDP, M2M_AS_RS y API_GATEWAY_FEDERADO",
  "monedaCosto": "USD"
}
```

**Escenario** Representa una arquitectura evaluable del modelo. En esta tesis solo son escenarios evaluables las tres arquitecturas OAuth 2.0/OIDC.

Atributos principales:

- id: código del escenario.
- nombre: nombre legible de la arquitectura.
- descripcion: resumen técnico del patrón arquitectónico.

Ejemplo:

```
{
  "id": "SPA_BFF_IDP",
  "nombre": "SPA estática + BFF + IdP corporativo",
  "descripcion": "El navegador usa sesión/cookie con el BFF; el BFF ejecuta OIDC Authorization Code + PKCE c"
}
```

**FiltroEscenario** Representa la vista operativa que usa la UI para cargar solo los riesgos, controles y relaciones aplicables al escenario seleccionado. No duplica el catálogo; solo referencia identificadores.

Atributos principales:

- escenarioId: escenario al que pertenece el filtro.
- riesgosCatalogoIds: riesgos activados para el escenario.
- controlesEsperadosIds: controles disponibles para el escenario.
- riesgoControlIds: relaciones riesgo-control aplicables.

Ejemplo:

```
{
  "escenarioId": "SPA_BFF_IDP",
```



```

"riesgosCatalogoIds": ["RG-001", "RG-002", "RE1-001", "RE1-002"],
"controlesEsperadosIds": ["CG-001", "CG-006", "CE1-001", "CE1-003"],
"riesgoControlIds": ["RC-001", "RC-003", "RC-017", "RC-020"]
}

```

**RiesgoCatalogo** Representa un riesgo predefinido por la tesis. Puede ser global o específico de un escenario.

Atributos principales:

- id: identificador del riesgo.
- tipo: GLOBAL o ESPECIFICO.
- nombre: etiqueta corta del riesgo.
- descripcion: explicación del evento o condición de riesgo.
- escenarioIds: escenarios donde aplica.
- referencias: fuentes que sustentan el riesgo.

Ejemplo:

```

{
  "id": "RE1-001",
  "tipo": "ESPECIFICO",
  "escenarioIds": ["SPA_BFF_IDP"],
  "nombre": "Intercepción o sustitución del authorization_code y downgrade PKCE",
  "descripcion": "Un tercero obtiene o sustituye el authorization_code, o fuerza PKCE mal configurado, para",
  "referencias": ["RFC-7636", "RFC-9700", "OWASP-OAUTH2"]
}

```

**ControlEsperado** Representa un control que debería existir para mitigar uno o más riesgos.

Atributos principales:

- id: identificador del control.
- tipo: GLOBAL o ESPECIFICO.
- nombre: etiqueta corta del control.
- descripcion: explicación del control esperado.
- costoTotal: estimación de esfuerzo incremental.
- escenarioIds: escenarios donde aplica.
- referencias: fuentes que sustentan el control.

El costoTotal no es un precio de licenciamiento ni un costo contractual. En el catálogo se interpreta como rango estimado de esfuerzo para análisis, configuración, pruebas y despliegue. Por eso se expresa con min, max y medio. El valor medio se usa como base para las fórmulas de eficiencia, mientras que min y max permiten análisis de sensibilidad.

Ejemplo:

```

{
  "id": "CE3-002",
  "tipo": "ESPECIFICO",
  "nombre": "Token exchange o downscoping cuando cambia la frontera de confianza",
  "costoTotal": {
    "moneda": "USD",
    "min": 2000,
    "max": 8000,
    "medio": 5000
  },
  "referencias": ["RFC-9700", "KEYCLOAK-TOKEN-EXCHANGE"]
}

```

**RiesgoControl** Representa la relación muchos-a-muchos entre un riesgo y un control. Es la pieza que da valor analítico al catálogo, porque permite medir cuánto aporta un control específico frente a un riesgo específico.

Atributos principales:

- **id**: identificador de la relación.
- **riesgoId**: riesgo que se busca mitigar.
- **controlId**: control que aporta mitigación.
- **pesoMitigacion**: proporción esperada del aporte del control frente al riesgo.
- **gradoMitigacion**: lectura cualitativa del aporte esperado.
- **obligatorio**: indica si el control es crítico para ese riesgo.
- **justificacion**: razón técnica de la relación.

Ejemplo:

```
{
  "id": "RC-017",
  "riesgoId": "RE1-001",
  "controlId": "CE1-001",
  "pesoMitigacion": 0.7,
  "gradoMitigacion": "ALTO",
  "obligatorio": true,
  "justificacion": "PKCE con S256 mitiga la interceptación del authorization_code y el canje por terceros."
}
```

**ReferenciaTecnica** Representa la fuente que sustenta un riesgo, un control o una relación riesgo-control. Permite trazabilidad entre catálogo y literatura técnica.

Atributos principales:

- **id**: identificador usado por el catálogo.
- **tipo**: RFC, OWASP, NIST, OIDS o documentación oficial.
- **nombre**: nombre de la fuente.
- **seccion**: parte relevante.
- **url**: enlace público.

Ejemplo:

```
{
  "id": "RFC-7636",
  "tipo": "RFC",
  "nombre": "Proof Key for Code Exchange by OAuth Public Clients",
  "seccion": "Abstract; 1; 4; 7.2",
  "url": "https://datatracker.ietf.org/doc/html/rfc7636"
}
```

**ConfiguracionScore** Representa los valores numéricos y ponderadores usados para homologar controles. En el modelo implementado, la notación queda alineada con las ecuaciones LaTeX mostradas por el formulario web: score base, normalización, factor de evidencia y score final del control.

Atributos principales:

- **madurez**: mapeo de diseñada, declarada, implementada y auditada.
- **automatizacion**: mapeo de manual, semiautomático y automático.
- **momento**: mapeo de correctivo, detectivo y preventivo.
- **periodicidad**: mapeo de ocasional, periódico y permanente.
- **alcance**: mapeo de específico y general.
- **formulaScoreControl**: fórmula ponderada del control.
- **formulaEficaciaFrenteAlRiesgo**: agregación por riesgo.

- formulaCostoAsignado: distribución del costo compartido.

Ejemplo:

```
{
  "madurez": {
    "diseñada": 0,
    "declarada": 25,
    "implementada": 75,
    "auditada": 100
  },
  "formulaScoreControl": "S_control = norm(0.40*madurez + 0.20*automatizacion + 0.10*momento + 0.20*periodic..."
}
```

En claves JSON se evita el uso de tildes y caracteres especiales. Por eso diseñada se representa como disenada.

**EvaluacionArquitectura** Representa una aplicación concreta del modelo sobre una arquitectura, ambiente y fecha. Permite repetir evaluaciones sin modificar el catálogo.

Atributos principales:

- id: identificador de la evaluación.
- fecha: fecha del levantamiento.
- evaluador: persona o rol que aplica el modelo.
- ambiente: entorno evaluado.
- escenarioId: arquitectura seleccionada.
- catalogoId: catálogo usado.
- versionCatalogo: versión del catálogo.
- monedaEvaluacion: moneda usada para costos e impacto.
- tasaCambioReferencia: tasa usada si se convierten monedas.

Ejemplo:

```
{
  "id": "EVA-2026-001",
  "fecha": "2026-06-03",
  "evaluador": "Fredy Rosero",
  "ambiente": "Preproduccion",
  "escenarioId": "SPA_BFF_IDP",
  "catalogoId": "CAT-OAUTH2-OIDC-2026-01",
  "versionCatalogo": "1.0.0",
  "monedaEvaluacion": "USD",
  "tasaCambioReferencia": {
    "COP_USD": 4000
  }
}
```

**RegistroMatriz** Representa una fila de evaluación. No evalúa un riesgo aislado ni un control aislado, sino una relación riesgo-control aplicada a un activo y a una evidencia concreta.

Atributos principales:

- id: identificador del registro.
- riesgoControlId: relación evaluada.
- riesgoId: riesgo caracterizado.
- controlId: control evaluado.
- activoAfectado: activo o componente en riesgo.
- tipoImpacto: lucro cesante, fraude, sanción, reputacional u operativo.

- **impactoEconomicoEstimado**: estimación monetaria del impacto.
- **amenaza**: causa o actor de amenaza.
- **vulnerabilidad**: condición habilitante.
- **probabilidad**: baja, media o alta.
- **impacto**: bajo, medio, alto o crítico.
- **nivelRiesgo**: resultado cualitativo de probabilidad e impacto.
- **costoAsignado**: costo distribuido del control frente al riesgo.
- **aporteEficacia**: contribución del control a la cobertura del riesgo.
- **observacion**: conclusión breve del registro.

Ejemplo:

```
{
  "id": "RM-001-1",
  "riesgoControlId": "RC-017",
  "riesgoId": "RE1-001",
  "controlId": "CE1-001",
  "activoAfectado": "authorization_code",
  "tipoImpacto": "lucroCesante",
  "impactoEconomicoEstimado": 12500,
  "amenaza": "Atacante obtiene el authorization_code durante el callback",
  "vulnerabilidad": "PKCE ausente o aceptación de plain",
  "costoAsignado": 1650,
  "aporteEficacia": 0.455
}
```

**EvaluacionControl** Representa la calificación del control en las cinco dimensiones definidas por la metodología y la evidencia disponible. En la notación del formulario, el valor operativo `scoreControl` corresponde a  $S_{control}$ , es decir, un valor normalizado entre 0 y 1 que se muestra como porcentaje.

Ejemplo:

```
{
  "madurez": "Implementado",
  "automatizacion": "Automático",
  "momento": "Preventivo",
  "periodicidad": "Permanente",
  "alcance": "General",
  "evidenceFactor": "Independiente o auditada",
  "baseScore": 1.0,
  "scoreControl": 1.0
}
```

**Evidencia** Representa el soporte usado para asignar los valores del control. La evidencia no reemplaza el juicio del evaluador; lo respalda.

Ejemplo:

```
{
  "esperada": "Configuración del cliente OIDC con PKCE S256",
  "encontrada": "Captura de configuración del IdP y logs del BFF",
  "fuente": "Consola IdP, repositorio BFF, logs de callback",
  "suficiencia": "Suficiente"
}
```

**EvaluacionRiesgo** Agrega todos los registros asociados a un mismo riesgo. Permite calcular cobertura, eficacia frente al riesgo, costo asignado total, beneficio de mitigación estimado y exposición observada.

Ejemplo:

```
{
  "riesgoId": "RE1-001",
  "impactoEconomicoEstimado": 12500,
  "coberturaEsperada": 1,
  "coberturaObservada": 0.7425,
  "eficaciaFrenteAlRiesgo": 0.7425,
  "costoAsignadoTotal": 3750,
  "beneficioMitigacionEstimado": 9281.25,
  "eficienciaFrenteAlRiesgo": 2.48,
  "nivelExposicionObservado": "Medio"
}
```

**EvaluacionGlobal** Consolida todas las evaluaciones de riesgo de una arquitectura. Permite comparar escenarios con una misma lógica.

Ejemplo:

```
{
  "registrosEvaluados": 5,
  "riesgosEvaluados": 2,
  "scorePromedioControl": 63,
  "eficienciaCuantitativa": 1.84,
  "eficaciaCuantitativa": 0.55,
  "eficienciaCualitativa": "Es parcialmente eficiente",
  "eficaciaCualitativa": "Es parcialmente eficaz"
}
```

**5.1.3. Catálogo maestro propuesto a partir de investigación técnica** Como parte del diseño del modelo se construyó un catálogo maestro de riesgos y controles para los tres escenarios OAuth 2.0/OIDC definidos en la tesis, a partir de una revisión dirigida de fuentes técnicas y normativas. Este catálogo no es una lista libre diligenciada por el usuario. Es un insumo predefinido, versionado y trazable. Su aporte consiste en traducir fuentes dispersas sobre OAuth 2.0, OIDC, seguridad de APIs, evaluación de riesgos y controles de seguridad en una estructura evaluable.

El valor del catálogo es doble:

1. **Valor metodológico:** convierte fuentes dispersas en un conjunto estructurado de riesgos, controles y relaciones de mitigación.
2. **Valor operativo:** permite que el entregable web cargue riesgos y controles aplicables según el escenario, sin depender de que el evaluador invente la matriz desde cero.

La construcción del catálogo siguió esta lógica:

El catálogo investigado cubre:

```
{
  "riesgosGlobales": 5,
  "riesgosEspecificos": 9,
  "controlesGlobales": 8,
  "controlesEspecificos": 12,
  "relacionesRiesgoControl": 48
}
```

**Leyenda de identificadores** Los identificadores del catálogo siguen una convención simple:

- RG-\*: riesgo global OAuth 2.0/OIDC.
- RE1-\*: riesgo específico del escenario SPA+BFF+IdP.

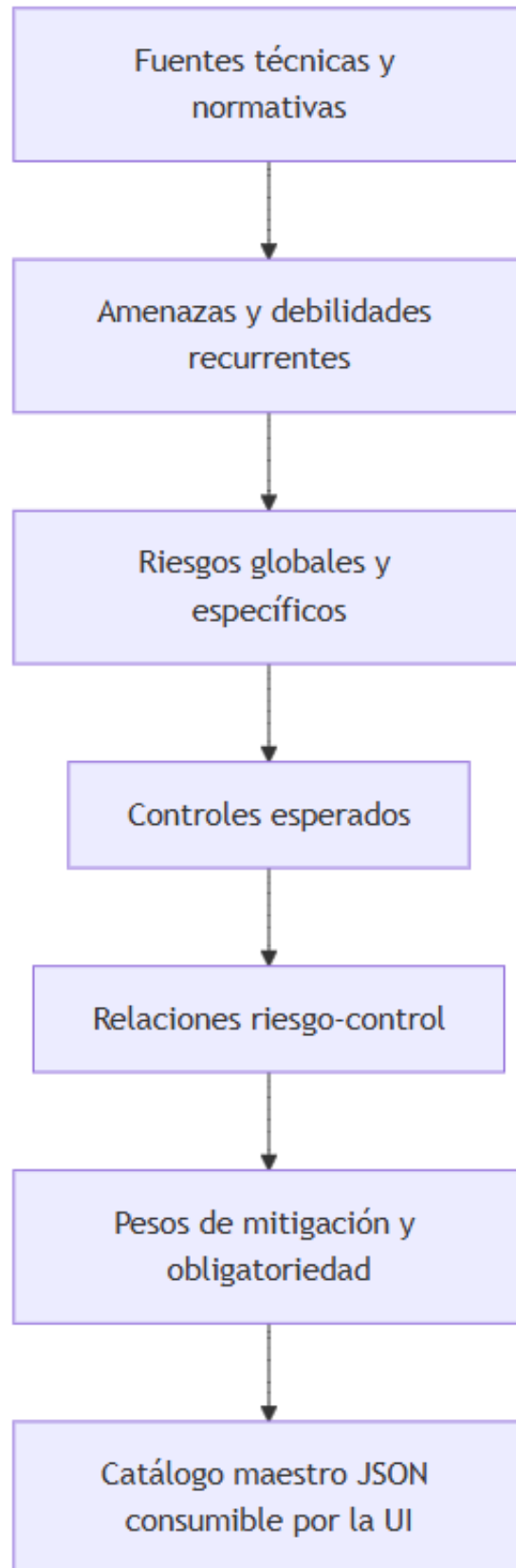


Figura 20: diagram

- RE2-\*: riesgo específico del escenario M2M AS=RS.
- RE3-\*: riesgo específico del escenario API Gateway federado.
- CG-\*: control global OAuth 2.0/OIDC.
- CE1-\*: control específico del escenario SPA+BFF+IdP.
- CE2-\*: control específico del escenario M2M AS=RS.
- CE3-\*: control específico del escenario API Gateway federado.
- RC-\*: relación riesgo-control.

**Validaciones mínimas del catálogo** Para que el catálogo sea trazable y no solo una colección artesanal de objetos JSON, debe cumplir validaciones mínimas:

1. Todo `riesgoControl.riesgoId` debe existir en `riesgosCatalogo`.
2. Todo `riesgoControl.controlId` debe existir en `controlesEsperados`.
3. Todo `filtroEscenario` debe referenciar solo IDs válidos.
4. Todo control relacionado con un riesgo debe tener `costoTotal` definido.
5. Toda referencia usada por riesgos, controles o relaciones debe existir en `referenciasTecnicas`.
6. Para cada riesgo, la suma de `pesoMitigacion` de sus relaciones aplicables debería ser 1.0, salvo justificación documentada.
7. Los costos e impactos usados en una evaluación concreta deben estar expresados en una misma moneda antes de calcular eficiencia.

**Parafraseo de fuentes usadas para construir el catálogo** La selección de riesgos y controles se fundamenta en las siguientes lecturas técnicas:

- **RFC 9700** actualiza y extiende la guía de seguridad de OAuth 2.0 frente a amenazas modernas. Por eso se usa como fuente transversal para controles de seguridad actuales, restricción de privilegios, protección de flujos redirigidos y prevención de replay.
- **RFC 6750** establece el uso de bearer tokens y advierte que exponer tokens en URI tiene debilidades de seguridad, especialmente por logs e historial. De allí provienen controles como usar `Authorization: Bearer` y evitar tokens en query string.
- **RFC 7636** explica el ataque de interceptación del código de autorización y la mitigación mediante `code_verifier` y `code_challenge`. También justifica exigir S256 para evitar métodos que expongan el verifier.
- **RFC 7662** define introspección de tokens y el campo `active`, útil para validar tokens opacos o consultar estado, expiración, cliente, scopes, audiencia y emisor.
- **OpenID Connect Core** define el `nonce` como mecanismo para asociar la sesión del cliente con el `id_token` y mitigar replay. Por eso aparece como control crítico en el escenario SPA+BFF+IdP.
- **OWASP OAuth2 Cheat Sheet** refuerza el uso de Authorization Code + PKCE, el uso de state o nonce según el flujo, la restricción de privilegios y la prevención de replay.
- **OWASP Session Management Cheat Sheet** justifica controles de cookie como `HttpOnly`, `Secure` y `SameSite`, especialmente cuando la SPA opera con sesión frente al BFF.
- **Microsoft Entra ID** enfatiza que los clientes deben tratar los access tokens como opacos y que las APIs deben validar tokens destinados a ellas, en especial la audiencia `aud`.
- **Keycloak** documenta introspección y token exchange, incluyendo filtrado por `audience` y `downscoping`, lo cual alimenta los controles del escenario API Gateway federado.
- **Auth0** recomienda proteger claves, usar HTTPS, no incluir datos sensibles innecesarios en tokens y definir expiración, lo cual alimenta controles de secretos, transporte, claims mínimos y vigencia.
- **NIST SP 800-30** aporta la estructura de evaluación de riesgo basada en amenaza, vulnerabilidad, probabilidad e impacto.
- **NIST SP 800-53** aporta familias de controles como acceso, auditoría, configuración, identificación/autenticación, protección del sistema y monitoreo.

El catálogo no busca copiar literalmente esas fuentes. Su aporte consiste en traducirlas a una estructura evaluable para tres arquitecturas concretas.

**Riesgos globales OAuth 2.0/OIDC** Los riesgos globales aplican a cualquier arquitectura del modelo que use OAuth 2.0/OIDC:

1. Exposición o replay de access\_token bearer.
2. Validación insuficiente de token.
3. Scopes, recursos o audiencias sobreasignados.
4. Gobierno débil de clientes, secretos y claves.
5. Trazabilidad y monitoreo insuficientes.

**Riesgos específicos por escenario** Para **SPA estática + BFF + IdP corporativo**, el catálogo incluye:

1. Intercepción o sustitución del authorization\_code y downgrade de PKCE.
2. Exposición de tokens o material de sesión en el navegador.
3. Callback OAuth/OIDC vulnerable por state, nonce o CSRF incorrectos.

Para **M2M con Authorization Server como Resource Server**, el catálogo incluye:

1. Abuso de cliente técnico privilegiado.
2. Secreto, certificado o clave M2M expuestos o sin rotación.
3. Segregación insuficiente cuando el Authorization Server actúa también como Resource Server.

Para **API Gateway federado + Authorization Server corporativo**, el catálogo incluye:

1. Confusión de audiencia entre Gateway y APIs internas.
2. Confianza ciega del backend en el Gateway.
3. Propagación de token sin downscoping o sin token exchange.

**Controles globales y específicos** Los controles globales definidos por el catálogo son:

1. Validación completa de token en Resource Server o Gateway.
2. Scopes mínimos y restricción de audience o recurso.
3. Política de vigencia y refresh tokens basada en riesgo.
4. Gobierno de clientes OAuth.
5. Gestión segura de secretos, claves y JWKS.
6. Transporte seguro y no exposición de tokens en URI.
7. Logging, correlación y monitoreo de autenticación y autorización.
8. Tokens sender-constrained para casos de alto riesgo.

Los controles específicos complementan esos controles globales según la arquitectura seleccionada:

- En **SPA+BFF+IdP**, el foco está en PKCE con S256, state, nonce, custodia server-side de tokens y cookie segura.
- En **M2M AS=RS**, el foco está en credenciales protegidas en vault, clientes por propósito, autenticación fuerte y recertificación por client\_id.
- En **API Gateway federado**, el foco está en audiencia por API destino, revalidación mínima en backend, token exchange o downscoping y trazabilidad Gateway-AS-RS.

**5.1.4. Estructura JSON del catálogo maestro** En el repositorio del entregable web, el catálogo puede materializarse como archivos JSON separados para facilitar mantenimiento y trazabilidad:

```
data/
├─ catalogo/
│  ├─ catalogo.json
│  ├─ escenarios.json
│  ├─ riesgos.json
│  ├─ controles.json
│  ├─ riesgo-controles.json
│  ├─ referencias.json
│  └─ mapeo-score.json
```



```

├─ filtros/
│   ├── spa-bff-idp.json
│   ├── m2m-as-rs.json
│   └─ api-gateway-federado.json
├─ evaluaciones/
│   └─ ejemplo-spa-bff-idp.json
└─ catalogo-maestro.json

```

Los archivos separados son la forma mantenible del catálogo. El archivo `catalogo-maestro.json` funciona como un bundle autocontenido para pruebas, demostración, revisión académica o carga rápida de la UI con un único `fetch()`.

El catálogo maestro tiene esta estructura:

```

{
  "catalogoEvaluacion": {},
  "escenarios": [],
  "riesgosCatalogo": [],
  "controlesEsperados": [],
  "riesgoControles": [],
  "referenciasTecnicas": [],
  "mapeoScoreInicial": {},
  "filtros": {}
}

```

**Filtros por escenario** El catálogo maestro no se duplica por arquitectura. Cada escenario tiene un filtro que referencia los IDs aplicables:

```

{
  "escenarioId": "SPA_BFF_IDP",
  "riesgosCatalogoIds": [
    "RG-001",
    "RG-002",
    "RG-003",
    "RG-004",
    "RG-005",
    "RE1-001",
    "RE1-002",
    "RE1-003"
  ],
  "controlesEsperadosIds": [
    "CG-001",
    "CG-002",
    "CG-003",
    "CG-004",
    "CG-005",
    "CG-006",
    "CG-007",
    "CG-008",
    "CE1-001",
    "CE1-002",
    "CE1-003",
    "CE1-004"
  ]
}

```

Con esta estructura, la UI realiza el siguiente flujo:

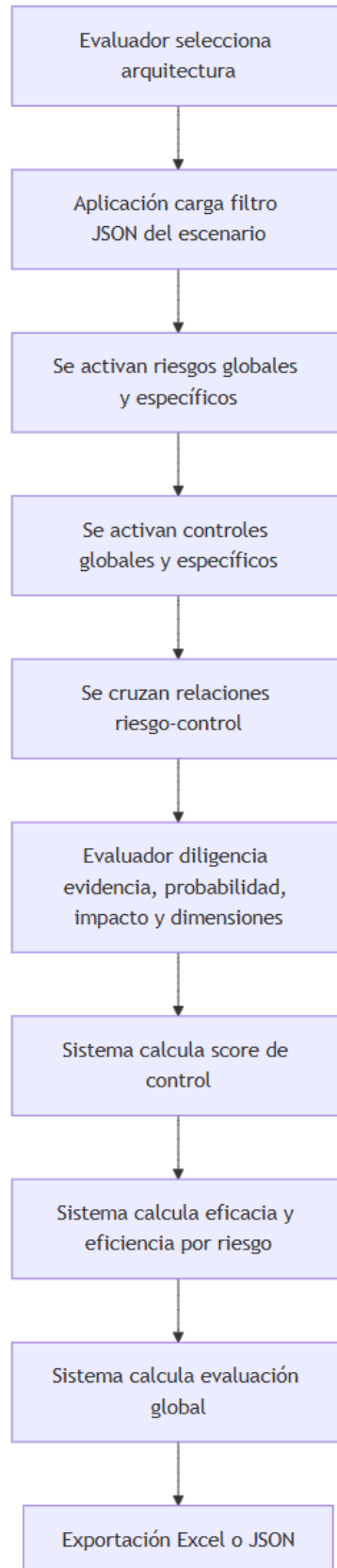


Figura 21: diagram

**5.1.5. Peso de mitigación y costo compartido** pesoMitigacion expresa el aporte esperado de un control frente a un riesgo. No indica que el control esté implementado ni que sea eficaz. Solo indica cuánto debería pesar ese control en la mitigación del riesgo si se implementa y evidencia correctamente.

Ejemplo:

```
{
  "id": "RC-017",
  "riesgoId": "RE1-001",
  "controlId": "CE1-001",
  "pesoMitigacion": 0.7,
  "gradoMitigacion": "ALTO",
  "obligatorio": true
}
```

Lectura:

Para el riesgo RE1-001, PKCE con S256 representa el 70 % de la mitigación esperada. No basta con que existan controles complementarios si este control principal no está implementado.

Para el mismo riesgo, el catálogo puede tener otros controles:

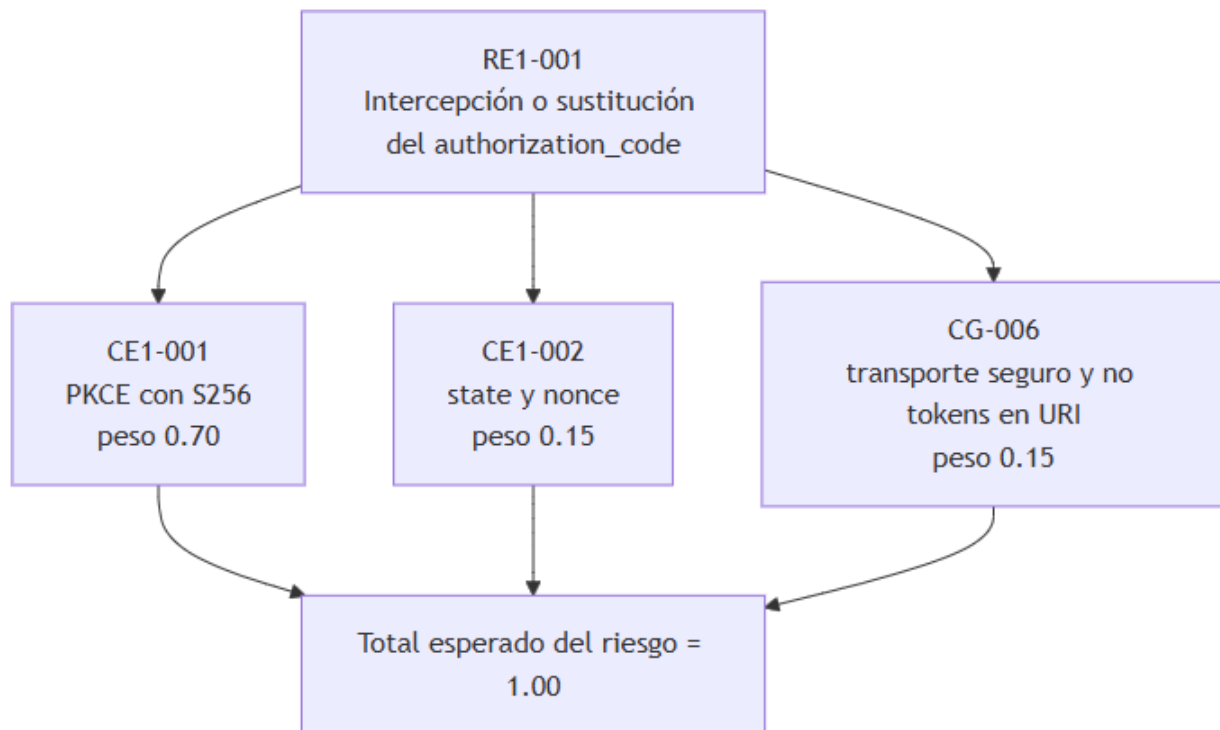


Figura 22: diagram

Esta estructura evita una evaluación binaria. Si PKCE falla, el riesgo no queda cubierto, pero los controles complementarios pueden aportar mitigación residual.

**Regla de costo compartido** Cuando un mismo control mitiga varios riesgos, su costo no debe contarse completo en cada riesgo. Si se hiciera así, la matriz inflaría artificialmente el costo total.

La regla propuesta es:

```

costoAsignado(r,c) =
costoMedio(c) * pesoMitigacion(r,c)
/
suma(pesoMitigacion(x,c) para todos los riesgos activos x del escenario)

```

Donde:

- r: riesgo evaluado.
- c: control evaluado.
- costoMedio(c): valor medio de costoTotal del control.
- pesoMitigacion(r,c): peso del control frente al riesgo.
- x: otros riesgos activos del escenario mitigados por el mismo control.

Ejemplo con CE3-002, token exchange o downscoping:

costoMedio(CE3-002) = 5000 USD

RE3-001 usa CE3-002 con peso 0.35

RE3-003 usa CE3-002 con peso 0.50

denominador = 0.35 + 0.50 = 0.85

costoAsignado(RE3-001, CE3-002) = 5000 \* 0.35 / 0.85 = 2058.82 USD

costoAsignado(RE3-003, CE3-002) = 5000 \* 0.50 / 0.85 = 2941.18 USD

La suma de costos asignados conserva el costo medio original del control. Así se evita duplicar el costo cuando el mismo control beneficia varios riesgos.

**Moneda de evaluación** El catálogo expresa costoTotal en USD. Si el impacto económico del riesgo se estima en COP, debe convertirse antes de calcular eficiencia. La evaluación concreta debe declarar monedaEvaluacion y, si aplica, tasaCambioReferencia.

Ejemplo:

```

{
  "monedaEvaluacion": "USD",
  "tasaCambioReferencia": {
    "COP_USD": 4000
  },
  "impactoEconomicoOriginal": {
    "valor": 50000000,
    "moneda": "COP"
  },
  "impactoEconomicoEstimado": {
    "valor": 12500,
    "moneda": "USD"
  }
}

```

Esta regla evita dividir beneficios estimados en COP entre costos estimados en USD.

**5.1.6. Notación matemática del score, ponderadores y fórmulas implementadas en la UI** El formulario web no solo calcula resultados, sino que también muestra ecuaciones en LaTeX para que el evaluador vea cómo se obtiene el score de cada control, la eficacia por riesgo, la eficiencia por riesgo y los promedios globales. Por esta razón, la tesis adopta en esta sección la misma notación usada por el formulario.

La notación se organiza en cuatro niveles:

1. **Nivel de dimensión del control:** madurez, automatización, momento, periodicidad, alcance funcional y evidencia.
2. **Nivel de control:** cálculo de  $(S_{\{base(raw)\}})$ ,  $(S_{\{base\}})$ ,  $(F_{\{ev\}})$  y  $(S_{\{control\}})$ .
3. **Nivel riesgo-control:** cálculo del aporte ponderado ( $A_{\{i\}}$ ) mediante el peso ( $w_{\{i\}}$ ).
4. **Nivel de riesgo y evaluación global:** cálculo de  $(E_r)$ ,  $(\square_r)$ ,  $(E_{\{global\}})$  y  $(\square_{\{global\}})$ .

**Variables principales** Para un control (c), se usan las siguientes variables:

- $(M_c)$ : valor numérico de la **madurez** del control.
- $(A_c)$ : valor numérico de la **automatización** del control.
- $(T_c)$ : valor numérico del **momento** del control.
- $(P_c)$ : valor numérico de la **periodicidad** del control.
- $(L_c)$ : valor numérico del **alcance funcional** del control.
- $(f_{\{ev\}}(c))$ : valor bruto asociado a la suficiencia de evidencia.
- $(F_{\{ev\}}(c))$ : factor de evidencia normalizado.
- $(S_{\{base(raw)\}}(c))$ : score base antes de normalización.
- $(S_{\{base\}}(c))$ : score base normalizado.
- $(S_{\{control\}}(c))$ : score final del control.
- $(w_i)$ : peso de mitigación de la relación entre el riesgo y el control (i).
- $(A_{\{i\}})$ : contribución ponderada del control (i) frente a un riesgo específico.

**Ponderadores de las dimensiones del control** En esta versión, el formulario y la tesis usan los siguientes ponderadores:

$$\alpha_M = 0.40, \quad \alpha_A = 0.20, \quad \alpha_T = 0.10, \quad \alpha_P = 0.20, \quad \alpha_L = 0.10$$

Por tanto:

$$\alpha_M + \alpha_A + \alpha_T + \alpha_P + \alpha_L = 1$$

La fórmula base queda:

$$S_{base(raw)}(c) = 0.40M_c + 0.20A_c + 0.10T_c + 0.20P_c + 0.10L_c$$

Estos ponderadores no pretenden ser universales para cualquier organización. Son una decisión metodológica del modelo propuesto y deben interpretarse como una escala semicuantitativa para comparar controles de forma consistente dentro de los tres escenarios evaluados.

**Justificación de los ponderadores de dimensión** La asignación de ponderadores responde a la importancia relativa de cada dimensión en una evaluación de aseguramiento orientada a riesgos.

**\*\*Madurez ( $\square_M=0.40$ ).**

Es la dimensión de mayor peso porque indica si el control pasó de una intención o declaración a una implementación verificable. En una evaluación de controles, un control meramente declarado no debería recibir una calificación cercana a un control implementado o auditado. Por eso la madurez concentra el 40 % del score base.

**\*\*Automatización ( $\square_A=0.20$ ).**

La automatización reduce dependencia de actividades manuales, disminuye variabilidad operativa y mejora repetibilidad. En arquitecturas OAuth 2.0/OIDC, controles como validación de tokens, aplicación de políticas de scopes, verificación de audience, rotación de claves o logging tienen mayor confiabilidad cuando están automatizados.

**\*\*Periodicidad ( $\square_P=0.20$ ).**

La periodicidad mide si el control opera de forma permanente, periódica u ocasional. En seguridad de identidad, un

control ejecutado solo de forma eventual puede no cubrir ventanas críticas de exposición. Por eso la periodicidad recibe el mismo peso que la automatización.

**\*\*Momento ( $\square_T=0.10$ ).\*\***

El momento distingue controles preventivos, detectivos y correctivos. Se privilegia lo preventivo, pero se evita sobre-dimensionar esta dimensión porque controles detectivos y correctivos también pueden ser relevantes en aseguramiento, especialmente para monitoreo, respuesta, auditoría y trazabilidad.

**\*\*Alcance funcional ( $\square_L=0.10$ ).\*\***

El alcance indica si el control cubre un componente específico o la arquitectura de forma más general. Se le asigna un peso moderado porque un control específico puede ser altamente pertinente para un riesgo crítico. Por ejemplo, PKCE es específico del flujo de autorización, pero puede ser el control más importante frente a la interceptación del `authorization_code`. Por esta razón, el modelo evita penalizar excesivamente controles específicos.

En síntesis, el modelo prioriza que el control esté **maduro, automatizado y operando con periodicidad suficiente**, sin perder de vista su momento y alcance.

**Mapeo cualitativo de dimensiones** El catálogo puede expresar los valores de las dimensiones en escala de 0 a 100. La UI los normaliza antes de calcular el score operativo.

Ejemplo de mapeo usado por la tesis:

```
{
  "madurez": {
    "declarado": 25,
    "diseñado": 0,
    "implementado": 75,
    "auditado": 100
  },
  "automatizacion": {
    "manual": 0,
    "semiautomatico": 50,
    "automatico": 100
  },
  "momento": {
    "correctivo": 0,
    "detectivo": 50,
    "preventivo": 100
  },
  "periodicidad": {
    "ocasional": 25,
    "periodico": 75,
    "permanente": 100
  },
  "alcance": {
    "especifico": 25,
    "general": 100
  },
  "factorEvidencia": {
    "sinEvidencia": 0,
    "parcial": 0.5,
    "suficiente": 0.85,
    "independienteOAditada": 1
  }
}
```

En claves JSON se evita el uso de tildes y caracteres especiales. Por eso diseñado se representa como `diseñado`.

**Normalización** La forma implementada en JavaScript es tolerante a dos escalas posibles del catálogo: valores entre 0 y 1, o valores entre 0 y 100. Para evitar ambigüedad, el formulario normaliza cualquier valor mayor que 1 dividiéndolo entre 100:

$$\text{norm}(x) = \begin{cases} 0, & x \text{ no es numérico} \\ \min\left(1, \max\left(0, \frac{x}{100}\right)\right), & x > 1 \\ \min(1, \max(0, x)), & 0 \leq x \leq 1 \end{cases}$$

**Score base del control** Para un control (c), primero se calcula el score base sin evidencia:

$$S_{base(raw)}(c) = 0.40M_c + 0.20A_c + 0.10T_c + 0.20P_c + 0.10L_c$$

Luego se normaliza:

$$S_{base}(c) = \text{norm}(S_{base(raw)}(c))$$

Esta separación es importante porque permite diferenciar entre el **diseño operativo del control** y la **evidencia disponible**. En la UI, esta lectura se presenta como eficiencia cualitativa del control.

**Factor de evidencia** La evidencia se representa mediante ( $F_{\{ev\}}$ ). El formulario toma el valor asociado a la suficiencia de evidencia y lo normaliza:

$$F_{ev}(c) = \text{norm}(f_{ev}(c))$$

Donde ( $f_{\{ev\}}(c)$ ) puede tomar valores como 0, 0.5, 0.85 o 1, según la evidencia sea inexistente, parcial, suficiente o independiente/auditada.

El factor de evidencia funciona como un ajuste multiplicativo. Esto evita que un control con buen diseño declarado, pero sin evidencia verificable, obtenga un aporte alto dentro de la matriz. En otras palabras, el modelo distingue entre:

- control diseñado o declarado;
- control implementado;
- control evidenciado;
- control auditado o validado independientemente.

**Score final del control** El score final del control es el producto entre el score base normalizado y el factor de evidencia normalizado:

$$S_{control}(c) = S_{base}(c) \times F_{ev}(c)$$

El resultado ( $S_{\{control\}}(c)$ ) está en escala de 0 a 1 y la UI lo muestra como porcentaje. Por ejemplo, ( $S_{\{control\}}=0.85$ ) se presenta como 85 %.

**Ejemplo de cálculo del score de un control bueno** Supóngase el control CE1-001, asociado al uso de Authorization Code + PKCE con S256:

$$M = 100, \quad A = 100, \quad T = 100, \quad P = 100, \quad L = 100, \quad f_{ev} = 1$$

El cálculo base es:

$$\begin{aligned}
S_{base(raw)} &= 0.40(100) + 0.20(100) + 0.10(100) + 0.20(100) + 0.10(100) \\
&= 40 + 20 + 10 + 20 + 10 \\
&= 100
\end{aligned}$$

Normalización:

$$S_{base} = \text{norm}(100) = 1$$

Factor de evidencia:

$$F_{ev} = \text{norm}(1) = 1$$

Score final:

$$S_{control} = S_{base} \times F_{ev} = 1 \times 1 = 1$$

Lectura: el control alcanza 100 % porque está auditado, automatizado, es preventivo, permanente, tiene alcance general y cuenta con evidencia independiente o auditada.

**Ejemplo de cálculo del score de un control malo** Ahora considérese un caso débil del mismo control:

$$M = 25, \quad A = 0, \quad T = 0, \quad P = 25, \quad L = 25, \quad f_{ev} = 0$$

$$\begin{aligned}
S_{base(raw)} &= 0.40(25) + 0.20(0) + 0.10(0) + 0.20(25) + 0.10(25) \\
&= 10 + 0 + 0 + 5 + 2.5 \\
&= 17.5
\end{aligned}$$

$$S_{base} = \text{norm}(17.5) = 0.175$$

$$F_{ev} = \text{norm}(0) = 0$$

$$S_{control} = 0.175 \times 0 = 0$$

Lectura: aunque exista alguna declaración o diseño preliminar, la ausencia de evidencia suficiente hace que el control no aporte mitigación demostrable dentro del modelo.

**Peso de mitigación (w\_i) de cada control** Cada relación riesgo-control tiene un peso de mitigación (w\_i). Este peso no corresponde al score del control ni al costo del control. Es un parámetro del catálogo que expresa la contribución técnica esperada de un control frente a un riesgo específico.

Para un riesgo (r) con (n\_r) controles asociados, el modelo busca que los pesos de mitigación cumplan:

$$\sum_{i=1}^{n_r} w_i = 1$$

Si en una implementación o filtro la suma no fuera exactamente 1, la fórmula de eficacia divide por ( $\sum w_i$ ) para normalizar el resultado.



El peso ( $w_i$ ) se asigna con base en la relación causal entre el control y el riesgo. No todos los controles que aparecen en una relación mitigan con la misma fuerza. Algunos controles atacan directamente la causa principal del riesgo; otros reducen exposición, mejoran detección o fortalecen trazabilidad.

**Criterios para justificar ( $w_i$ )** La asignación de ( $w_i$ ) debe justificarse en el campo justificación de cada relación RiesgoControl. Para mantener consistencia, la tesis usa los siguientes criterios:

**1. Proximidad causal.**

Un control recibe mayor peso si mitiga directamente la condición que materializa el riesgo. Por ejemplo, PKCE con S256 está causalmente más cerca de la interceptación del `authorization_code` que un control general de logging.

**2. Especificidad frente al riesgo.**

Un control específico del escenario puede recibir mayor peso si fue diseñado para ese riesgo. Un control global puede recibir menor peso si aporta de forma transversal pero no resuelve por sí solo el evento de riesgo.

**3. Carácter obligatorio.**

Un control marcado como obligatorio puede recibir un peso alto cuando su ausencia impide una mitigación razonable del riesgo. Sin embargo, obligatoriedad y peso no son equivalentes: un control obligatorio puede requerir controles complementarios.

**4. Cobertura del vector de ataque.**

Un control recibe mayor peso si cubre la mayor parte del vector de ataque o de la condición habilitante. Si solo cubre una fase secundaria, su peso debe ser menor.

**5. Dependencia de otros controles.**

Si un control solo funciona correctamente cuando otros controles están presentes, su peso debe reflejar esa dependencia. Por ejemplo, la validación de `state` ayuda contra CSRF, pero no reemplaza PKCE frente al canje indebido de un código interceptado.

**6. Evidencia técnica y fuentes normativas.**

La ponderación debe estar respaldada por referencias técnicas del catálogo, como RFCs, OWASP, guías de proveedor o estándares de gestión de riesgos. La fuente no define automáticamente el número, pero sí sustenta la relación de mitigación.

**Rangos orientativos para ( $w_i$ )** Para evitar asignaciones arbitrarias, se usan rangos orientativos:

- ( $0.50 \leq w_i \leq 0.80$ ): control principal o nuclear del riesgo.
- ( $0.20 \leq w_i < 0.50$ ): control relevante, pero no suficiente por sí solo.
- ( $0.05 \leq w_i < 0.20$ ): control complementario, transversal, detectivo o de soporte.
- ( $w_i < 0.05$ ): aporte marginal; debería evitarse salvo justificación explícita.

Estos rangos no reemplazan el juicio del evaluador ni la justificación técnica. Sirven para que el catálogo mantenga coherencia interna.

**Ejemplo de justificación de ( $w_i$ ): riesgo RE1-001** El riesgo RE1-001 corresponde a la interceptación o sustitución del `authorization_code` y downgrade de PKCE. Para este riesgo, el control específico CE1-001 recibe un peso alto:

$$w_{CE1-001} = 0.70$$

La justificación es que Authorization Code + PKCE con S256 mitiga directamente el canje indebido del código de autorización, porque el atacante no puede completar el intercambio sin conocer el `code_verifier`. En cambio, controles como `state`, `nonce` o transporte seguro son necesarios, pero complementarios para este riesgo particular.

Una distribución conceptual puede representarse así:

$$w_{CE1-001} + w_{CE1-002} + w_{CG-006} = 0.70 + 0.15 + 0.15 = 1$$

Lectura:

- (CE1-001): control principal frente al canje indebido del código.
- (CE1-002): control complementario de ligadura transaccional mediante state y nonce.
- (CG-006): control transversal de transporte seguro y no exposición de tokens o artefactos sensibles en URI.

**Aporte de eficacia de un control frente a un riesgo** Para un control ( $c_i$ ) asociado a un riesgo ( $r$ ), el aporte de eficacia es:

$$Aporte_i(r, c_i) = w_i \cdot S_{control}(c_i)$$

Este valor no se interpreta como eficacia global del control. Solo indica cuánto aporta ese control, con ese peso, frente a ese riesgo específico. El formulario muestra este valor como parte del paso a paso del control.

**Eficacia frente al riesgo** La eficacia frente al riesgo, representada en el formulario como ( $E_r$ ), agrega todos los controles asociados a un riesgo:

$$E_r = \frac{\sum_{i=1}^{n_r} w_i \cdot S_{control}(c_i)}{\sum_{i=1}^{n_r} w_i}$$

Donde:

- ( $E_r$ ) es la eficacia frente al riesgo ( $r$ ).
- ( $n_r$ ) es el número de controles asociados al riesgo ( $r$ ).
- ( $w_i$ ) es el peso de mitigación de la relación riesgo-control.
- ( $S_{\{control\}}(c_i)$ ) es el score final normalizado del control ( $c_i$ ).

Ejemplo:

$$\begin{aligned} E_{RE1-001} &= \frac{0.70(1.00) + 0.15(0.85) + 0.15(0.90)}{0.70 + 0.15 + 0.15} \\ &= \frac{0.70 + 0.1275 + 0.135}{1.00} \\ &= 0.9625 \end{aligned}$$

Lectura: el riesgo RE1-001 queda cubierto en 96.25 % bajo los controles y evidencias observadas.

**Costo asignado del control** Un mismo control puede mitigar varios riesgos. Para evitar doble conteo, el costo se distribuye por peso relativo dentro del conjunto de relaciones activas del escenario:

$$C_{asig}(r, c) = C(c) \cdot \frac{w_{r,c}}{\sum_{x \in R_c} w_{x,c}}$$

Donde:

- ( $C_{\{asig\}}(r, c)$ ) es el costo asignado del control ( $c$ ) al riesgo ( $r$ ).
- ( $C(c)$ ) es el costo ingresado o sugerido del control.
- ( $w_{\{r,c\}}$ ) es el peso de mitigación del control ( $c$ ) frente al riesgo ( $r$ ).
- ( $R_c$ ) es el conjunto de riesgos activos del escenario que usan el control ( $c$ ).

El costo total asignado al riesgo es:

$$C_r = \sum_{i=1}^{n_r} C_{asig}(r, c_i)$$

**Beneficio de mitigación estimado** El beneficio de mitigación estimado se calcula multiplicando el impacto económico estimado del riesgo por su eficacia observada:

$$B_r = I_r \cdot E_r$$

Donde:

- (B<sub>r</sub>) es el beneficio de mitigación estimado.
- (I<sub>r</sub>) es el impacto económico estimado del riesgo.
- (E<sub>r</sub>) es la eficacia frente al riesgo.

Los valores económicos deben estar expresados en la misma moneda antes de calcular eficiencia.

**Eficiencia frente al riesgo** La eficiencia frente al riesgo se representa con la letra griega ( $\eta$ ) y relaciona beneficio estimado contra costo asignado:

$$\eta_r = \frac{B_r}{C_r}$$

Si (C<sub>r</sub>=0) y (B<sub>r</sub>>0), la UI representa el resultado como ( $\infty$ ). En la práctica, ese caso debe revisarse porque puede indicar ausencia de costo registrado o un control sin estimación económica.

Ejemplo:

$$\begin{aligned} B_r &= 100000 \times 0.85 = 85000 \\ C_r &= 10000 \\ \eta_r &= \frac{85000}{10000} = 8.5 \end{aligned}$$

Lectura: por cada unidad monetaria asignada a controles del riesgo, el modelo estima 8.5 unidades monetarias de exposición mitigada. No debe interpretarse como retorno financiero exacto, sino como indicador semicuantitativo para priorización.

**Agregación global** Para una evaluación con (m) riesgos activos, la eficacia global se calcula como promedio simple de las eficacias por riesgo:

$$E_{global} = \frac{1}{m} \sum_{r=1}^m E_r$$

La eficiencia global se calcula como promedio simple de las eficiencias por riesgo:

$$\eta_{global} = \frac{1}{m} \sum_{r=1}^m \eta_r$$

Esta es la misma notación que usa el formulario cuando muestra las ecuaciones globales:

$$\begin{aligned} E_{global} &= \frac{1}{m} \sum_{r=1}^m E_r \\ \eta_{global} &= \frac{1}{m} \sum_{r=1}^m \eta_r \end{aligned}$$

**Umbral es cualitativos** La UI usa umbrales para traducir valores numéricos a etiquetas cualitativas.

Para eficacia y score:

$$Q(x) = \begin{cases} \text{Alta,} & x \geq 0.75 \\ \text{Media,} & 0.50 \leq x < 0.75 \\ \text{Baja,} & 0 \leq x < 0.50 \end{cases}$$

Para exposición observada:

$$Exposicion(E_r) = \begin{cases} \text{Bajo,} & E_r \geq 0.75 \\ \text{Medio,} & 0.50 \leq E_r < 0.75 \\ \text{Alto,} & 0.25 \leq E_r < 0.50 \\ \text{Crítico,} & 0 \leq E_r < 0.25 \end{cases}$$

Para eficiencia:

$$Q_\eta(\eta) = \begin{cases} \text{Alta,} & \eta \geq 1 \\ \text{Media,} & 0.50 \leq \eta < 1 \\ \text{Baja,} & 0 \leq \eta < 0.50 \end{cases}$$

**Relación entre la tesis y el formulario** La tesis define las variables, fórmulas, ponderadores y reglas de interpretación. El formulario las operacionaliza en JavaScript y las renderiza mediante MathJax. Por eso los símbolos ( $S_{\text{base(raw)}}$ ), ( $S_{\text{base}}$ ), ( $F_{\text{ev}}$ ), ( $S_{\text{control}}$ ), ( $w_i$ ), ( $A_{\text{aporte}_i}$ ), ( $E_r$ ), ( $\square_r$ ), ( $E_{\text{global}}$ ) y ( $\square_{\text{global}}$ ) deben mantenerse estables entre la tesis, el código y el informe exportado.

**5.1.7. Estructura del resultado de evaluación** El modelo produce dos artefactos JSON diferenciados: el **catálogo maestro** y el **resultado de evaluación**. El catálogo es estable y predefinido por la tesis; el resultado es el artefacto diligenciado por el evaluador para una arquitectura y momento concretos.

Ejemplo mínimo de resultado:

```
{
  "evaluacionArquitectura": {
    "id": "EVA-2026-001",
    "fecha": "2026-06-03",
    "evaluador": "Fredy Rosero",
    "ambiente": "Preproduccion",
    "escenarioId": "SPA_BFF_IDP",
    "catalogoId": "CAT-OAUTH2-OIDC-2026-01",
    "versionCatalogo": "1.0.0",
    "monedaEvaluacion": "USD",
    "tasaCambioReferencia": {
      "COP_USD": 4000
    }
  },
  "registrosMatriz": [
    {
      "id": "RM-001-1",
      "riesgoControlId": "RC-017",
      "riesgoId": "RE1-001",
      "controlId": "CE1-001",
      "activoAfectado": "authorization_code",
    }
  ]
}
```

```

"tipoImpacto": "lucroCesante",
"impactoEconomicoOriginal": {
  "valor": 50000000,
  "moneda": "COP"
},
"impactoEconomicoEstimado": {
  "valor": 12500,
  "moneda": "USD"
},
"amenaza": "Atacante obtiene el authorization_code durante el callback",
"vulnerabilidad": "PKCE ausente, PKCE mal configurado o aceptación de code_challenge_method=plain",
"probabilidad": "Media",
"impacto": "Alto",
"nivelRiesgo": "Alto",
"costoAsignado": 1650,
"evaluacionControl": {
  "madurez": "Implementada",
  "automatizacion": "Automatico",
  "momento": "Preventivo",
  "periodicidad": "Permanente",
  "alcance": "Especifico",
  "scoreControl": 82.5,
  "scoreControlNormalizado": 0.825
},
"aporteEficacia": 0.5775,
"evidencias": [
  {
    "esperada": "Configuración del cliente OIDC con PKCE S256",
    "encontrada": "Captura de configuración del IdP y logs del BFF",
    "fuente": "Consola IdP, repositorio BFF, logs de callback",
    "suficiencia": "Suficiente"
  }
],
"observacion": "PKCE se evidencia como implementado; falta complementar con prueba negativa de rechazo
}
],
"evaluacionesRiesgo": [
  {
    "riesgoId": "RE1-001",
    "impactoEconomicoEstimado": {
      "valor": 12500,
      "moneda": "USD"
    },
    "coberturaEsperada": 1,
    "coberturaObservada": 0.5775,
    "eficaciaFrenteAlRiesgo": 0.5775,
    "costoAsignadoTotal": 1650,
    "beneficioMitigacionEstimado": 7218.75,
    "eficienciaFrenteAlRiesgo": 4.38,
    "nivelExposicionObservado": "Medio"
  }
],
"evaluacionGlobal": {
  "registrosEvaluados": 1,

```

```

    "riesgosEvaluados": 1,
    "scorePromedioControl": 82.5,
    "eficienciaCuantitativa": 4.38,
    "eficaciaCuantitativa": 0.5775,
    "eficienciaCualitativa": "Es eficiente",
    "eficaciaCualitativa": "Es parcialmente eficaz"
  }
}

```

Cada RegistroMatriz evalúa una relación riesgo-control concreta. El campo aporteEficacia es el producto entre scoreControlNormalizado y pesoMitigacion. Las evaluacionesRiesgo agregan todos los registros asociados a un mismo riesgo. La evaluacionGlobal consolida todas las evaluaciones de riesgo y permite comparar escenarios.

## 5.2. Fundamento del modelo de evaluación

**5.2.1. Relación riesgo, control y auditoría** El modelo de evaluación se aplica en un único corte sobre el estado actual de la arquitectura revisada. No presupone una secuencia antes/después ni una fase de implementación dentro del mismo instrumento. En su lugar, se operacionaliza mediante cuatro momentos: selección del catálogo aplicable, caracterización del riesgo, evaluación de controles y evaluación global del sistema. Si posteriormente se aplican correcciones, una nueva aplicación del modelo constituye una reevaluación independiente.

La lógica base es:

Esta estructura organiza la aplicación del modelo en cuatro momentos:

1. **Selección de riesgos y controles aplicables:** parte del catálogo de riesgos globales OAuth 2.0/OIDC, delimita la arquitectura evaluada y activa tanto los riesgos específicos como los controles aplicables definidos por el modelo.
2. **Caracterización del riesgo:** registra el activo en riesgo, la amenaza, la vulnerabilidad, la probabilidad y el impacto de cada riesgo aplicable.
3. **Evaluación individual del control:** levanta costo, estado y evidencia del control, clasifica sus cinco dimensiones y homologa sus valores para obtener un score comparable.
4. **Evaluación global del sistema:** integra los resultados por riesgo para producir medidas agregadas de eficiencia, eficacia y nivel de exposición observado.

En la implementación web, el primer momento se materializa como un catálogo precargado. La selección de una de las tres arquitecturas habilita automáticamente los riesgos globales OAuth 2.0/OIDC, los riesgos específicos del escenario y los controles esperados asociados. El algoritmo no recibe riesgos libres escritos por el usuario; lo dinámico son los datos de evaluación diligenciados para cada registro.

Los casos base sin OAuth 2.0/OIDC no se incluyen en esta lógica del modelo de evaluación porque no contienen Authorization Server, clientes OAuth, tokens OAuth 2.0/OIDC, scopes ni validación federada de tokens. Sirven como contraste conceptual, no como objeto de evaluación de la matriz.

**5.2.2. Riesgos transversales y específicos** Los riesgos transversales son aquellos que pueden afectar a cualquier escenario OAuth 2.0/OIDC. No dependen de un flujo particular, sino de debilidades comunes de gobierno, configuración, validación u operación.

Ejemplos:

- clientes OAuth sin inventario ni responsable;
- emisión de tokens con scopes excesivos;
- tokens con vigencia inadecuada;
- falta de rotación de secretos o certificados;
- aceptación de algoritmos inseguros;
- ausencia de monitoreo sobre emisión y uso de tokens;
- falta de trazabilidad sobre decisiones de autorización;
- configuración inconsistente entre ambientes.

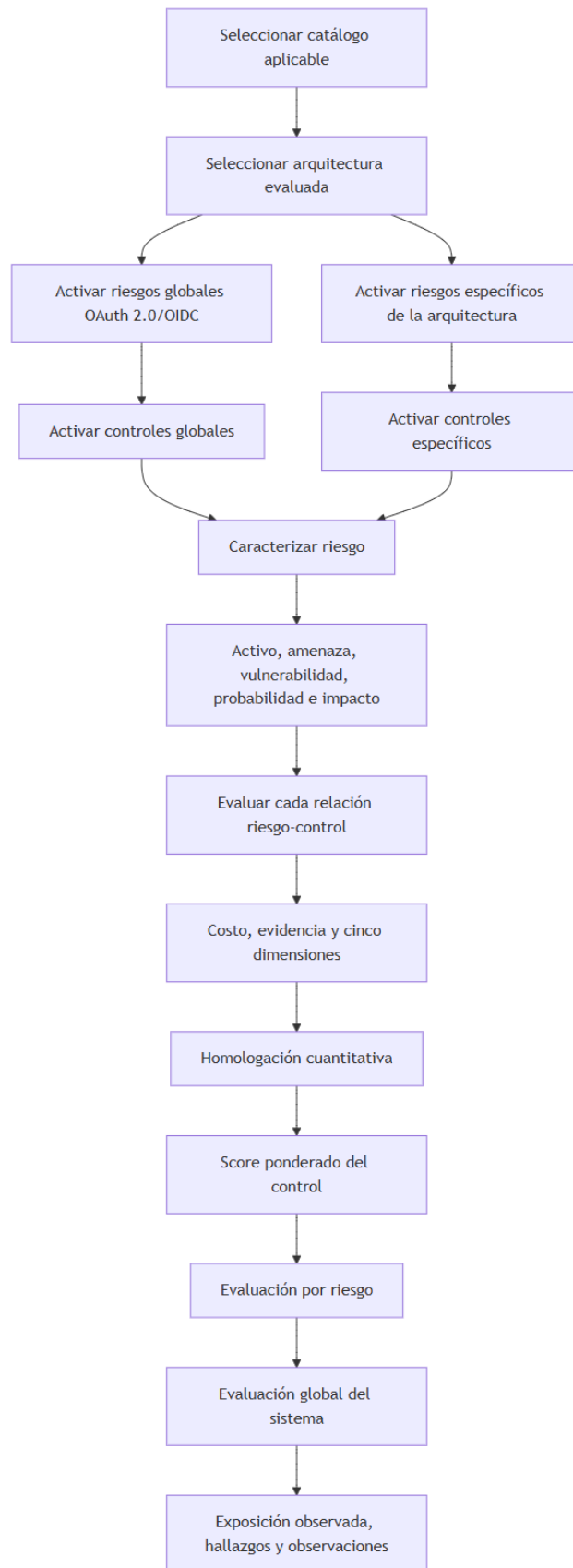


Figura 23: diagram  
78

Los riesgos específicos son aquellos que emergen de la arquitectura, el flujo y la frontera de confianza de un escenario particular. No reemplazan a los riesgos transversales, sino que los complementan.

Ejemplos:

- ausencia de state o nonce en el escenario 1;
- secretos hardcodeados o privilegios excesivos en clientes M2M del escenario 2;
- confusión de audiencia o propagación indebida de tokens en el escenario 3.

El modelo trabaja con un único corte de evaluación sobre la arquitectura tal como existe al momento del levantamiento. Para cada riesgo se debe identificar: activo afectado, amenaza o causa, vulnerabilidad, probabilidad e impacto.

**5.2.3. Controles transversales** Los controles transversales son controles esperados para toda arquitectura OAuth 2.0/OIDC, independientemente del escenario específico.

Ejemplos:

- política de registro y gobierno de clientes OAuth;
- definición de scopes mínimos;
- políticas de vigencia de tokens;
- gestión de secretos y certificados;
- rotación de claves y publicación de JWKS;
- logging de autenticación, emisión y validación de tokens;
- monitoreo de uso anómalo de clientes, tokens o scopes;
- revisión periódica de clientes y permisos;
- segregación de ambientes;
- gestión de cambios sobre el Authorization Server.

Estos controles establecen una línea mínima de gobierno. Después, cada escenario agrega controles específicos.

**5.2.4. Controles específicos por escenario** Cada escenario introduce una superficie de riesgo distinta. Por eso la matriz no evalúa todos los escenarios con la misma lista de controles.

**Arquitectura 1: SPA + BFF + IdP corporativo** Controles específicos:

- Authorization Code Flow con PKCE;
- code\_challenge\_method=S256;
- validación de state;
- validación de nonce;
- redirect URI exacta;
- tokens custodiados server-side en el BFF;
- cookie HttpOnly, Secure y SameSite;
- protección CSRF;
- evidencia de que la SPA no almacena tokens.

**Arquitectura 2: M2M con AS como RS** Controles específicos:

- uso controlado de client\_credentials;
- autenticación robusta del cliente técnico;
- scopes mínimos para APIs administrativas;
- cliente distinto por ambiente y propósito;
- secretos o certificados en vault;
- rotación periódica;
- revocación de clientes inactivos;
- monitoreo por client\_id;
- trazabilidad de acciones administrativas;
- revisión periódica de privilegios.



### Arquitectura 3: API Gateway federado + AS corporativo    Controles específicos:

- validación de firma o introspection;
- validación de issuer;
- validación de audience por dominio;
- validación de expiración;
- validación de scopes;
- política común de validación entre Gateway y Resource Servers;
- revalidación mínima en backends críticos;
- token exchange cuando cambie la audiencia o frontera de confianza;
- reducción de scopes al intercambiar tokens;
- correlación de logs Gateway-backend;
- actualización y caché controlada de JWKS.

**5.2.5. Score del control, eficacia del riesgo y eficiencia del sistema**    El enfoque de Valencia Duque permite hablar de eficiencia y eficacia del control, pero en esta tesis se operacionaliza en tres niveles para evitar ambigüedad:

1. **Score del control:** mide qué tan fuerte está un control según madurez, automatización, momento, periodicidad y alcance.
2. **Eficacia frente al riesgo:** mide qué tanto queda cubierto un riesgo al agregar los controles asociados y sus pesos de mitigación.
3. **Eficiencia frente al riesgo o global:** relaciona beneficio de mitigación estimado con costo asignado.

La relación conceptual es:

Así se evita decir que un control tiene una eficacia única. Un mismo control puede ser muy eficaz para un riesgo y marginal para otro. Esa diferencia se modela con `pesoMitigacion`.

**5.2.6. Dimensiones de evaluación del control**    Cada control se clasifica mediante cinco dimensiones.

**Madurez**    Describe el estado del control frente a su formalización, diseño, implementación y verificación.

Valores:

- Diseñada.
- Declarada.
- Implementada.
- Auditada.

**Automatización**    Describe cuánto depende el control de intervención humana.

Valores:

- Manual.
- Semiautomático.
- Automático.

**Momento de actuación**    Describe cuándo actúa el control frente al riesgo.

Valores:

- Preventivo.
- Detectivo.
- Correctivo.

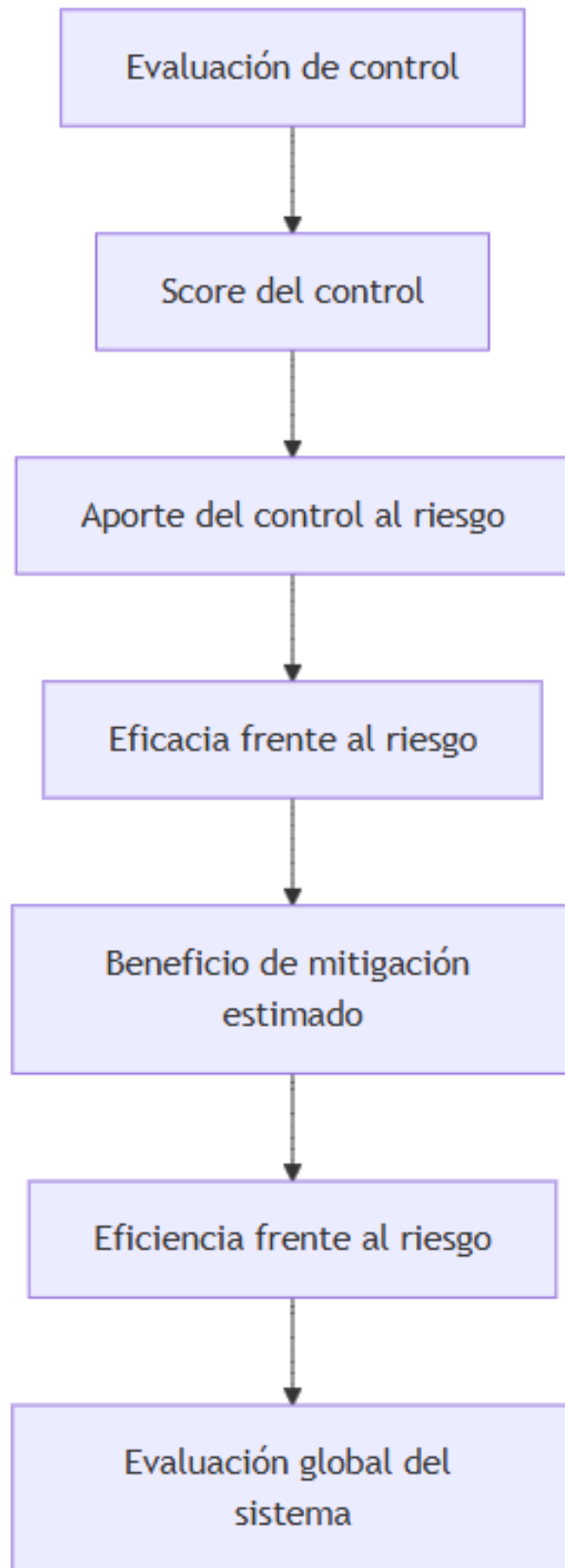


Figura 24: diagram  
81

**Periodicidad** Describe con qué frecuencia se ejecuta o verifica el control.

Valores:

- Ocasional.
- Periódico.
- Permanente.

**Alcance funcional** Describe la cobertura del control dentro de la arquitectura evaluada.

Valores:

- Específico.
- General.

**5.2.7. Nivel de exposición observado** La matriz opera en un único momento de evaluación y produce una conclusión sobre la exposición observada en la arquitectura revisada. Para ello articula tres elementos:

1. **Riesgo identificado:** riesgo relevante en el escenario evaluado.
2. **Estado observado del control:** valoración de diseño, operación, score y evidencia disponible.
3. **Nivel de exposición observado:** conclusión sobre la exposición actual del escenario.

La relación conceptual es:

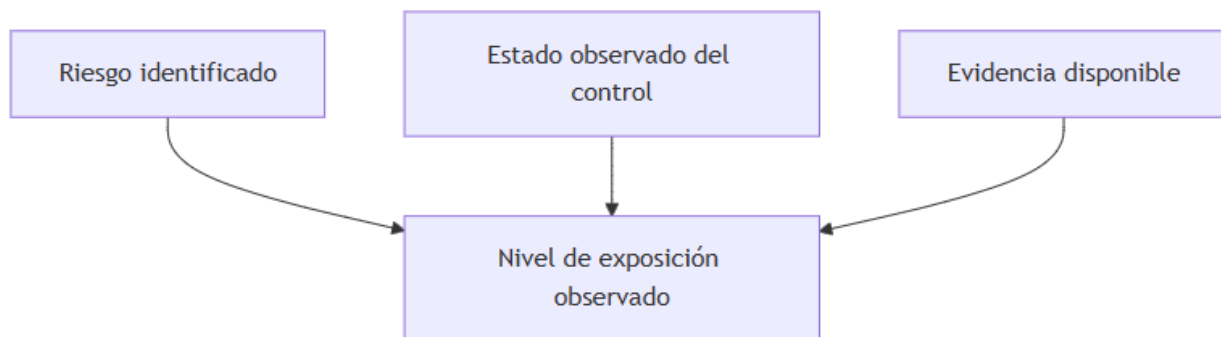


Figura 25: diagram

Si posteriormente se aplican correcciones, la tesis no asume una segunda fase interna del mismo modelo, sino una nueva evaluación sobre un nuevo estado de la arquitectura. La tesis puede usar homologación cualitativa o semicuantitativa, siempre que mantenga trazabilidad entre riesgo, control, evidencia y conclusión.

### 5.3. Criterios de evaluación

La matriz utiliza criterios orientados a determinar si el control es pertinente, suficiente, verificable y útil para reducir el riesgo identificado.

Los criterios principales son:

1. **Pertinencia:** el control está directamente relacionado con el riesgo.
2. **Suficiencia:** el control cubre los elementos mínimos para mitigar el riesgo.
3. **Evidencia:** el control puede verificarse con documentación, configuración, logs, pruebas o revisión técnica.
4. **Madurez:** el control está declarado, diseñado, implementado o auditado.
5. **Automatización:** el control es manual, semiautomático o automático.
6. **Momento:** el control es preventivo, detectivo o correctivo.
7. **Periodicidad:** el control es ocasional, periódico o permanente.

8. **Alcance:** el control es específico o general.
  9. **Eficiencia:** el diseño del control es adecuado frente al riesgo.
  10. **Eficacia:** existe evidencia de operación real.
  11. **Nivel de exposición observado:** se caracteriza la exposición actual del escenario evaluado.
  12. **Trazabilidad:** el control se relaciona con una referencia técnica, normativa o conceptual.
- 

## 5.4. Estructura de la matriz

La matriz se organiza en cuatro bloques lógicos.

### 5.4.1. Bloque 1: Riesgo

Este bloque identifica qué puede fallar.

Campos:

1. escenario OAuth 2.0/OIDC;
2. tipo de riesgo: global o específico;
3. activo afectado;
4. riesgo identificado;
5. amenaza o causa;
6. vulnerabilidad o condición habilitante;
7. probabilidad;
8. impacto;
9. nivel del riesgo identificado;
10. impacto económico estimado o lucro cesante.

### 5.4.2. Bloque 2: Relación riesgo-control

Este bloque identifica qué control mitiga qué riesgo y con qué peso.

Campos:

1. control esperado;
2. tipo de control: global o específico;
3. identificador riesgoControlId;
4. pesoMitigacion;
5. obligatoriedad;
6. costo total del control;
7. costo asignado al riesgo.

### 5.4.3. Bloque 3: Evaluación del control y evidencia

Este bloque registra el estado observado del control.

Campos:

1. madurez;
2. automatización;
3. momento;
4. periodicidad;
5. alcance funcional;
6. score del control;
7. evidencia esperada;
8. evidencia encontrada;
9. fuente de evidencia;
10. suficiencia de evidencia.

### 5.4.4. Bloque 4: Evaluación por riesgo y evaluación global

Este bloque consolida los resultados.

Campos:

1. aporte de eficacia del control;
2. eficacia frente al riesgo;
3. eficiencia frente al riesgo;
4. nivel de exposición observado;
5. hallazgo, brecha u observación;
6. referencia normativa o técnica.

La estructura general se representa así:

Esta estructura corrige la inconsistencia entre el esquema del modelo y la matriz: primero se identifica el riesgo, luego la relación riesgo-control, después la evidencia y finalmente el resultado por riesgo y global. La matriz no parte de los casos base; parte de los escenarios OAuth 2.0/OIDC.

---

## 5.5. Campos de la matriz

**5.5.1. Escenario** Identifica el escenario OAuth 2.0/OIDC evaluado.

Valores esperados:

1. Arquitectura 1: SPA estática + BFF + IdP corporativo.
2. Arquitectura 2: M2M con Authorization Server como Resource Server.
3. Arquitectura 3: API Gateway federado con Authorization Server corporativo.
4. Transversal OAuth 2.0/OIDC, cuando el riesgo o control aplique a todos los escenarios.

Los casos base 0A y 0B no son valores válidos de este campo porque no hacen parte de la matriz del modelo de evaluación.

**5.5.2. Tipo de riesgo** Clasifica si el riesgo es global o específico.

- **Global:** aplica a toda arquitectura OAuth 2.0/OIDC.
- **Específico:** aplica a uno de los tres escenarios evaluados.

**5.5.3. Activo afectado** Identifica el activo de información, componente o elemento técnico afectado.

Ejemplos:

- authorization\_code;
- access\_token;
- id\_token;
- refresh\_token;
- client\_id;
- client\_secret;
- certificado o llave privada;
- scopes;
- claims;
- issuer;
- audience;
- JWKS;
- Authorization Server;
- Resource Server;
- API Gateway;
- logs de autenticación/autorización.

**5.5.4. Riesgo identificado** Describe el evento de riesgo que se observa como relevante en el escenario evaluado.

Ejemplos:

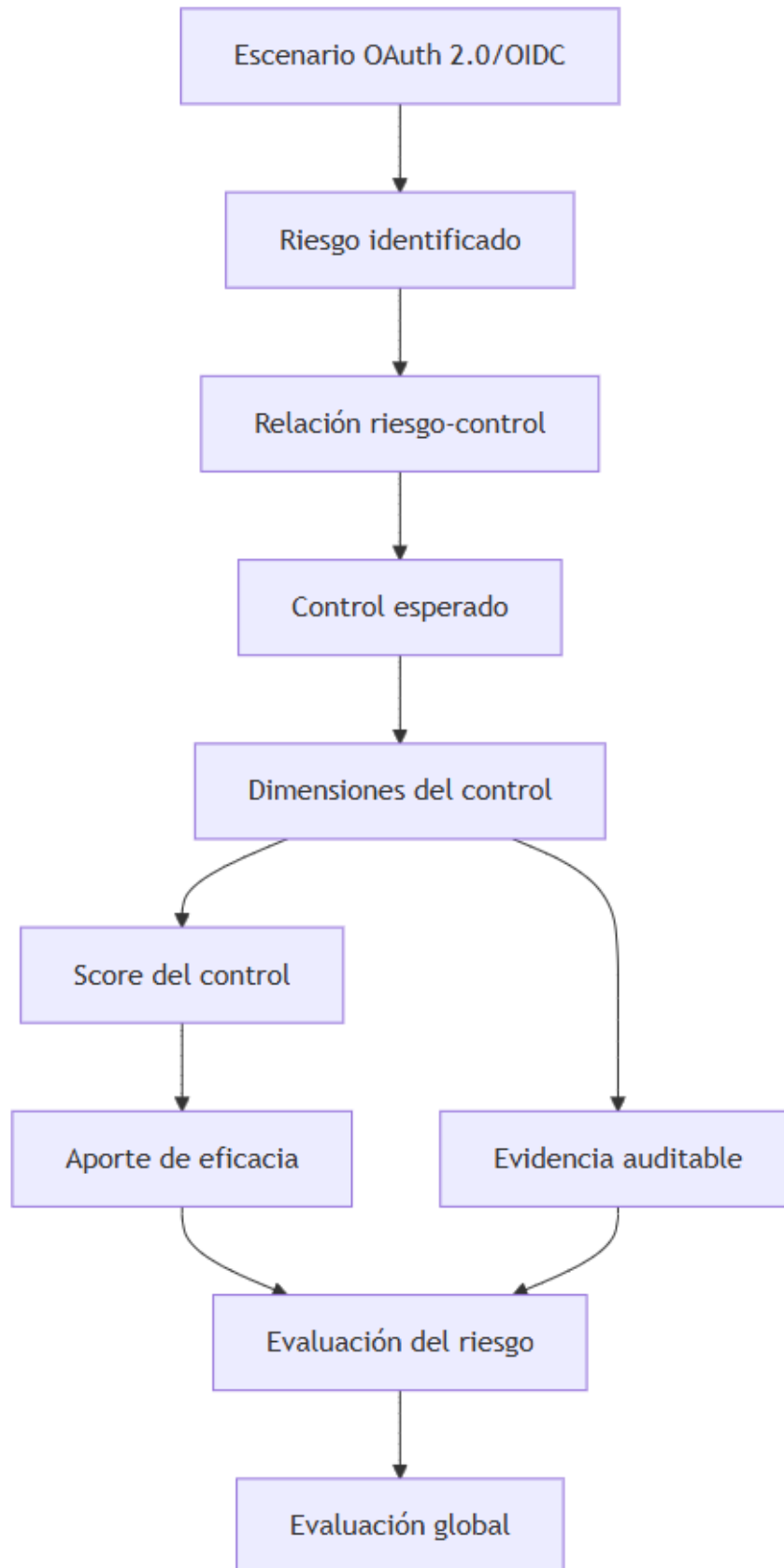


Figura 26: diagram  
85

- interceptación del `authorization_code`;
- exposición de tokens en navegador;
- aceptación de tokens inválidos;
- confusión de audiencia;
- uso excesivo de privilegios;
- abuso de cliente técnico;
- pérdida de trazabilidad;
- propagación indebida de tokens.

**5.5.5. Amenaza o causa** Describe la condición que puede materializar el riesgo.

Ejemplos:

- falta de PKCE;
- redirect URI mal registrada;
- ausencia de validación de `state` o `nonce`;
- falta de validación de `audience`;
- scopes demasiado amplios;
- secretos hardcodeados;
- JWKS desactualizado;
- ausencia de monitoreo;
- Gateway con validación parcial;
- backend que confía ciegamente en el Gateway.

**5.5.6. Vulnerabilidad** Describe la condición técnica, operativa o documental que habilita el riesgo.

Ejemplos:

- configuración débil del cliente OAuth;
- credencial expuesta;
- token aceptado fuera de su audiencia;
- falta de segregación por ambiente;
- política de scopes demasiado amplia;
- ausencia de evidencia de rotación.

**5.5.7. Probabilidad** Estima posibilidad de materialización.

Valores sugeridos:

- Baja.
- Media.
- Alta.

**5.5.8. Impacto** Estima consecuencia sobre confidencialidad, integridad, disponibilidad, trazabilidad o cumplimiento.

Valores sugeridos:

- Bajo.
- Medio.
- Alto.
- Crítico.

**5.5.9. Nivel del riesgo identificado** Resultado de combinar probabilidad e impacto del riesgo caracterizado.

Valores sugeridos:

- Bajo.
- Medio.

- Alto.
- Crítico.

**5.5.10. Impacto económico estimado** Registra la estimación monetaria usada para calcular eficiencia.

Puede representar:

- lucro cesante;
- fraude potencial;
- sanción;
- costo operativo;
- impacto reputacional monetizado;
- costo de remediación.

Si el valor original está en una moneda distinta a la moneda del catálogo, debe convertirse antes de calcular eficiencia.

**5.5.11. Control esperado** Describe el control que debería existir para mitigar el riesgo.

Ejemplos:

- Authorization Code + PKCE con S256;
- validación de state;
- validación de nonce;
- redirect URI exacta;
- validación de firma;
- validación de issuer;
- validación de audience;
- validación de expiración;
- validación de scopes;
- token introspection;
- token exchange;
- gestión de secretos en vault;
- rotación de credenciales;
- monitoreo por client\_id.

**5.5.12. Relación riesgo-control** Identifica la relación específica evaluada mediante riesgoControlId. Esta relación contiene el pesoMitigacion, la obligatoriedad y la justificación técnica de por qué el control mitiga el riesgo.

**5.5.13. Costo total y costo asignado** El costoTotal pertenece al control esperado. El costoAsignado pertenece a la relación evaluada en la matriz. Si un control mitiga varios riesgos, el costo se distribuye según pesoMitigacion para evitar doble conteo.

**5.5.14. Dimensiones del control** Registra la valoración del control en las cinco dimensiones definidas por la metodología:

- **Madurez:** diseñada, declarada, implementada, auditada.
- **Automatización:** manual, semiautomático, automático.
- **Momento:** preventivo, detectivo, correctivo.
- **Periodicidad:** ocasional, periódico, permanente.
- **Alcance funcional:** específico, general.

Para conservar comparabilidad entre registros, cada dimensión debe diligenciarse con un único valor por control.

**5.5.15. Score del control** Es la calificación cuantitativa del control usada por la UI y por la matriz. La notación coincide con el formulario web:



$$S_{base(raw)}(c) = 0.40M_c + 0.20A_c + 0.10T_c + 0.20P_c + 0.10L_c$$

$$S_{base}(c) = \text{norm}(S_{base(raw)}(c))$$

$$F_{ev}(c) = \text{norm}(f_{ev}(c))$$

$$S_{control}(c) = S_{base}(c) \times F_{ev}(c)$$

El valor  $S_{control}$  está en escala de 0 a 1 y se muestra como porcentaje. Por tanto,  $S_{control} = 0.85$  equivale a 85 %.

**5.5.16. Evidencia auditable esperada** Describe qué evidencia debería existir.

Ejemplos:

- configuración del Authorization Server;
- configuración del cliente OAuth 2.0/OIDC;
- configuración del API Gateway;
- configuración del Resource Server;
- política de scopes;
- evidencia de rotación;
- logs de autenticación;
- logs de emisión de tokens;
- logs de validación o introspection;
- pruebas técnicas;
- revisión puntual de código;
- documentación de arquitectura;
- registros de monitoreo o alertamiento.

**5.5.17. Evidencia encontrada** Describe la evidencia efectivamente disponible durante la evaluación.

La matriz debe diferenciar entre:

- evidencia suficiente;
- evidencia parcial;
- evidencia declarada pero no demostrada;
- ausencia de evidencia.

**5.5.18. Fuente de evidencia** Identifica de dónde proviene la evidencia.

Ejemplos:

- consola del IdP;
- archivo de configuración;
- repositorio;
- pipeline;
- API Gateway;
- logs SIEM;
- trazas APM;
- prueba técnica;
- documento de arquitectura;
- entrevista técnica.

**5.5.19. Aporte de eficacia** Mide cuánto aporta el control evaluado a la cobertura del riesgo:

$$Aporte_i(r, c_i) = w_i \cdot S_{control}(c_i)$$

**5.5.20. Eficacia frente al riesgo** Agrega los aportes de todos los controles asociados a un riesgo:

$$E_r = \frac{\sum_{i=1}^{n_r} w_i \cdot S_{control}(c_i)}{\sum_{i=1}^{n_r} w_i}$$

**5.5.21. Eficiencia frente al riesgo** Relaciona beneficio de mitigación estimado y costo asignado total:

$$B_r = I_r \cdot E_r$$

$$\eta_r = \frac{B_r}{C_r}$$

Donde  $I_r$  es el impacto económico estimado,  $E_r$  la eficacia frente al riesgo y  $C_r$  el costo asignado total de los controles del riesgo.

**5.5.22. Nivel de exposición observado** Describe la exposición que se observa en la arquitectura evaluada al momento del levantamiento, a partir del riesgo identificado, el score de los controles y la evidencia disponible.

Valores sugeridos:

- Bajo.
- Medio.
- Alto.
- Crítico.

**5.5.23. Hallazgo, brecha u observación** Registra la conclusión evaluativa.

Ejemplos:

- brecha de diseño;
- brecha de implementación;
- falta de evidencia;
- control manual débil;
- control no periódico;
- exceso de privilegios;
- ausencia de trazabilidad;
- observación de mejora.

**5.5.24. Referencia normativa o técnica** Relaciona el control con una fuente.

Ejemplos:

- RFC 6750;
- RFC 7636;
- RFC 7662;
- RFC 8693;
- RFC 9068;
- RFC 9700;
- OpenID Connect Core 1.0;
- OWASP OAuth 2.0 Cheat Sheet;

- OWASP ASVS;
- NIST SP 800-30;
- NIST SP 800-53;
- ISO/IEC 27002;
- Valencia Duque.

## 5.6. Plantilla inicial de la matriz

La plantilla inicial se limita a los tres escenarios OAuth 2.0/OIDC. Los casos base no se incluyen.

En el entregable web, esta plantilla se materializa como un catálogo precargado. El formulario carga automáticamente los riesgos y controles asociados al escenario seleccionado, mientras que el evaluador diligencia los datos de análisis, costo, evidencia, estado del control y observaciones de cada registro.

### Registro 1: Transversal OAuth 2.0/OIDC

- **Escenario:** Transversal OAuth 2.0/OIDC.
- **Riesgo identificado:** emisión, aceptación o uso indebido de tokens.
- **Activo afectado:** Authorization Server, clientes OAuth, tokens y logs de autorización.
- **Amenaza / causa:** configuración débil, falta de gobierno de clientes o ausencia de monitoreo.
- **Control esperado:** política central de emisión, validación, vigencia, scopes, monitoreo y gobierno de clientes OAuth.
- **Relación riesgo-control:** relación global del catálogo, por ejemplo RG-002 con CG-001.
- **Dimensiones del control:** madurez, automatización, momento, periodicidad y alcance funcional.
- **Score del control:** calculado con la fórmula ponderada de la sección 5.1.6.
- **Evidencia esperada:** política de clientes, configuración del AS, logs de emisión, inventario de clientes y revisión de scopes.
- **Fuente de evidencia:** consola AS, documentación, SIEM, repositorio de arquitectura.
- **Resultado esperado:** aporte de eficacia al riesgo y exposición observada.
- **Referencia:** RFC 9700, RFC 7662, NIST SP 800-53.

### Registro 2: Escenario 1 - SPA + BFF + IdP corporativo

- **Escenario:** SPA estática + BFF + IdP corporativo.
- **Riesgo identificado:** intercepción de código, exposición de tokens o sesión mal vinculada.
- **Activo afectado:** authorization\_code, id\_token, access\_token, refresh\_token, cookie de sesión.
- **Amenaza / causa:** falta de PKCE, validación débil de state/nonce, redirect URI débil o tokens en navegador.
- **Control esperado:** Authorization Code + PKCE con S256, validación de state y nonce, redirect URI exacta, tokens server-side y cookie segura.
- **Relación riesgo-control:** ejemplo RE1-001 con CE1-001 mediante RC-017.
- **Dimensiones del control:** implementada, automático, preventivo, permanente, específico.
- **Score del control:** ejemplo 82.5 para PKCE implementado y evidenciado.
- **Evidencia esperada:** configuración del cliente OIDC, callback, código BFF, configuración de cookies, logs IdP/BFF y prueba de ausencia de tokens en la SPA.
- **Fuente de evidencia:** IdP, repositorio BFF, navegador DevTools, logs.
- **Resultado esperado:** eficacia frente al riesgo y nivel de exposición observado.
- **Referencia:** RFC 7636, OIDC Core, RFC 9700.

### Registro 3: Escenario 2 - M2M con AS como RS

- **Escenario:** M2M con Authorization Server como Resource Server.
- **Riesgo identificado:** abuso de cliente técnico privilegiado.
- **Activo afectado:** client\_secret, certificado, access\_token M2M, API administrativa.
- **Amenaza / causa:** secretos expuestos, scopes excesivos, cliente sin responsable o ausencia de rotación.

- **Control esperado:** `client_credentials` con credencial protegida en vault, scopes mínimos, rotación y trazabilidad por `client_id`.
- **Relación riesgo-control:** ejemplo RE2-001 con CE2-002.
- **Dimensiones del control:** por diligenciar según evidencia.
- **Score del control:** calculado con la fórmula ponderada.
- **Evidencia esperada:** inventario de clientes, configuración de scopes, vault, logs de emisión y consumo, evidencia de rotación.
- **Fuente de evidencia:** consola AS, vault, SIEM, pipeline, documentación IAM.
- **Resultado esperado:** eficacia frente al riesgo y exposición observada.
- **Referencia:** RFC 9700, NIST SP 800-53, OWASP API Security.

#### Registro 4: Escenario 3 - API Gateway federado

- **Escenario:** API Gateway federado con Authorization Server corporativo.
- **Riesgo identificado:** confusión de audiencia o propagación indebida de tokens.
- **Activo afectado:** `access_token`, audiencia, scopes, claims, trazabilidad Gateway-backend.
- **Amenaza / causa:** Gateway valida parcialmente el token, propaga el mismo token a múltiples APIs internas o backend confía ciegamente en el Gateway.
- **Control esperado:** validación de firma/introspection, `issuer`, `audience`, expiración, scopes, autorización por operación y token exchange cuando cambie la audiencia.
- **Relación riesgo-control:** ejemplo RE3-001 con CE3-001 o CE3-002.
- **Dimensiones del control:** por diligenciar según evidencia.
- **Score del control:** calculado con la fórmula ponderada.
- **Evidencia esperada:** configuración Gateway, JWKS/introspection, reglas de audiencia, trazas, logs y pruebas de token exchange o propagación.
- **Fuente de evidencia:** Gateway, AS, Resource Servers, SIEM, APM.
- **Resultado esperado:** eficacia frente al riesgo y exposición observada.
- **Referencia:** RFC 7662, RFC 8693, RFC 9700.

---

## 6. Experimento / Aplicación de la matriz a seis corridas demostrativas

Este capítulo presenta la aplicación del modelo sobre **seis corridas demostrativas** derivadas de los tres escenarios OAuth 2.0/OIDC definidos por la tesis. La decisión metodológica consiste en construir, para cada escenario arquitectónico, un caso con evidencia favorable y un caso con evidencia débil:

1. SPA\_BFF\_IDP\_BUENO
2. SPA\_BFF\_IDP\_MALO
3. M2M\_AS\_RS\_BUENO
4. M2M\_AS\_RS\_MALO
5. API\_GATEWAY\_FEDERADO\_BUENO
6. API\_GATEWAY\_FEDERADO\_MALO

Estos seis escenarios de demostración no representan seis arquitecturas distintas, sino seis **aplicaciones controladas de la matriz**: dos por cada arquitectura evaluable. Esta decisión permite observar cómo cambia el resultado cuando el catálogo de riesgos y controles se mantiene constante, pero la evidencia, la madurez, la automatización, el momento, la periodicidad y el alcance funcional de los controles cambian.

La fuente operativa de este capítulo es el archivo `demo-prediligenciamiento.json`. Ese archivo debe entenderse como un artefacto derivado del capítulo 6: serializa en JSON los valores definidos aquí para prediligenciar el formulario web. Por tanto, los datos de activos, amenazas, vulnerabilidades, hallazgos, impacto económico estimado, evidencias y fuentes de evidencia no se generan de forma arbitraria desde la aplicación; se derivan de los escenarios demostrativos documentados en esta sección y en el anexo correspondiente.

## 6.1. Estructura de los datos demostrativos

Cada corrida demostrativa incluye dos bloques:

1. **Contexto de evaluación:** organización ficticia, evaluador, ambiente, sistema evaluado y notas de alcance.
2. **Datos de matriz:** riesgos caracterizados y controles evaluados.

Los campos usados por riesgo son:

- **asset:** activo en riesgo.
- **threat:** amenaza o causa.
- **vulnerability:** vulnerabilidad o condición habilitante.
- **probability:** probabilidad estimada.
- **impact:** impacto estimado.
- **businessLoss:** lucro cesante o impacto económico estimado del activo.
- **finding:** hallazgo, brecha u observación.

Los campos usados por control son:

- **controlCost:** costo del control.
- **maturity:** madurez.
- **automation:** automatización.
- **timing:** momento.
- **periodicity:** periodicidad.
- **scope:** alcance funcional.
- **evidenceFactor:** suficiencia de la evidencia.
- **evidenceFound:** evidencia encontrada.
- **evidenceSource:** fuente de evidencia.

La estructura conceptual del prediligenciamiento es:

## 6.2. Criterio para construir casos buenos y malos

En los casos **buenos**, los controles aparecen con valores altos en las dimensiones de evaluación:

- madurez auditada o implementada;
- automatización automática;
- momento preventivo;
- periodicidad permanente;
- alcance general cuando el control cubre la arquitectura;
- evidencia suficiente o independiente/auditada.

En los casos **malos**, los controles se configuran con valores débiles:

- controles declarados o apenas diseñados;
- operación manual;
- momento correctivo;
- periodicidad ocasional;
- alcance específico o fragmentado;
- evidencia ausente o parcial.

La comparación no busca afirmar que una organización real se encuentra en uno de esos extremos. Su función es validar que el modelo reacciona de forma esperada: cuando la evidencia y las dimensiones del control son fuertes, la eficacia frente al riesgo aumenta; cuando la evidencia es débil, la exposición observada permanece alta o crítica.

## 6.3. Escenario demostrativo: SPA estática + BFF + IdP corporativo

Para este escenario se documentan dos corridas: una buena y una mala. En ambas se conserva la misma arquitectura base y el mismo catálogo aplicable; lo que cambia es la calidad de la implementación y de la evidencia.

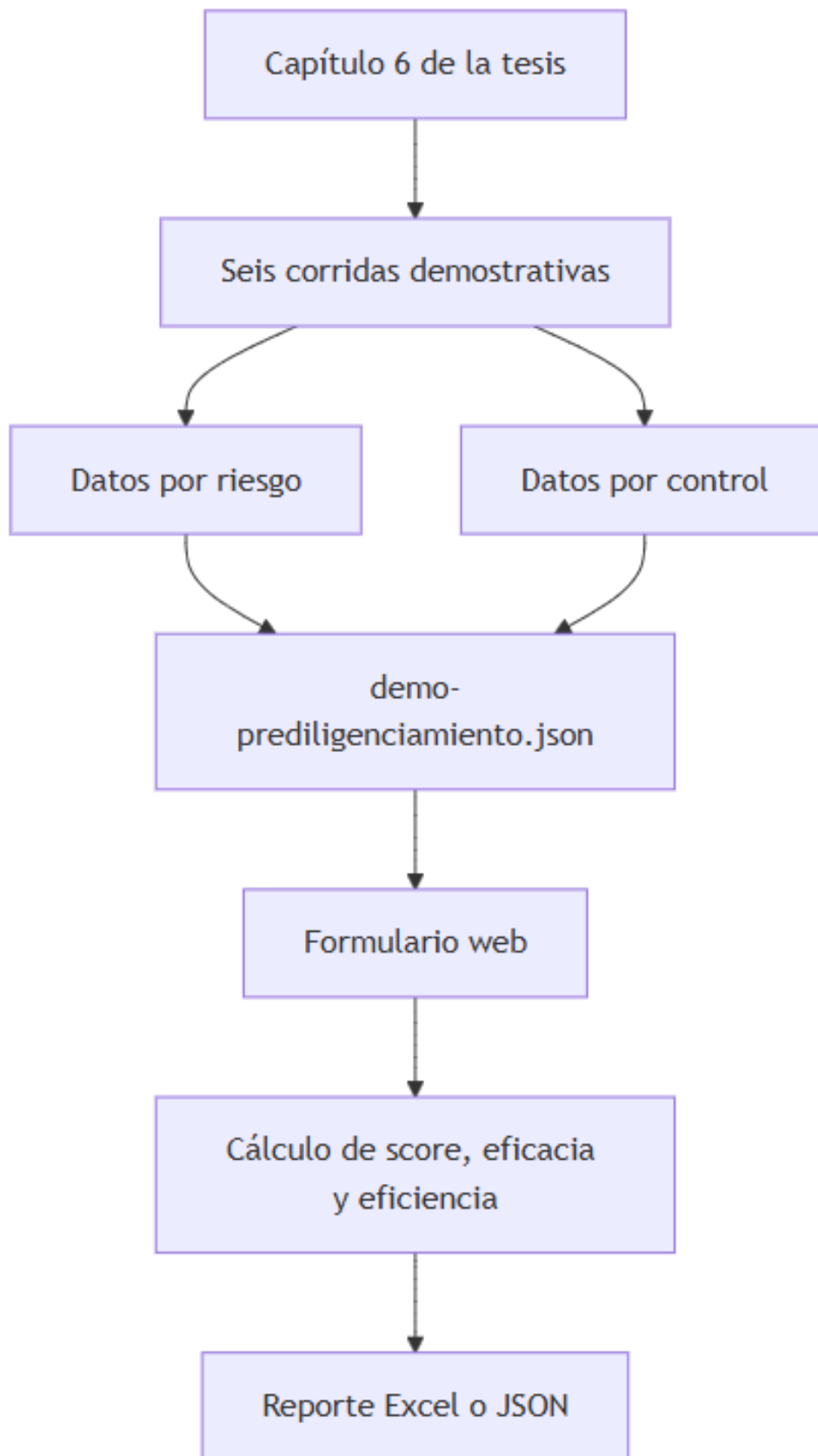


Figura 27: diagram

### 6.3.1. Contextos de evaluación

| Campo            | Caso bueno                    | Caso malo                    |
|------------------|-------------------------------|------------------------------|
| Organización     | Banco Demo OAuth - Caso bueno | Banco Demo OAuth - Caso malo |
| Ambiente         | Preproducción                 | Preproducción                |
| Sistema evaluado | Portal SPA con BFF endurecido | Portal SPA con BFF débil     |

**6.3.2. Riesgos específicos prediligenciados** Los siguientes riesgos específicos muestran cómo se materializan los campos del formulario. Los riesgos globales también se incluyen en el JSON del anexo, pero aquí se resaltan los riesgos propios del escenario para evitar duplicar narrativa.

#### Caso bueno:

| Riesgo  | Activo                               | Prob. | Impacto | Impacto económico |
|---------|--------------------------------------|-------|---------|-------------------|
| RE1-001 | authorization_code en callback OIDC  | media | alto    | 100,000           |
| RE1-002 | material de sesión en navegador      | media | alto    | 150,000           |
| RE1-003 | integridad de transacción OAuth/OIDC | media | medio   | 90,000            |

#### Caso malo:

| Riesgo  | Activo                               | Prob. | Impacto | Impacto económico |
|---------|--------------------------------------|-------|---------|-------------------|
| RE1-001 | authorization_code en callback OIDC  | alta  | critico | 125,000           |
| RE1-002 | material de sesión en navegador      | alta  | critico | 187,500           |
| RE1-003 | integridad de transacción OAuth/OIDC | alta  | critico | 112,500           |

**6.3.3. Ejemplos de controles prediligenciados** Los controles siguientes ilustran la diferencia entre un caso con evidencia fuerte y un caso con evidencia débil. El detalle completo de evidencias y fuentes de evidencia se conserva en el anexo JSON.

#### Caso bueno:

| Control | Madurez  | Autom.     | Momento    | Periodic.  | Alcance | Score  |
|---------|----------|------------|------------|------------|---------|--------|
| CE1-001 | auditado | automatico | preventivo | permanente | general | 100.00 |
| CE1-003 | auditado | automatico | preventivo | permanente | general | 100.00 |
| CE1-004 | auditado | automatico | preventivo | permanente | general | 100.00 |

#### Caso malo:

| Control | Madurez   | Autom. | Momento    | Periodic. | Alcance    | Score |
|---------|-----------|--------|------------|-----------|------------|-------|
| CE1-001 | declarado | manual | correctivo | ocasional | especifico | 17.50 |
| CE1-003 | declarado | manual | correctivo | ocasional | especifico | 17.50 |
| CE1-004 | declarado | manual | correctivo | ocasional | especifico | 17.50 |

### 6.3.4. Resultado cuantitativo del escenario

| Corrida | Score prom. control | Eficacia global | Eficiencia global | Eficacia cual. | Eficiencia cual. |
|---------|---------------------|-----------------|-------------------|----------------|------------------|
| Bueno   | 99.62               | 99.75 %         | 28.80             | Alta           | Alta             |

| Corrida | Score prom. control | Eficacia global | Eficiencia global | Eficacia cual. | Eficiencia cual. |
|---------|---------------------|-----------------|-------------------|----------------|------------------|
| Malo    | 14.81               | 15.25 %         | 5.60              | Baja           | Alta             |

Lectura: la diferencia principal entre las dos corridas no está en el catálogo, sino en la evidencia y en el estado del control. La corrida buena muestra controles con mayor score promedio y mayor eficacia frente al riesgo. La corrida mala conserva impacto económico relevante, pero con controles débiles; por eso la exposición observada se mantiene alta o crítica en los riesgos más sensibles.

#### 6.4. Escenario demostrativo: M2M con Authorization Server como Resource Server

Para este escenario se documentan dos corridas: una buena y una mala. En ambas se conserva la misma arquitectura base y el mismo catálogo aplicable; lo que cambia es la calidad de la implementación y de la evidencia.

##### 6.4.1. Contextos de evaluación

| Campo            | Caso bueno                    | Caso malo                     |
|------------------|-------------------------------|-------------------------------|
| Organización     | Fintech Demo M2M - Caso bueno | Fintech Demo M2M - Caso malo  |
| Ambiente         | Producción                    | Producción                    |
| Sistema evaluado | Servicios M2M IAM gobernados  | Servicios M2M IAM con brechas |

**6.4.2. Riesgos específicos prediligenciados** Los siguientes riesgos específicos muestran cómo se materializan los campos del formulario. Los riesgos globales también se incluyen en el JSON del anexo, pero aquí se resaltan los riesgos propios del escenario para evitar duplicar narrativa.

##### Caso bueno:

| Riesgo  | Activo                       | Prob. | Impacto | Impacto económico |
|---------|------------------------------|-------|---------|-------------------|
| RE2-001 | cliente técnico privilegiado | media | crítico | 120,000           |
| RE2-002 | credenciales M2M             | media | alto    | 95,000            |
| RE2-003 | plano AS+RS administrativo   | media | alto    | 105,000           |

##### Caso malo:

| Riesgo  | Activo                       | Prob. | Impacto | Impacto económico |
|---------|------------------------------|-------|---------|-------------------|
| RE2-001 | cliente técnico privilegiado | alta  | crítico | 150,000           |
| RE2-002 | credenciales M2M             | alta  | crítico | 118,750           |
| RE2-003 | plano AS+RS administrativo   | alta  | crítico | 131,250           |

**6.4.3. Ejemplos de controles prediligenciados** Los controles siguientes ilustran la diferencia entre un caso con evidencia fuerte y un caso con evidencia débil. El detalle completo de evidencias y fuentes de evidencia se conserva en el anexo JSON.

##### Caso bueno:

| Control | Madurez  | Autom.     | Momento    | Periodic.  | Alcance | Score  |
|---------|----------|------------|------------|------------|---------|--------|
| CE2-001 | auditado | automatico | preventivo | permanente | general | 100.00 |
| CE2-002 | auditado | automatico | preventivo | permanente | general | 100.00 |
| CE2-004 | auditado | automatico | preventivo | permanente | general | 100.00 |



**Caso malo:**

| Control | Madurez   | Autom. | Momento    | Periodic. | Alcance    | Score |
|---------|-----------|--------|------------|-----------|------------|-------|
| CE2-001 | declarado | manual | correctivo | ocasional | especifico | 17.50 |
| CE2-002 | declarado | manual | correctivo | ocasional | especifico | 17.50 |
| CE2-004 | declarado | manual | correctivo | ocasional | especifico | 17.50 |

**6.4.4. Resultado cuantitativo del escenario**

| Corrida | Score prom. control | Eficacia global | Eficiencia global | Eficacia cual. | Eficiencia cual. |
|---------|---------------------|-----------------|-------------------|----------------|------------------|
| Bueno   | 99.62               | 94.75 %         | 43.78             | Alta           | Alta             |
| Malo    | 14.81               | 14.12 %         | 8.33              | Baja           | Alta             |

Lectura: la diferencia principal entre las dos corridas no está en el catálogo, sino en la evidencia y en el estado del control. La corrida buena muestra controles con mayor score promedio y mayor eficacia frente al riesgo. La corrida mala conserva impacto económico relevante, pero con controles débiles; por eso la exposición observada se mantiene alta o crítica en los riesgos más sensibles.

**6.5. Escenario demostrativo: API Gateway federado + Authorization Server corporativo**

Para este escenario se documentan dos corridas: una buena y una mala. En ambas se conserva la misma arquitectura base y el mismo catálogo aplicable; lo que cambia es la calidad de la implementación y de la evidencia.

**6.5.1. Contextos de evaluación**

| Campo            | Caso bueno                                    | Caso malo                                 |
|------------------|-----------------------------------------------|-------------------------------------------|
| Organización     | Holding APIs Federadas - Caso bueno           | Holding APIs Federadas - Caso malo        |
| Ambiente         | Producción                                    | Producción                                |
| Sistema evaluado | Gateway federado con validación y downscoping | Gateway federado con propagación insegura |

**6.5.2. Riesgos específicos prediligenciados** Los siguientes riesgos específicos muestran cómo se materializan los campos del formulario. Los riesgos globales también se incluyen en el JSON del anexo, pero aquí se resaltan los riesgos propios del escenario para evitar duplicar narrativa.

**Caso bueno:**

| Riesgo  | Activo                             | Prob. | Impacto | Impacto económico |
|---------|------------------------------------|-------|---------|-------------------|
| RE3-001 | audience por API destino           | media | alto    | 110,000           |
| RE3-002 | confianza backend en gateway       | media | alto    | 115,000           |
| RE3-003 | propagación de permisos entre APIs | media | critico | 140,000           |

**Caso malo:**

| Riesgo  | Activo                             | Prob. | Impacto | Impacto económico |
|---------|------------------------------------|-------|---------|-------------------|
| RE3-001 | audience por API destino           | alta  | critico | 137,500           |
| RE3-002 | confianza backend en gateway       | alta  | critico | 143,750           |
| RE3-003 | propagación de permisos entre APIs | alta  | critico | 175,000           |

**6.5.3. Ejemplos de controles prediligenciados** Los controles siguientes ilustran la diferencia entre un caso con evidencia fuerte y un caso con evidencia débil. El detalle completo de evidencias y fuentes de evidencia se conserva en el anexo JSON.

**Caso bueno:**

| Control | Madurez      | Autom.     | Momento    | Periodic.  | Alcance | Score  |
|---------|--------------|------------|------------|------------|---------|--------|
| CE3-001 | auditado     | automatico | preventivo | permanente | general | 100.00 |
| CE3-002 | implementado | automatico | preventivo | permanente | general | 90.00  |
| CE3-004 | auditado     | automatico | preventivo | permanente | general | 100.00 |

**Caso malo:**

| Control | Madurez   | Autom. | Momento    | Periodic. | Alcance    | Score |
|---------|-----------|--------|------------|-----------|------------|-------|
| CE3-001 | declarado | manual | correctivo | ocasional | especifico | 17.50 |
| CE3-002 | declarado | manual | correctivo | ocasional | especifico | 17.50 |
| CE3-004 | declarado | manual | correctivo | ocasional | especifico | 17.50 |

**6.5.4. Resultado cuantitativo del escenario**

| Corrida | Score prom. control | Eficacia global | Eficiencia global | Eficacia cual. | Eficiencia cual. |
|---------|---------------------|-----------------|-------------------|----------------|------------------|
| Bueno   | 99.05               | 83.31 %         | 26.16             | Alta           | Alta             |
| Malo    | 15.12               | 12.89 %         | 4.91              | Baja           | Alta             |

Lectura: la diferencia principal entre las dos corridas no está en el catálogo, sino en la evidencia y en el estado del control. La corrida buena muestra controles con mayor score promedio y mayor eficacia frente al riesgo. La corrida mala conserva impacto económico relevante, pero con controles débiles; por eso la exposición observada se mantiene alta o crítica en los riesgos más sensibles.

**6.6. Comparación transversal de las seis corridas**

La siguiente tabla resume las seis corridas demostrativas. Esta comparación es posible porque todas usan el mismo catálogo maestro, la misma lógica de cálculo y la misma estructura de formulario.

| Corrida demostrativa | Score prom. control | Eficacia global | Eficiencia global | Eficacia cual. | Eficiencia cual. |
|----------------------|---------------------|-----------------|-------------------|----------------|------------------|
| SPA+BFF+IdP bueno    | 99.62               | 99.75 %         | 28.80             | Alta           | Alta             |
| SPA+BFF+IdP malo     | 14.81               | 15.25 %         | 5.60              | Baja           | Alta             |
| M2M AS=RS bueno      | 99.62               | 94.75 %         | 43.78             | Alta           | Alta             |
| M2M AS=RS malo       | 14.81               | 14.12 %         | 8.33              | Baja           | Alta             |
| API Gateway bueno    | 99.05               | 83.31 %         | 26.16             | Alta           | Alta             |
| API Gateway malo     | 15.12               | 12.89 %         | 4.91              | Baja           | Alta             |

La eficiencia cuantitativa debe leerse junto con la eficacia. En algunos casos malos puede aparecer una eficiencia relativa alta porque el costo declarado del control es bajo frente al impacto económico estimado. Sin embargo, si la eficacia es baja, el resultado no debe interpretarse como una arquitectura saludable. Por esa razón, la lectura ejecutiva debe considerar simultáneamente:

1. score promedio de control;
2. eficacia frente al riesgo;
3. nivel de exposición observado;
4. suficiencia de evidencia;
5. eficiencia económica de la mitigación.

#### 6.7. Resultados por riesgo en cada corrida

Las siguientes tablas presentan el resultado consolidado por riesgo. Se omiten las descripciones largas para conservar legibilidad; los valores narrativos completos de activo, amenaza, vulnerabilidad, hallazgo, evidencia y fuente de evidencia quedan en el anexo JSON.

##### SPA+BFF+IdP bueno

| Riesgo  | Nivel riesgo | Eficacia | Exposición | Eficiencia | Costo asignado |
|---------|--------------|----------|------------|------------|----------------|
| RG-001  | Alto         | 98.00 %  | Bajo       | 8.73       | 7,293.94 USD   |
| RG-002  | Alto         | 100.00 % | Bajo       | 35.97      | 2,223.82 USD   |
| RG-003  | Medio        | 100.00 % | Bajo       | 24.10      | 2,074.51 USD   |
| RG-004  | Medio        | 100.00 % | Bajo       | 13.22      | 3,403.33 USD   |
| RG-005  | Medio        | 100.00 % | Bajo       | 15.10      | 2,648.33 USD   |
| RE1-001 | Alto         | 100.00 % | Bajo       | 55.54      | 1,800.54 USD   |
| RE1-002 | Alto         | 100.00 % | Bajo       | 28.75      | 5,217.42 USD   |
| RE1-003 | Medio        | 100.00 % | Bajo       | 48.96      | 1,838.10 USD   |

##### SPA+BFF+IdP malo

| Riesgo  | Nivel riesgo | Eficacia | Exposición | Eficiencia | Costo asignado |
|---------|--------------|----------|------------|------------|----------------|
| RG-001  | Crítico      | 11.50 %  | Crítico    | 1.28       | 7,293.94 USD   |
| RG-002  | Crítico      | 11.50 %  | Crítico    | 5.17       | 2,223.82 USD   |
| RG-003  | Crítico      | 15.50 %  | Crítico    | 4.67       | 2,074.51 USD   |
| RG-004  | Crítico      | 17.50 %  | Crítico    | 2.89       | 3,403.33 USD   |
| RG-005  | Crítico      | 16.00 %  | Crítico    | 3.02       | 2,648.33 USD   |
| RE1-001 | Crítico      | 16.00 %  | Crítico    | 11.11      | 1,800.54 USD   |
| RE1-002 | Crítico      | 16.50 %  | Crítico    | 5.93       | 5,217.42 USD   |
| RE1-003 | Crítico      | 17.50 %  | Crítico    | 10.71      | 1,838.10 USD   |

##### M2M AS=RS bueno

| Riesgo  | Nivel riesgo | Eficacia | Exposición | Eficiencia | Costo asignado |
|---------|--------------|----------|------------|------------|----------------|
| RG-001  | Alto         | 60.00 %  | Medio      | 5.35       | 7,850.54 USD   |
| RG-002  | Crítico      | 100.00 % | Bajo       | 66.20      | 1,812.74 USD   |
| RG-003  | Alto         | 100.00 % | Bajo       | 73.44      | 1,497.74 USD   |
| RG-004  | Alto         | 100.00 % | Bajo       | 56.12      | 2,494.81 USD   |
| RG-005  | Alto         | 100.00 % | Bajo       | 35.71      | 2,520.09 USD   |
| RE2-001 | Crítico      | 100.00 % | Bajo       | 56.94      | 2,107.33 USD   |
| RE2-002 | Alto         | 98.00 %  | Bajo       | 11.76      | 7,919.05 USD   |
| RE2-003 | Alto         | 100.00 % | Bajo       | 44.72      | 2,347.70 USD   |

##### M2M AS=RS malo

| Riesgo  | Nivel riesgo | Eficacia | Exposición | Eficiencia | Costo asignado |
|---------|--------------|----------|------------|------------|----------------|
| RG-001  | Crítico      | 4.50 %   | Crítico    | 0.50       | 7,850.54 USD   |
| RG-002  | Crítico      | 11.50 %  | Crítico    | 9.52       | 1,812.74 USD   |
| RG-003  | Crítico      | 15.50 %  | Crítico    | 14.23      | 1,497.74 USD   |
| RG-004  | Crítico      | 17.50 %  | Crítico    | 12.28      | 2,494.81 USD   |
| RG-005  | Crítico      | 16.00 %  | Crítico    | 7.14       | 2,520.09 USD   |
| RE2-001 | Crítico      | 17.50 %  | Crítico    | 12.46      | 2,107.33 USD   |
| RE2-002 | Crítico      | 16.00 %  | Crítico    | 2.40       | 7,919.05 USD   |
| RE2-003 | Crítico      | 14.50 %  | Crítico    | 8.11       | 2,347.70 USD   |

#### API Gateway bueno

| Riesgo  | Nivel riesgo | Eficacia | Exposición | Eficiencia | Costo asignado |
|---------|--------------|----------|------------|------------|----------------|
| RG-001  | Alto         | 50.00 %  | Medio      | 5.15       | 8,250.00 USD   |
| RG-002  | Alto         | 100.00 % | Bajo       | 51.05      | 1,860.83 USD   |
| RG-003  | Alto         | 80.00 %  | Bajo       | 44.54      | 1,616.67 USD   |
| RG-004  | Medio        | 65.00 %  | Medio      | 13.37      | 3,403.33 USD   |
| RG-005  | Medio        | 80.00 %  | Bajo       | 18.47      | 2,598.33 USD   |
| RE3-001 | Alto         | 96.50 %  | Bajo       | 27.71      | 3,831.37 USD   |
| RE3-002 | Alto         | 100.00 % | Bajo       | 29.63      | 3,880.78 USD   |
| RE3-003 | Crítico      | 95.00 %  | Bajo       | 19.39      | 6,858.68 USD   |

#### API Gateway malo

| Riesgo  | Nivel riesgo | Eficacia | Exposición | Eficiencia | Costo asignado |
|---------|--------------|----------|------------|------------|----------------|
| RG-001  | Crítico      | 5.75 %   | Crítico    | 0.74       | 8,250.00 USD   |
| RG-002  | Crítico      | 11.50 %  | Crítico    | 7.34       | 1,860.83 USD   |
| RG-003  | Crítico      | 12.00 %  | Crítico    | 8.35       | 1,616.67 USD   |
| RG-004  | Crítico      | 11.37 %  | Crítico    | 2.92       | 3,403.33 USD   |
| RG-005  | Crítico      | 12.50 %  | Crítico    | 3.61       | 2,598.33 USD   |
| RE3-001 | Crítico      | 17.50 %  | Crítico    | 6.28       | 3,831.37 USD   |
| RE3-002 | Crítico      | 15.00 %  | Crítico    | 5.56       | 3,880.78 USD   |
| RE3-003 | Crítico      | 17.50 %  | Crítico    | 4.47       | 6,858.68 USD   |

## 6.8. Relación entre el capítulo 6 y el formulario web

El archivo demo-prediligenciamiento.json permite cargar en el formulario los seis escenarios demostrativos sin que el evaluador tenga que escribir manualmente todos los campos. En términos de trazabilidad, la relación es la siguiente:

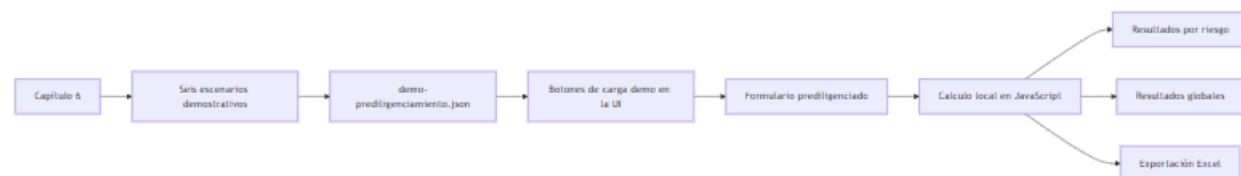


Figura 28: diagram

Esta separación permite que el modelo sea revisable en tres niveles:

1. **Nivel académico:** la tesis explica el origen de los valores, los riesgos, controles y fórmulas.

2. **Nivel de datos:** el JSON conserva los seis conjuntos de valores de prediligenciamiento.
3. **Nivel operativo:** la UI consume el JSON y reproduce la evaluación.

**6.8.1. Correspondencia de notación entre tesis y formulario** El formulario usa MathJax para renderizar las ecuaciones en el navegador. La notación mostrada en la UI coincide con la usada en la sección 5.1.6:

$$\begin{aligned}
 S_{base(raw)} &= 0.40M + 0.20A + 0.10T + 0.20P + 0.10L \\
 S_{base} &= \text{norm}(S_{base(raw)}) \\
 F_{ev} &= \text{norm}(f_{ev}) \\
 S_{control} &= S_{base} \times F_{ev}
 \end{aligned}$$

Para cada relación riesgo-control, el formulario muestra también el peso de mitigación y el aporte ponderado:

$$Aporte_i = w_i \times S_{control}(c_i)$$

Para cada riesgo, el formulario muestra:

$$E_r = \frac{\sum(w_i \cdot s_i)}{\sum w_i}$$

Donde  $s_i$  corresponde al  $S_{control}$  del control  $i$  y  $w_i$  al pesoMitigacion definido en la relación riesgo-control.

También muestra la eficiencia por riesgo:

$$\eta_r = \frac{Beneficio}{Costo} = \frac{B_r}{C_r}$$

Finalmente, la UI muestra las ecuaciones globales:

$$\begin{aligned}
 E_{global} &= \frac{1}{m} \sum_{r=1}^m E_r \\
 \eta_{global} &= \frac{1}{m} \sum_{r=1}^m \eta_r
 \end{aligned}$$

Esta correspondencia evita que el formulario sea una caja negra. El evaluador puede ver los valores diligenciados, los resultados parciales y la expresión matemática usada para calcular cada resultado.

## 6.9. Síntesis del experimento

La aplicación de la matriz en seis corridas demuestra que el modelo no se limita a listar controles. El mismo catálogo produce resultados distintos cuando cambian las evidencias y el estado real de los controles. Esto confirma la utilidad del enfoque de aseguramiento: la arquitectura no se considera segura por usar OAuth 2.0 u OIDC, sino por demostrar controles diseñados, implementados, evidenciados y trazables frente a riesgos específicos.

La comparación bueno/malo por escenario también permite validar el valor del catálogo maestro. Los riesgos globales se mantienen constantes, los riesgos específicos cambian según la arquitectura y los controles esperados se ponderan mediante relaciones riesgo-control. De esta forma, el modelo evita dos extremos: una lista genérica de buenas prácticas sin contexto, y una evaluación puramente subjetiva sin trazabilidad técnica.

## 7. Análisis de resultados

Este capítulo analiza los resultados obtenidos al aplicar la matriz a las seis corridas demostrativas del capítulo 6. La lectura de resultados se concentra en validar si el modelo permite diferenciar entre arquitecturas con controles fuertes y arquitecturas con controles débiles, manteniendo constante el catálogo de riesgos y controles.

La comparación demuestra que la matriz no funciona como una lista binaria de cumplimiento. El resultado cambia según la madurez, automatización, momento, periodicidad, alcance y evidencia de los controles observados.

---

### 7.1. Riesgos más relevantes identificados

Los riesgos más relevantes no son iguales en todos los escenarios. Cada arquitectura desplaza la exposición hacia activos y fronteras distintas.

En **SPA+BFF+IdP**, los riesgos más sensibles se concentran en:

- interceptación o sustitución del `authorization_code`;
- exposición de tokens o material de sesión en el navegador;
- validación débil de `state` y `nonce`;
- configuración insegura de `cookie` o `callback`;
- custodia incorrecta de tokens fuera del BFF.

En **M2M AS=RS**, los riesgos más sensibles se concentran en:

- abuso de cliente técnico privilegiado;
- secretos o certificados expuestos;
- scopes administrativos excesivos;
- falta de rotación;
- ausencia de responsable del cliente técnico;
- concentración lógica entre emisión de tokens y APIs administrativas.

En **API Gateway federado**, los riesgos más sensibles se concentran en:

- confusión de audiencia;
- propagación excesiva del token original;
- backends que confían ciegamente en el Gateway;
- ausencia de token exchange o downscoping;
- validación inconsistente entre Gateway y Resource Servers;
- trazabilidad fragmentada extremo a extremo.

La matriz permite observar que la seguridad de OAuth 2.0/OIDC depende de la arquitectura. Un mismo control transversal, como validación de tokens o logging, puede tener diferente importancia según el escenario.

---

### 7.2. Controles críticos por escenario

El análisis de los seis casos permite identificar controles críticos por arquitectura.

**7.2.1. SPA+BFF+IdP** Los controles críticos son:

- Authorization Code + PKCE con S256;
- rechazo de `plain` cuando no esté justificado;
- validación de `state`;
- validación de `nonce`;
- redirect URI exacta;
- tokens custodiados server-side;
- `cookie HttpOnly, Secure y SameSite`;

- evidencia de que la SPA no conserva tokens.

En este escenario, PKCE es nuclear para mitigar interceptación del código de autorización, pero no es suficiente por sí solo. Debe complementarse con controles de sesión, callback, cookies, CSP y custodia server-side.

#### 7.2.2. M2M AS=RS Los controles críticos son:

- cliente técnico con dueño y propósito definido;
- scopes mínimos;
- separación por ambiente;
- secretos o certificados en vault;
- rotación de credenciales;
- autenticación robusta del cliente;
- monitoreo por `client_id`;
- recertificación periódica de permisos.

En este escenario, el riesgo no está en un usuario humano, sino en identidades no humanas. La trazabilidad por `client_id` se vuelve indispensable para atribuir operaciones.

#### 7.2.3. API Gateway federado Los controles críticos son:

- validación de issuer;
- validación de audience;
- validación de expiración;
- validación de scopes;
- revalidación mínima en backends críticos;
- token exchange cuando cambia la audiencia;
- reducción de scopes o downscoping;
- correlación de logs entre Gateway, Authorization Server y Resource Servers.

En este escenario, el Gateway no debería ser el único punto de confianza. La matriz penaliza diseños donde los backends aceptan contexto propagado sin validación mínima.

---

### 7.3. Evidencias auditables más importantes

La evidencia es el elemento que transforma una declaración en una evaluación sustentada. Los demos malos muestran que controles declarados, sin evidencia, no aportan mitigación demostrable.

Las evidencias más relevantes son:

- configuración del cliente OAuth/OIDC;
- configuración de PKCE y redirect URI;
- logs de callback, login y emisión de tokens;
- configuración de cookies;
- evidencia de tokens fuera del navegador;
- inventario de clientes técnicos;
- configuración de scopes y audiencias;
- evidencia de vault y rotación de secretos;
- configuración del Gateway;
- reglas de introspection o JWKS;
- pruebas de rechazo de tokens inválidos;
- trazas y logs correlados entre componentes.

La evidencia debe ser suficiente para comprobar operación, no solo intención. Por ejemplo, una política que exige rotación de secretos no prueba que la rotación se haya ejecutado. Se requiere evidencia de ejecución o configuración vigente.

---

#### 7.4. Diferencias entre autenticación, autorización y federación desde el riesgo

El análisis confirma que autenticación, autorización y federación generan riesgos distintos.

La **autenticación** responde quién es el sujeto. En OIDC, esto se representa mediante el `id_token` y claims asociados. Sus riesgos se relacionan con replay, sustitución, nonce, emisor, audiencia y validez del evento de autenticación.

La **autorización** responde qué puede hacer el sujeto o cliente. En OAuth 2.0, esto se representa mediante `access_token`, scopes, audiencias y políticas del Resource Server. Sus riesgos se relacionan con scopes excesivos, audiencia incorrecta, tokens expirados o validación incompleta.

La **federación** introduce relaciones de confianza entre componentes. Sus riesgos se relacionan con qué emisor se acepta, qué claims se consumen, qué componente valida y qué evidencia queda disponible para auditoría.

Esta diferencia es crítica: una arquitectura puede autenticar correctamente al usuario y, aun así, autorizar mal el acceso a una API.

---

#### 7.5. Hallazgos sobre el uso seguro de OAuth 2.0/OIDC

El experimento deja varios hallazgos:

1. **OAuth 2.0/OIDC no es un control único.**

Es una base técnica que requiere controles complementarios de configuración, validación, evidencia y operación.

2. **El patrón BFF reduce exposición de tokens en navegador, pero no elimina riesgos de sesión.**

La cookie, CSRF, XSS, sesión server-side y trazabilidad siguen siendo controles críticos.

3. **La identidad no humana requiere gobierno propio.**

Los clientes M2M no deben tratarse como integraciones secundarias. Pueden tener privilegios altos y baja visibilidad si no se gobiernan.

4. **El Gateway no debe concentrar toda la confianza.**

En arquitecturas federadas, los Resource Servers críticos deben validar al menos audiencia, scopes y autorización de negocio.

5. **La evidencia cambia el resultado.**

Un control declarado sin evidencia suficiente no debe aportar el mismo valor que un control auditado.

6. **La eficiencia económica no debe leerse aislada.**

Un caso malo puede mostrar eficiencia aparente alta si el costo declarado del control es bajo frente al impacto estimado. Por eso la eficacia y el nivel de exposición observado son indispensables.

---

#### 7.6. Valor de la matriz como artefacto de evaluación

La matriz aporta valor porque permite:

- cargar riesgos y controles predefinidos por arquitectura;
- evitar que cada evaluador invente su propio checklist;
- relacionar cada riesgo con controles y pesos de mitigación;
- diferenciar controles globales y específicos;
- registrar evidencia;
- calcular score de control;
- calcular eficacia por riesgo;
- calcular eficiencia por riesgo;
- generar resultados comparables entre escenarios.



El valor principal no está en producir un número final, sino en obligar a documentar la cadena completa:

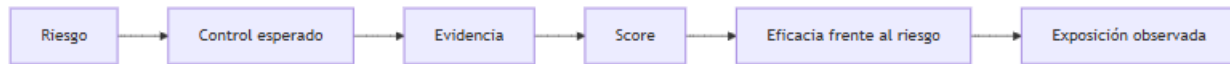


Figura 29: diagram

Esta cadena hace que la evaluación sea trazable, revisable y justificable ante un tercero.

## 8. Conclusiones y trabajo futuro

### 8.1. Conclusiones frente al objetivo general

El objetivo general de esta tesis fue diseñar un modelo de evaluación de controles de seguridad para arquitecturas de identidad federada basadas en OAuth 2.0 y OpenID Connect desde un enfoque de aseguramiento orientado a riesgos.

Este objetivo se cumple mediante la construcción de un modelo que integra:

- escenarios arquitectónicos representativos;
- catálogo maestro de riesgos y controles;
- relaciones riesgo-control con pesos de mitigación;
- dimensiones de evaluación de controles;
- evidencia auditable;
- fórmulas de score, eficacia y eficiencia;
- entregable web para aplicar la evaluación.

La tesis demuestra que OAuth 2.0/OIDC no debe evaluarse como una etiqueta tecnológica, sino como una arquitectura de confianza compuesta por clientes, Authorization Server, Resource Servers, tokens, sesiones, scopes, audiencias, secretos, trazabilidad y evidencia.

---

### 8.2. Conclusiones frente a los objetivos específicos

**Objetivo específico 1** Se caracterizaron conceptos fundamentales como identidad digital, autenticación, autorización, federación, tokens, clientes, Authorization Server, Resource Server, PKCE, introspection, token exchange, BFF, cookies y API Gateway.

La principal conclusión es que la seguridad de identidad federada depende de comprender qué componente autentica, qué componente autoriza, quién emite el token, quién lo valida y qué evidencia queda disponible.

**Objetivo específico 2** Se identificaron riesgos globales y específicos asociados a OAuth 2.0/OIDC. Los riesgos globales incluyen exposición de tokens, validación insuficiente, scopes excesivos, gobierno débil de clientes y trazabilidad insuficiente. Los riesgos específicos dependen de la arquitectura: SPA+BFF+IdP, M2M AS=RS y API Gateway federado.

La principal conclusión es que el riesgo no es uniforme. Cambia según el flujo, el tipo de cliente, el rol del componente y la frontera de confianza.

**Objetivo específico 3** Se definieron criterios de evaluación para analizar controles mediante dimensiones de madurez, automatización, momento, periodicidad, alcance funcional y evidencia. También se definieron ponderadores, pesos de mitigación, costo asignado, eficacia y eficiencia.

La principal conclusión es que un control no debe evaluarse solo por existir. Debe evaluarse por su estado, evidencia y contribución al riesgo específico.

**Objetivo específico 4** Se aplicó el modelo a tres escenarios OAuth 2.0/OIDC mediante seis corridas demostrativas: una buena y una mala por cada escenario.

La principal conclusión es que el modelo diferencia adecuadamente entre controles fuertes y controles débiles. Manteniendo el mismo catálogo, la eficacia cambia cuando cambian la evidencia y el estado de los controles.

**Objetivo específico 5** Se condensó el modelo en un entregable web basado en catálogo JSON, formulario de evaluación, cálculo local de resultados y exportación de informe.

La principal conclusión es que el entregable web operacionaliza la tesis y permite reproducir los cálculos definidos en el documento, evitando que el modelo quede como un artefacto puramente teórico.

---

### 8.3. Aporte de la tesis

El aporte principal de esta tesis es un modelo aplicado para evaluar controles de seguridad en arquitecturas OAuth 2.0/OIDC desde la relación entre riesgo, control, evidencia y resultado.

El aporte se materializa en cuatro artefactos:

1. **Marco arquitectónico:** dos casos base y tres escenarios evaluables.
2. **Catálogo maestro:** riesgos, controles, relaciones, pesos y referencias.
3. **Matriz de evaluación:** campos, dimensiones, fórmulas y reglas de cálculo.
4. **Entregable web:** formulario, prediligenciamiento demo y exportación de resultados.

El modelo permite pasar de afirmaciones generales como “usamos OAuth 2.0” a preguntas auditables como:

- ¿qué riesgos aplica esta arquitectura?
  - ¿qué controles deberían existir?
  - ¿qué evidencia sustenta el estado del control?
  - ¿qué tan eficaz es el conjunto de controles frente al riesgo?
  - ¿qué exposición permanece?
- 

### 8.4. Limitaciones del trabajo

La tesis tiene las siguientes limitaciones:

1. **No cubre todos los flujos OAuth 2.0/OIDC.**  
Se concentra en Authorization Code + PKCE, Client Credentials, introspection, validación bearer y token exchange.
2. **No reemplaza una auditoría formal.**  
El modelo produce una evaluación académica y semicuantitativa. Requiere juicio profesional para aplicarse en entornos reales.
3. **Los costos son estimaciones.**  
El cálculo de eficiencia depende del costo asignado y del impacto económico estimado. Estos valores pueden variar por organización.
4. **Los pesos de mitigación son parametrizables.**  
Aunque se justifican metodológicamente, pueden ajustarse según criticidad, apetito de riesgo y contexto.
5. **La evidencia depende de disponibilidad.**  
Si la organización no conserva logs, configuraciones o pruebas, el modelo puede penalizar controles aunque existan parcialmente.
6. **El entregable web no automatiza una auditoría completa.**  
La UI guía la evaluación, pero no inspecciona automáticamente todos los sistemas ni valida técnicamente todos los controles.

---

## 8.5. Trabajo futuro

Como trabajo futuro se proponen las siguientes líneas:

1. **Validación con expertos.**  
Aplicar el modelo con arquitectos de seguridad, auditores, responsables IAM y equipos de desarrollo para ajustar pesos, campos y evidencias.
2. **Expansión de escenarios.**  
Incluir aplicaciones móviles, device authorization grant, CIBA, OAuth 2.1, FAPI, mTLS sender-constrained tokens y DPoP.
3. **Automatización de evidencias.**  
Integrar conectores con proveedores como Keycloak, Microsoft Entra ID, Auth0, Okta o API Gateways para extraer configuración y logs.
4. **Mejora del cálculo de riesgo residual.**  
Incorporar modelos más refinados para combinar probabilidad, impacto, eficacia y exposición residual.
5. **Generación de reportes ejecutivos.**  
Complementar el Excel con reportes PDF o dashboards que permitan priorizar riesgos y controles.
6. **Control de versionamiento del catálogo.**  
Publicar versiones del catálogo y permitir comparación entre evaluaciones históricas.
7. **Pruebas técnicas automatizadas.**  
Incorporar validadores de redirect URI, PKCE, JWKS, claims, scopes, expiración, audience e issuer.
8. **Extensión a Zero Trust e identidad de cargas de trabajo.**  
Relacionar el modelo con patrones de identidad no humana, workload identity, service mesh y autorización continua.

## 9. Referencias

Valencia Duque, F. J. *Aseguramiento y auditoría de tecnologías de información orientados a riesgos: un enfoque basado en estándares internacionales*.

Hardt, D. (2012). *The OAuth 2.0 Authorization Framework* (RFC 6749). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6749>

Jones, M., & Hardt, D. (2012). *The OAuth 2.0 Authorization Framework: Bearer Token Usage* (RFC 6750). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6750>

Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., & Mortimore, C. (2014). *OpenID Connect Core 1.0*. OpenID Foundation. [https://openid.net/specs/openid-connect-core-1\\_0.html](https://openid.net/specs/openid-connect-core-1_0.html)

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Signature (JWS)* (RFC 7515). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7515>

Jones, M., & Hildebrand, J. (2015). *JSON Web Encryption (JWE)* (RFC 7516). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7516>

Jones, M. (2015). *JSON Web Key (JWK)* (RFC 7517). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7517>

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>

Sakimura, N., Bradley, J., & Jones, M. (2015). *Proof Key for Code Exchange by OAuth Public Clients* (RFC 7636). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7636>

Richer, J. (2015). *OAuth 2.0 Token Introspection* (RFC 7662). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7662>

Jones, M., Nadalin, A., Campbell, B., Bradley, J., & Mortimore, C. (2020). *OAuth 2.0 Token Exchange* (RFC 8693). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8693>

Campbell, B., Bradley, J., Sakimura, N., & Lodderstedt, T. (2020). *OAuth 2.0 Mutual-TLS Client Authentication and Certificate-Bound Access Tokens* (RFC 8705). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8705>

Bertocci, V. (2021). *JSON Web Token (JWT) Profile for OAuth 2.0 Access Tokens* (RFC 9068). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9068>

Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., & Waite, D. (2023). *OAuth 2.0 Demonstrating Proof of Possession* (RFC 9449). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9449>

Lodderstedt, T., Bradley, J., Labunets, A., Fett, D., & Campbell, B. (2024). *Best Current Practice for OAuth 2.0 Security* (RFC 9700). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9700>

OWASP Foundation. *OAuth 2.0 Cheat Sheet*. [https://cheatsheetseries.owasp.org/cheatsheets/OAuth2\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/OAuth2_Cheat_Sheet.html)

OWASP Foundation. *Session Management Cheat Sheet*. [https://cheatsheetseries.owasp.org/cheatsheets/Session\\_Management\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html)

OWASP Foundation. *Cross-Site Request Forgery Prevention Cheat Sheet*. [https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site\\_Request\\_Forgery\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html)

OWASP Foundation. *Application Security Verification Standard*. <https://owasp.org/www-project-application-security-verification-standard/>

National Institute of Standards and Technology. (2012). *Guide for Conducting Risk Assessments* (NIST SP 800-30 Rev. 1). <https://csrc.nist.gov/publications/detail/sp/800-30/rev-1/final>

National Institute of Standards and Technology. (2020). *Security and Privacy Controls for Information Systems and Organizations* (NIST SP 800-53 Rev. 5). <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>

National Institute of Standards and Technology. (2024). *The NIST Cybersecurity Framework 2.0*. <https://www.nist.gov/cyberframework>

International Organization for Standardization. (2022). *ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. <https://www.iso.org/standard/27001>

International Organization for Standardization. (2022). *ISO/IEC 27002:2022 Information security, cybersecurity and privacy protection — Information security controls*. <https://www.iso.org/standard/75652.html>

Microsoft. *Access tokens in the Microsoft identity platform*. Microsoft Learn. <https://learn.microsoft.com/en-us/entra/identity-platform/access-tokens>

Auth0. *Token Best Practices*. <https://auth0.com/docs/secure/tokens/token-best-practices>

Keycloak. *Securing applications and services guide*. [https://www.keycloak.org/docs/latest/securing\\_apps/](https://www.keycloak.org/docs/latest/securing_apps/)

Keycloak. *Server Administration Guide*. [https://www.keycloak.org/docs/latest/server\\_admin/](https://www.keycloak.org/docs/latest/server_admin/)

Keycloak. *Token exchange*. <https://www.keycloak.org/securing-apps/token-exchange>

## 10. Anexos

Los anexos de esta tesis se referencian como archivos externos del repositorio. No se embeben en el cuerpo del documento para evitar duplicar contenido extenso, mantener control de versiones y permitir que la UI consuma los mismos artefactos que se citan en la tesis.

### 10.1. Artefactos de datos del modelo

- Catálogo maestro JSON
- Datos de prediligenciamiento
- Ejemplo de resultado de evaluación

- Modelo de dominio (Mermaid)

## **10.2. Diagramas de escenarios OAuth 2.0/OIDC**

- Escenario 1 - Despliegue (PlantUML)
- Escenario 1 - Despliegue (PNG)
- Escenario 1 - Secuencia (PlantUML)
- Escenario 1 - Secuencia (PNG)
- Escenario 2 - Despliegue (PlantUML)
- Escenario 2 - Despliegue (PNG)
- Escenario 2 - Secuencia (PlantUML)
- Escenario 2 - Secuencia (PNG)
- Escenario 3 - Despliegue (PlantUML)
- Escenario 3 - Despliegue (PNG)
- Escenario 3 - Secuencia (PlantUML)
- Escenario 3 - Secuencia (PNG)

## **10.3. Diagramas de casos base (contraste conceptual)**

- Caso base 0A - Despliegue (PlantUML)
- Caso base 0A - Despliegue (PNG)
- Caso base 0A - Secuencia (PlantUML)
- Caso base 0A - Secuencia (PNG)
- Caso base 0B - Despliegue (PlantUML)
- Caso base 0B - Despliegue (PNG)
- Caso base 0B - Secuencia (PlantUML)
- Caso base 0B - Secuencia (PNG)

## **10.4. Referencia del entregable web**

- Repositorio del proyecto
- Sitio publicado en GitHub Pages

Los enlaces anteriores apuntan a rutas relativas reales de la versión actual del proyecto (./anexos/\*). Si la estructura de carpetas cambia, esta sección debe actualizarse en la misma revisión del repositorio.